Prague University of Economics and Business

Faculty of Informatics and Statistics

APPLICATION OF PROJECT MANAGEMENT METHODS

BACHELOR THESIS

Study programme: Information Media and Services

Author: Olena Rudnyeva

Bachelor Thesis Supervisor: Mgr. Ing. Viktor Chrobok, Ph.D.

Prague, June 2025

Acknowledgement

I would like to thank Mgr. Ing. Viktor Chrobok, Ph.D., for his professional guidance, valuable advice, and patience. I am also grateful to doc. Ing. Adam Borovička, Ph.D., for his support during the course Introduction to Project Management. Furthermore, I extend my gratitude to the entire academic staff of the university and to my family for their continuous support throughout my study.

Abstract

Effective project scheduling is essential for success and competitive advantage in IT outsourcing. This bachelor's thesis compares two key project management methodologies—the Critical Path Method (CPM) and the Program Evaluation and Review Technique (PERT)—within a real-world software development project context. The research addresses typical project management challenges, including schedule uncertainties and risk management, using Monte Carlo simulations and the Fuzzy Delphi method. Results highlight the strengths and limitations of CPM and PERT, offering valuable insights for IT outsourcing firms to improve project scheduling and decision-making processes. This study contributes to existing project management literature by providing practical evidence of the methodologies' effectiveness in a realistic scenario.

Keywords

Project Management, Critical Path Method (CPM), Program Evaluation and Review Technique (PERT), Monte Carlo Simulation, Fuzzy Delphi Method, IT Outsourcing, Software Development, Risk Management, Greenfield Project, Project Scheduling.

Contents

A List of Figures

A List of Tables

1 Introduction

In the dynamic environment of IT outsourcing, having a clear and believable project timeline is key to gaining clients' trust. While a strong track record and favorable reviews undoubtedly help in attracting business, project managers have more direct control over costs, quality, and time — the three vertices of the project management triangle (or iron triangle). In software development, where delays can shatter marketing plans, postpone product launches, and escalate delivery costs, missed deadlines may lead to monetary losses and damage an organization's reputation. Consequently, effective scheduling becomes a major competitive advantage for IT vendors striving for punctual, transparent delivery.

Starting a greenfield project — building an application entirely from scratch — compounds these challenges, as the development team must navigate every phase of the software development lifecycle (SDLC), from requirements gathering and design to coding, testing, and deployment. Handling each step internally raises the potential for schedule slips, making the choice of a robust scheduling method critical. In this thesis, a real-life greenfield project executed by Limestone Digital s.r.o. in 2024 is examined to assess how well different scheduling approaches function in a practical IT outsourcing context. This project was managed using scientifically grounded project management methods that emphasized a data-driven framework for planning and execution.

Focusing on delivering more accurate and flexible scheduling, the thesis compares two principal methods — Critical Path Method (CPM) and PERT (Program Evaluation and Review Technique) — supported by additional tools like Monte Carlo simulation, Fuzzy Delphi, and Gantt charts. While CPM and PERT form the primary basis of comparison, these complementary methods help address uncertainties, gather expert consensus, and visualize timelines more effectively. By applying them to a real software development scenario, the study investigates their ability to handle risks, maintain reliable schedules, and provide clear insights into deadlines. Ultimately, the goal is to offer IT outsourcing firms, particularly those taking on greenfield projects with fixed scopes, a more solid foundation for managing timelines, mitigating risks, and delivering on client expectations.

2 Theoretical Part

The theoretical part of this bachelor's thesis provides an overview of key concepts and methodological foundations essential for the subsequent analysis of project scheduling techniques. It introduces core principles of project management from the perspective of operations research, with a focus on deterministic and probabilistic methods such as CPM, PERT, and Monte Carlo simulation.

This section outlines the theoretical background necessary for understanding the application of these methods in real-world project environments. All definitions and methodological approaches presented here form the analytical basis for the computations and evaluations carried out in the practical part of the thesis.

2.1 Project Management

A project is defined as a temporary endeavor undertaken to create a unique product, service, or result (PMBOK, 2017). This means that every project has a clearly defined beginning and end, distinguishing it from routine operational work. Moreover, the result of a project is unique — not necessarily globally new, but developed specifically for a given client, organization, or context. Even if similar solutions exist, the way a project adapts and implements them makes the outcome original in its environment. In addition to its temporary and unique nature, a project is also constrained by limited resources such as time, budget, personnel, and technology. These constraints require structured planning and active management to ensure successful delivery. In contrast, non-project work is continuous, repetitive, and focused on maintaining existing processes rather than delivering new value.

Project management is critical to the success of a project. The main professional organizations define it as follows. According to the Association for Project Management (APM, 2025), project management is the application of processes, methods, skills, knowledge, and experience to achieve specific project objectives in accordance with established acceptance criteria and within agreed-upon parameters. Most importantly, project management always has an end result in mind and is subject to time and budget constraints. The Project Management Body of Knowledge (PMBOK), published by the Project Management Institute (PMI, 2021), defines project management as a discipline that encompasses the processes of initiating, planning, executing, controlling, and completing the work of a team to achieve specific objectives and meet established success criteria. This definition emphasizes the phased and comprehensive nature of the project life cycle. For its part, the Project Management Institute defines project management as the systematic and professional application of processes designed to help a team complete a project using available resources. In this context, a project is viewed as a set of tasks or goals whose outcomes lead to specific benefits. A comparative analysis of these definitions allows us to conclude that project management is an integration of knowledge and management tools

that ensures the achievement of unique results under conditions of limited resources and time. While implementing any project management method, it is essential to clarify existing conditions and constraints, as this ensures methodological relevance and supports effective planning, execution, and control. To define project constraints and limitations, the project management triangle—also known as the Iron Triangle or the Triple Constraints model—is commonly applied. According to Microsoft (2021), the triple constraint means that if you change a project's cost, time, or scope, at least one of the other two must change as well. This model presents three key elements of a project: time, cost, and scope, all of which are interdependent and must be balanced throughout the project lifecycle. The triangle is one of the few visual models that clearly demonstrates how changes to any of these constraints can significantly alter the course of a project. It demonstrates that shortening the timeline may require more funding or a reduced scope, while increasing the scope demands either more time or additional resources. This confirms that accurate time planning is not only a technical necessity but one of the critical factors of project success. The following sections will describe the methods and practices that were applied in this work for project time planning.

2.2 Network Planning Technique

Network Planning Technique is a method used to organize, plan, and visualize complex projects in which many tasks have logical and temporal dependencies. As Kerzner (2009) notes, "The primary purpose of network planning is to eliminate the need for crisis management by providing a pictorial representation of the total program". This technique is used to sequence work in a way that minimizes the risk of delays, avoids resource conflicts, and ensures that project goals are achieved within established constraints.

The effectiveness of NPT is explained by its ability to identify the most important processes of a project and coordinate them in time. "It ensures that the most important processes of a project are identified and coordinated with each other in time." (ZEP GmbH, n.d., para. 1), thanks to this approach, the project team can see the overall structure of the project, manage priorities, and respond to deviations from the plan before they lead to delays or budget overruns.

The use of this technique is especially relevant in the field of software development, where many tasks are dependent on each other. For example, it is impossible to test a module until its functionality is fully implemented. In development projects, network diagrams allow you to accurately identify critical areas where even small delays can affect the release of the product on time. Therefore, such methods are widely used in IT projects and in agile and waterfall approaches.

There are two main types of network diagrams: Activity on Arrow (AoA) and Activity on Node (AoN). AoA (Arrow Diagramming Method, ADM) represents each project task as an arrow connecting two nodes - the start and end event. This type of diagram emphasizes the logical sequence of tasks. To reflect dependencies not directly related to the execution of tasks, dummy activities are used that do not require time to complete. Despite a certain cumbersomeness, ADM remains applicable in a number of engineering and construction

projects. AoN (Precedence Diagramming Method, PDM) is more flexible and widely used. Here, each task is represented by a node, and the dependencies between them are represented by arrows. This format allows for easy visualization of the main types of relationships between tasks (e.g., "finish-start") and simplifies adding information about duration, resources, and other parameters directly to the node. Thus, network planning is a key technique in the arsenal of a project manager, especially in conditions of high complexity and interdependence of tasks.

2.3 Critical Path Method (CPM)

Critical Path Method or Critical Path Analysis is one of the most common methods of project planning and scheduling. This method is based on the assessment of the longest sequence of tasks in the project. Before delving into the theoretical part of the method, it is necessary to pay attention to the history of its origin, which will allow us to more accurately assess the appropriateness of using this method. The development of the CPM method took place at the DuPont company, which in the 1950s was an engineering and industrial giant, actively investing in the construction of plants, the development of new technologies and materials. The planning methods that existed at that time did not cover all the needs of advanced engineering projects. Manual planning did not reflect the dependencies between tasks, did not provide a detailed overview of the "time buffers", and there were also difficulties with resource planning within long-term projects. Thus, the main goal was to develop a method that would optimize the approach to planning complex projects that have many tasks and dependencies. The development of CPM began in 1956, and was led by engineers Morgan Walker and James Kelley. The first results were demonstrated as early as 1957. Despite the fact that the first project for testing the method was artificially modeled, it was complex enough to evaluate the effectiveness of the method in conditions as close to reality as possible: A network model of 61 activities, 8 time constraints and 16 fictitious operations was used. As a result, the calculation of deadlines was successfully performed considering resource optimization. Even though the focus of this work is the calculation of the timeline in conditions of sufficient resources, resource optimization is also important for evaluating and comparing deadline estimation methods. CPM was first used on a real project in 1958. The project was dedicated to the construction of hydrogen and ammonia production plants. The plan included 846 operations. Optimization of the plan using CPM made it possible to reduce the project duration by 25% compared to the traditional approach. The project showed that the method works in real business conditions and complex external constraints.

The critical path methodology is based on constructing a topological representation of a project by decomposing it into elementary operational units – tasks. The process begins with defining the scope that needs to be completed within the project, after which the scope is decomposed into individual units of work (tasks) until each of them becomes detailed enough for estimation.

Once the list of tasks has been defined, dependencies are assigned to each task. This is necessary to correctly understand the constraints in the arrangement of task sequences. Within the framework of this stage, predecessors and successors of each task are identified,

i.e. logical dependencies are established that determine the order of tasks in the network schedule. A predecessor is a task whose completion or start is a prerequisite for the start or end of another task; a delay in a predecessor automatically postpones the deadlines of all its immediate successors. A successor, on the contrary, is a task whose execution directly depends on the results of its predecessor: it cannot be started until the relationship specified by the dependency type is satisfied. Thus, the correct definition of the predecessor → successor links forms the structural framework of the project, on the basis of which early and late dates, time reserves and the critical path itself are then calculated. This step is critical not only for planning the time required to complete the project, but also for assessing the required resources and determining risks.

After completing the stages of work breakdown and identifying logical dependencies between tasks, it is necessary to proceed to the graphical representation of the project structure. A network diagram based on the Activity-on-Arrow (AOA) method is constructed, where each activity is represented by a directed arrow, and nodes (events) represent the start or end of an activity. The direction of the arrow indicates the logical sequence of tasks, and the nodes serve as connectors reflecting the completion of one or more operations and the initiation of subsequent ones. This structure enables the clear identification of task dependencies and forms the basis for calculating early and late event times, determining time reserves, and identifying the critical path. The AOA diagram thus provides a visual framework for scheduling analysis and time-based project planning.

Once the dependencies diagram is created, each task must be estimated. The CPM method uses deterministic estimation, which means that for each unit of work, one specific value is determined that is considered the most probable or accurate. The estimate must be unambiguous, specific, and precise. Often, experts in the project domain are involved in the estimation.

Despite the fact that deterministic estimation is one of the most common estimation methods for project planning, it is often criticized. The main comments are the discrepancy between the actual deadlines and the planned ones. For example, Jakob Kaplan Moss (2021), as part of his series on estimating software project timelines, describes a case in which the deterministic estimate was two times less than the actual project deadlines. Moss comments on the deterministic approach: “This estimate turned out to be completely wrong. The project ended up taking a year longer than I originally planned” (Moss, 2021, para. 3). Also, Jufran Helmi, in his article “Comparing Deterministic and Probabilistic Scheduling in Project Management” notes: “Deterministic schedules assume that all estimates are correct, which can create problems when tasks are longer than expected. This overconfidence can lead to schedule slippage when uncertainties arise or when unforeseen risks materialize.” (Helmi, 2024, “Comparing Deterministic and Probabilistic Scheduling in Project Management” section, para. 2)

Despite the criticism of deterministic estimation in project management, the Critical Path Method can be a highly effective tool when properly applied. Numerous studies, including “Application of Project Management Techniques for Timeline and Budgeting Estimates of Startups,” demonstrate that CPM with deterministic estimation can provide high planning accuracy when certain conditions are met.

It is important to note that the effectiveness of deterministic estimation depends on contextual factors. As Helmi (2024) points out, the deterministic approach demonstrates high accuracy in projects with clearly defined boundaries and stable requirements.

In conclusion of the discussion of deterministic estimation, it can be concluded that this estimation is not suitable for projects in which the exact scope that will need to be completed within the project is not clear at the estimation stage, as well as for projects with a large number of unknowns. Since the team of experts has previously worked with projects similar to Sentinel Tenant Portal, and the project scope was fixed at the project discussion stage, it is reasonable to apply the deterministic estimation method to plan this project.

The next stage of the Critical Path Method involves calculating early parameters using the Forward Pass technique. In this thesis, the version of the Critical Path Method proposed by Kerzner is used as the primary analytical framework.

According to Kerzner (2017), "To calculate the earliest starting times, we must make a forward pass through the network (i.e., left to right). The earliest starting time of a successor activity is the latest of the earliest finish dates of the predecessors" (p. 421). This rule is formalized as:

$$ES_j = max\left(EF_{all\ predecessors}\right)$$

Where:

- ES_j — Early Start time for activity j
- $EF_{\text{all predecessors}}$— Early Finish times for all immediate predecessor activities of activity j

Kerzner (2017) also states: "The earliest finishing time is the total of the earliest starting time and the activity duration" (p. 421). This leads to the second formula:

$$EF_j = ES_j + D_j$$

Where:

- EF_j — Earliest Finish time for activity j
- ES_j — Earliest Start time for activity j
- D_j— Duration of activity j

This forward calculation continues through the entire project network and yields the earliest possible schedule from project start to finish.

The next step is the Backward Pass, which is used to determine the latest possible timing of each activity without delaying the overall project. Kerzner (2017) explains: "To calculate the latest times, we must make a backward pass through the network by calculating the latest finish time. Since the activity time is known, the latest starting time can be calculated by subtracting the activity time from the latest finishing time" (p. 421).

The Late Finish of the final activity is assumed to be equal to its Early Finish:

$$LF_j = EF_j$$

Where:

- LF_j — Latest allowable Finish time of activity j
- EF_j — Earliest Finish time of activity j, determined from the forward pass

The Late Start is then calculated as:

$$LS_j = LF_j - D_j$$

Where:

- LS_j — Latest allowable Start time of activity j
- LF_j — Latest Finish time
- D_j— Duration of activity j

Kerzner (2017) also notes: "The latest finishing time for an activity entering a node is the earliest starting time of the activities exiting the node" (p. 422). Based on this, the backward calculation for node-based time is:

$$LS_i = \min(LS_{\text{all successors}} - D_i)$$

Where:

- LS_i — Latest allowable Start time at node i

- $LS_{\text{all successors}}$ — Latest Start times of all successor activities of node i
- D_i — Duration of the operation departing from node i

Finally, the float (or slack) of an activity is calculated to determine how much the activity can be delayed without affecting the total project duration. As Kerzner (2017) summarizes, activities with zero float form the critical path:

$$\text{Float}_j = LS_j - ES_j = LF_j - EF_j$$

Where:

- Float_j — Total float (slack) of activity j
- LS_j , ES_j — Latest and earliest start times
- LF_j, EF_j — Latest and earliest finish times

Activities with zero float constitute the critical path — a sequence of tasks that defines the minimum project duration. Any delay in these activities directly extends the total project time.

In the final phase of the CPM approach, a detailed project schedule is constructed, integrating task dependencies, identifying opportunities for parallelization where path flexibility allows, and aligning the timeline with available resources. Thus, the CPM method clearly shows which tasks may start later and which cannot be shifted without delaying the project, which significantly simplifies the approach to resource planning. Detailed calculations of the CPM method on the Sentinel Tenant Portal project will be given in the practical part of this thesis.

2.4 Program Evaluation and Review Technique (PERT)

The second key method examined within this thesis is the Program Evaluation and Review Technique (PERT).

PERT originated in 1958, specifically developed for project planning within the Polaris submarine ballistic missile program. The methodology gained significant recognition through the publication of project analyses in professional journals such as those from International Business Machines and General Electric. Subsequently, the technique expanded beyond its maritime origins into various industries. As the distinguished scholar S. P. Nikanorov (1963) observed: "No major military or civilian project today can be imagined without the use of this system" (p.4). The methodology's application gradually extended to economic analysis, financial planning, construction management, and software development domains.

The PERT method is focused on analyzing the time parameters of a project and takes into account the uncertainty factor in the duration of activities. The main goal of the PERT method is to determine the expected duration of a project and assess the probability of its completion within the specified timeframe, considering the uncertainty in task times. PERT combines critical path analysis and statistical duration estimates, which allows for more realistic modeling of complex projects. This section will present a detailed explanation of the application of the PERT method.

At the core of PERT lies a stochastic approach where task duration is treated as a random variable due to inherent uncertainty. Rather than using a single deterministic estimate, PERT employs three time estimates: optimistic (best case), most likely, and pessimistic (worst case). This approach accounts for potential variations in actual completion times, reflecting that real time expenditures are unknown in advance and follow statistical distributions.

As Clifford Clark (1962) noted, "The distribution that first comes to the author's mind is the beta distribution" (p. 406), as it can be parametrized using minimum, most likely, and maximum estimates. PERT thus models each task using these three parameters (optimistic a, most likely m, and pessimistic b), capturing the full range of possible durations.

The beta distribution is particularly suitable for modeling task duration uncertainty for several reasons. It is defined on a finite interval [a, b], where a represents the minimum possible completion time and b the maximum possible time. This accurately reflects real-world constraints: tasks cannot be completed faster than some technical minimum and typically have an upper time limit. Additionally, experts can usually identify the most likely duration (m), which corresponds to the distribution's mode. The beta distribution can have a single peak within the interval [a, b], representing situations where one duration is most probable while others are less likely but still possible.

PERT begins with the detailed decomposition of the entire project scope. The decomposition of work is carried out similarly to the critical path method: the project volume is broken down into individual tasks until reaching a level of detail sufficient for estimation. Requirements for work units in PERT are similar to requirements in the critical path method. Decomposition should result in a list of work units that completely cover the scope that needs to be completed for project execution. After the complete decomposition of the project is conducted, dependencies between tasks are determined. Based on the relationships between tasks, a network diagram is constructed. When building the network diagram, the Activity on Arrow approach is used.

After defining the project structure and constructing a network diagram, the next step in applying the method is to estimate the duration of each task, taking into account the uncertainty inherent in the project environment. As mentioned earlier, the PERT method operates with probabilistic estimates, so three values are defined for each activity: the optimistic, most likely, and pessimistic durations. Following Kerzner's interpretation of the classical PERT method, the expected time is calculated using the following expression (Kerzner, 2017, p. 647):

$$t_e = \frac{a + 4m + b}{6}$$

Where:

- a represents the shortest possible time in which the task could be completed, assuming everything proceeds smoothly;
- m is the most probable time estimate, reflecting the normal course of events under typical conditions;
- b indicates the longest likely duration, accounting for potential obstacles or setbacks;
- t_e is the expected time for the activity — a statistical mean that integrates all three scenarios to support more realistic project scheduling.

Based on the calculated values, analogous to the Critical Path Method, a Forward Pass and Backward Pass through the network diagram is performed: using the forward pass, the earliest start and finish dates of tasks are determined, and using the backward pass, the latest start and finish dates are determined. The difference between the late and early start (or finish) dates of a task represents its float/slack time. Tasks with zero float/slack form the project's critical path. Although PERT is probabilistic in nature, the critical path is determined using fixed expected task durations, calculated using the PERT duration formula. As a result, the critical path is identified using a simplified deterministic procedure, based on a single averaged duration estimate instead of accounting for the full range of possible outcomes. Consequently, the critical path reflects the most likely scenario and does not consider potential duration variations that could change its structure in reality. This simplification streamlines the calculations, allowing the use of standard network analysis algorithms.

Once the tasks included in the critical path have been identified, it becomes possible to proceed to the analysis of the project's time risks. As mentioned earlier, the PERT method is based on the probabilistic assessment of task durations, which means that it is necessary to take into account the degree of uncertainty associated with these assessments. For this purpose, the method uses such statistical characteristics as standard deviation and dispersion.

It is assumed that most possible task duration values fall within the interval between the optimistic and pessimistic estimates. By analogy with the normal distribution, where about 99.7% of values are within ±3 standard deviations of the mean, the range between the pessimistic and optimistic estimates $b_{ij} - a_{ij}$ in PERT is assumed to be approximately 6σ. The standard deviation σ, which reflects the uncertainty of each activity duration, is derived from the assumed symmetric β-distribution (Kerzner, 2017, p. 429):

$$\sigma_{t_e} = \frac{b - a}{6}$$

Where:

- σ_{t_e} — the standard deviation of the activity's expected duration. It reflects the level of uncertainty or variability in the time required to complete a task.
- a — the optimistic time estimate, representing the shortest possible duration assuming everything proceeds ideally.
- b — the pessimistic time estimate, representing the longest likely duration under unfavorable but realistic conditions.

Accordingly, the variance — that is, the square of the standard deviation — is calculated using the following formula:

$$\sigma_{t_e}{}^2 = (\frac{b-a}{6})^2$$

The PERT method uses variance to quantify the uncertainty of task duration. It shows how much possible completion dates can deviate from the expected value. The higher the variance, the greater the uncertainty and the higher the risk of missing deadlines. Variance is calculated for all tasks on the critical path. The next step in the PERT method is to estimate the most probable project completion time and to assess the probability of meeting a specific deadline. Since this approach relies on probabilistic estimates of task durations, the total duration of the project along the critical path is treated as a random variable, T, representing the sum of the expected durations of all activities that constitute the critical path. PERT assumes that task durations on the critical path are independent random variables — meaning the variation in one activity does not affect the others — which justifies the use of standard rules for summing expectations and variances.

The expected duration of the project is given by:

$$T = \sum_{(i)\in CP} t_{e_i}$$

Where:

- T — the expected total project duration;
- CP — the set of activities on the critical path, i.e., the longest sequence of dependent tasks that determines the minimum completion time of the project;

- t_{e_i} — the expected duration of activity i, as calculated earlier using the weighted average of the three-point estimates.

This aggregation of expected durations provides a basis for evaluating the overall project timeline under uncertainty and supports subsequent probabilistic risk analysis.

The next step in the PERT method is to assess the degree of uncertainty associated with the project's expected duration. It is necessary to account for the overall variability of the estimates for the tasks on the critical path, which is achieved by calculating the project's variance. The variance of the total project duration, denoted as σ (T), is determined as the sum of the variances of the tasks comprising the critical path.

$$\sigma(T) = \sum_{i \in CP} \sigma_i^2$$

Where

- σ_i^2 — is the variance of the duration of the i-th task;
- CP— is the set of tasks on the critical path.

As stated earlier, task durations are independent of each other. This assumption allows us to apply the rules for adding mathematical expectations and variances adopted in probability theory.

According to the Central Limit Theorem, if the sum of independent random variables has a finite variance, its distribution tends to normal. Thus, the distribution of the total project duration can be approximated by a normal distribution with parameters T and σ(T).

The next step is to determine the probability of completing the project within a specified time frame. According to the definition of a project, every project is finite, meaning it has a defined completion deadline. This deadline — specifically, the number of days within which the project must be completed — is represented by the variable $T_{deadline}$. At this stage, the Z-score is introduced. According to Business and Technology University: "The Z-score corresponds to the number of standard deviations away from the mean." (Business and Technology University, 2023) This means that the Z-score quantitatively reflects how many standard deviations the deadline deviates from the expected average duration. The Z-score is calculated as follows:

$$Z = \frac{T_{deadline} - T}{\sqrt{\sigma(T)}}$$

Where:

- T_{deadline} is a specified planned completion date of the project;
- T is the expected total project duration;
- $\sigma(T)$ is a variance of total project duration.

The next step is to determine the probability of completing the project on time. As stated by Malcolm et al. (1959), the final step of the PERT analysis involves evaluating the cumulative distribution function of the normal approximation to obtain the completion probability:

$$P(T \leq T_{deadline}) = \Phi(Z)$$

Where:

- $P(T \leq T_{\text{deadline}})$ is the probability of meeting the project deadline;
- $\Phi(Z)$ is the value of the cumulative distribution function of the standard normal distribution.

It represents the probability that a standard normal random variable Z is less than or equal to a given value. In the context of project management, this value quantifies the likelihood of completing the project within the specified deadline, expressed as a proportion between 0 and 1.

The Function $\Phi(Z)$ for a given value of Z is determined through the integral of the normal distribution density:

$$\Phi(Z) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{Z} e^{-\frac{t^2}{2}}\, dt$$

However, to simplify calculations in practice, standard normal distribution tables are commonly used, providing pre-calculated values of $\Phi(Z)$ for various Z-scores.

Thus, the methodology for calculating the probability of project completion in the PERT method is based on the transition from individual probabilistic estimates of task duration to a complex statistical model describing the entire duration of the project. Calculating the Z-score and using the standard normal distribution function allows for a quantitative assessment of the risk of missing deadlines, as well as determining the probability of

meeting the established deadline. This approach not only provides the most probable project completion date, but also provides the project manager with a tool for analyzing time risks and making informed decisions.

PERT remains one of the most effective planning tools in modern software development projects, especially in the highly uncertain environment of startups and outsourcing IT companies. For example, Tech Innovator used PERT to plan the development of an AI-based application. Faster Capital in the article “The Impact of PERT on Startup Success: Lessons from the Field” notes, that the team planned each phase of development in detail, considering optimistic, realistic, and pessimistic estimates. As a result, they were able to speed up the product’s time to market and flexibly adjust the scope of work during the project, successfully launching the application before competitors (Faster Capital, 2014).

Despite the experience of the team involved in the Sentinel Tenant Portal project, there are several factors of uncertainty that will be discussed in greater detail in the practical section of this work. For projects of this nature, it is appropriate to apply stochastic methods for time estimation. In particular, the PERT method was specifically developed for projects characterized by high levels of uncertainty. As Passenheim (2010) notes, “PERT is used for projects with high uncertainty and little experience” (p. 28). Recent research supports this idea: “…project evaluation review technique is more effective when the duration of the project is uncertain, while the critical path method is effective when the project’s end time is certain” (Bagshaw, 2021, p. 215). In the context of the project under consideration, such an approach provides significant advantages. In the presence of numerous unknown factors (such as new requirements, failures, technological changes, etc.), the PERT method not only enables the construction of a schedule based on average expected durations but also provides an understanding of the probability of meeting the project deadline.

2.5 Monte Carlo

The Monte Carlo method, or statistical testing method, is based on repeated random experiments and the construction of statistical estimates for the desired values. It arose in the mid-20th century as part of research conducted at the Los Alamos Laboratory (USA) in solving problems in nuclear physics. The name of the method was given in honor of the Monte Carlo casino in Monaco, symbolizing the use of randomness in calculations. Since its inception, the Monte Carlo method has undergone a long development, and today it is used not only in physics, but also in economics, computer science, etc.

The mathematical basis of this method is the law of large numbers, which states: “the larger a sample size is, the more accurate it is regarding the average of the population being measured” (Investopedia, 2024,“Definition” section, para. 1). In other words, the law of large numbers means that the more observations we collect, the closer the average result will be to the true average value for the entire population. The Monte Carlo method, by generating a set of random outcomes and subsequent statistical analysis, allows for the approximate calculation of complex quantities that are inaccessible for precise analytical calculation. For example, multivariate distributions can be estimated by simulating various possible scenarios and averaging the results obtained. In the field of project management,

the Monte Carlo method has proven itself as an effective tool for planning under uncertainty.

As part of the practical section of this thesis, the Monte Carlo method will be used to assess the probability of project completion within the deadline, utilizing the estimates and values obtained through the PERT method. Despite the advantages of PERT (described in the previous section), this method in isolated application has certain limitations. One of the main issues is the use of a series of approximations, some of which are rather rough. As Van Slyke (1963) notes, "The PERT approach makes a number of approximations, some of which are rather gross" (p. 841). In particular, PERT assumes that the critical path remains unchanged, which does not always reflect reality. In the presence of highly variable activities, a shift in the critical path may occur, which PERT, without additional adjustments, is unable to account for.

To illustrate this, consider the following example: Task A may take as little as 2 days (optimistic estimate) or up to 10 days (pessimistic estimate), with the most likely duration being 5 days. Although the calculation of the critical path will be based on the deterministic expected duration of Task A — 5.33 days — under certain risk conditions affecting Task A, its actual duration may almost double (up to 10 days). Such variability in task durations increases the likelihood of a change in the critical path, requiring a more flexible approach than the classical PERT analysis.

This is why it is recommended to use Monte Carlo methods, which eliminate some of the unrealistic assumptions made in classical PERT analysis. As emphasized in the study: Monte Carlo methods avoid some of the unrealistic assumptions embedded in PERT, particularly the assumption that the critical path persists unchanged (Van Slyke, 1963). Moreover, Monte Carlo simulation allows for a more accurate estimation of the probability distribution of project duration: Monte Carlo methods are advised when a precise estimate of the probability distribution of the project duration is needed (Van Slyke, 1963).

Thus, although the PERT methodology remains an important tool in project management, it is recommended to complement it with Monte Carlo simulation to improve the accuracy and reliability of project duration estimates.

2.6 Risks evaluation using Fuzzy Delphi

The Fuzzy Delphi method is an extension of the classical Delphi technique that incorporates fuzzy set theory to manage uncertainty in expert evaluations. According to Barghi and Shadrokh Sikari (2020), "the fuzzy Delphi technique is, in fact, a combination of the Delphi method and the analysis of the collected data using the definitions of the theory of fuzzy sets." (p. 6) In other words, the Fuzzy Delphi approach maintains the iterative feedback mechanism inherent to the Delphi method and enhances it through the application of fuzzy logic. This allows expert opinions to be expressed as linguistic variables (e.g., "low," "medium," or "high" risk), enabling the use of fuzzy numbers when aggregating these judgments. This method is actively employed by Limestone Digital s.r.o., and its

effectiveness has been repeatedly demonstrated through the successful planning and execution of numerous projects. This section will outline the theoretical foundations of the Fuzzy Delphi method.

The initial phase of the methodology involves gathering a group of experts to participate in the risk assessment. Although Adler and Ziglio (1996) argue that the optimal number of experts is between 10 and 15, other researchers adopt a more flexible approach. For example, Ocampo et al. (2018) state that the quality of research results is not always directly related to the number of experts involved, and that methodologically valid research can be conducted even with a limited number of participants. In addition, Sikander Ali (2023) emphasizes that "the number of experts involved can vary depending on a variety of factors, including the complexity of the problem, the diversity of skills required, and the available resources" (para. 1).

In Limestone Digital s.r.o. projects, key experts from the project team participate in risk assessment. This approach ensures their deep understanding of the project details and the high level of professional expertise required for successful completion. The number of experts involved is proportional to the complexity of the project: the greater the project's scale and complexity, the larger the expert team involved. The company's experience shows that involving a project implementation team enhances participant motivation, deepens their domain-specific knowledge and, thus, allows for a more accurate and pragmatic assessment of project risks.

Following the formation of the expert focus group, a survey is conducted. Experts are provided with a questionnaire containing a list of risks or risk-related statements related to the project. Each risk must be evaluated in terms of its degree of impact on the project using linguistic terms on a given scale. Risks are evaluated according to two parameters: Probability and Impact. The Fuzzy Delphi method is frequently applied in scientific studies for expert risk assessment based on Probability and Impact parameters. Thus, the study by Tuni et al. (2023) describe an example of risk assessment: "Probability is defined as the likelihood that a particular risk will occur during a specific time frame…, whereas impact is defined as the severity of the financial effect should the risk occur within the specified time frame"(p.8).

The risk assessment uses a scale from 1 to 5, where each point corresponds to a specific lexical category. The scale is defined in such a way as to reflect the levels of risk severity for project deadlines from minimum to maximum:

Table 1 - Lexical Scale for Assessing Risk Severity

Score	**Lexical Equivalent**
5	High
4	Above Average
3	Average

2	Below Average
1	Low

This gradation allows experts to use clear qualitative definitions of risk, instead of dry numbers. As Tang Tsiao Yin and Hafiz Hanif (2024) explain, a fuzzy scale should employ an odd number of agreement levels — typically 3, 5, or 7 — and assigns each Likert point a fuzzy value between 0 and 1 to quantify the degree of agreement. In our case, a 5-point scale was chosen.

Following the initial assessment has been made, the experts are provided with a summary result, and a further round of questioning is conducted. During this step, the experts have the opportunity to exchange opinions on the risk assessment. This step increases the objectivity of the risk evaluation.

The Fuzzy Delphi method utilizes fuzzy numbers to account for uncertainty inherent in experts' subjective assessments. Expert ratings on a 1–5 scale are converted into triangular fuzzy numbers (TFNs), represented as triples (a, b, c), where a denotes the minimum value, b the most probable value, and c the maximum value of the assessment. This means that, instead of a single fixed value, a range of possible values is considered, with the midpoint of the range regarded as the most likely estimate. This approach reflects the fact that each linguistic evaluation possesses an inherent range of possible interpretations. As noted in the literature, "these fuzzy values provide an inclusive representative and a more precise expression of the experts' opinions" (Mohd Yusoff et al., 2024, p. 4).

Transitioning from a numerical scale to a TFN thus offers an enhanced representation of expert opinions, because recording judgments directly as fuzzy numbers further enriches the method (Roldán López de Hierro et al., 2021). This enables modeling the fuzziness of perceptions along a scale from "Low" to "High" and results in more informative and nuanced evaluations.

Consequently, each risk assessment provided by expert i for risk j is represented as a triangular fuzzy number:

$$\widetilde{r_{ij}} = (a_{ij},\ b_{ij},\ c_{ij})$$

Where:

- a_{ij}, b_{ij}, c_{ij} correspond to the minimum, most probable, and maximum values, respectively, of the risk assessment for risk j made by expert i.

After the collection of evaluations, each expert *i* is assigned a weight w_i, reflecting the degree of their competence and relevance to the evaluated project domain. In cases where all experts possess a similar level of qualification, equal weights may be assigned. However, if differences in experience, position, or the level of involvement in the project are known, it is recommended to apply normalized weights. The normalization process involves adjusting the weights so that their sum equals one. In this study, the assignment of expert weights was carried out based on the internal methodology of Limestone Digital s.r.o., developed through the synthesis of practical experience in the implementation of IT projects in the field of software development. It should be noted that the internal documents outlining the rules for weight distribution are subject to a non-disclosure agreement (NDA) and, therefore, cannot be directly disclosed or reproduced within this work. However, the general principles of assigning weights are described as follows: each expert was assigned a weight depending on his competence in the corresponding risk category. For example, when assessing technical risks, greater weight was assigned to developers, while in the area of organizational risks to project managers. A detailed description of the distribution of weights will be presented in the practical part of this work. The allocation of weights based on the competency principle allows for taking into account the specific knowledge of experts in various aspects of the project, thereby increasing the reliability of the final risk assessment. This practice is consistent with the recommendations of research, which emphasizes that different priorities are assigned to experts according to differences in experts' knowledge structures and domain experience and this improves the accuracy of results." (Lv et al., 2019).

Once the weights have been assigned to each expert and the triangular fuzzy risk evaluations have been obtained, each triangular fuzzy number provided by expert *i* for risk *j* is multiplied by the corresponding expert weight w_i:

$$w_i \times (a_{ij},\ b_{ij},\ c_{ij}) = (w_i \times a_{ij},\ w_i \times b_{ij},\ w_i \times c_{ij})$$

Thus, we consider the fact that the contribution of experts with higher competence has greater significance in the final result. After each triangular fuzzy number provided by expert *i* for risk *j* has been multiplied by its normalized weight w_i, the process of aggregating the results is carried out. Aggregation is performed separately for two characteristics of each risk: Probability and Impact. For each characteristic, all weighted triangular estimates are summed up for the corresponding components.

For Probability:

$$L_j^{(P)} = \sum_i \left(w_i \times a_{ij}^{(P)} \right)$$

$$M_j^{(P)} = \sum_i \left(w_i \times b_{ij}^{(P)} \right)$$

$$U_j^{(P)} = \sum_i \left(w_i \times c_{ij}^{(P)} \right)$$

For Impact:

$$L_j^{(I)} = \sum_i \left(w_i \times a_{ij}^{(I)} \right)$$

$$M_j^{(I)} = \sum_i \left(w_i \times b_{ij}^{(I)} \right)$$

$$U_j^{(I)} = \sum_i \left(w_i \times c_{ij}^{(I)} \right)$$

Where:

- $w_i \times a_{ij}^{(P)}, w_i \times b_{ij}^{(P)}, w_i \times c_{ij}^{(P)}$ - weighted minimum, most probable and maximum values of risk j Probability,
- $w_i \times a_{ij}^{(I)}, w_i \times b_{ij}^{(I)}, w_i \times c_{ij}^{(I)}$ - weighted minimum, most probable and maximum values of risk j Impact.

Thus, for each risk, two aggregated triangular fuzzy numbers are formed:

$$\widetilde{R_j^{(P)}} = \left(L_j^{(P)}, M_j^{(P)}, U_j^{(P)} \right)$$

$$\widetilde{R_j^{(I)}} = \left(L_j^{(I)}, M_j^{(I)}, U_j^{(I)} \right)$$

Where:

- $\widetilde{R_j^{(P)}}$ is an aggregated value for Probability;
- $\widetilde{R_j^{(I)}}$ is an aggregated value for Impact.

The resulting aggregated triangular fuzzy numbers will be used to conduct a Monte Carlo simulation. The simulation process takes into account various scenarios for the development of events based on the variation in the probability and impact of each risk on the project timeline.

2.7 Conclusion of the Theoretical Part

The theoretical part of this thesis systematically examined key approaches to project time management under uncertainty, as well as methods for assessing and modeling project risks. This review provided the necessary theoretical foundation for the practical application of the discussed techniques within the context of the Sentinel Tenant Portal project, developed in 2024 by Limestone Digital.

The first section outlined the conceptual framework of project management. A project was defined as a temporary endeavor undertaken to produce a unique result under constrained resources. Various definitions of project management proposed by leading international organizations were analyzed, with a particular emphasis on the project management triangle: time, cost, and scope. It was highlighted that time management is not only a technical necessity but a critical factor for overall project success, especially under tight delivery schedules.

Subsequently, the Network Planning Technique was examined as a means of visualizing project structure, identifying task dependencies, and isolating critical workflow elements. The discussion covered two main types of network diagrams—Activity on Arrow (AoA) and Activity on Node (AoN)—and emphasized their relevance for IT projects, where high task interdependency requires precise sequencing and control.

The following section focused on the Critical Path Method (CPM), which involves network modeling and deterministic time estimation. The principles of forward and backward passes, calculation of early and late start/finish times, float, and critical path identification were thoroughly reviewed. Despite criticism of deterministic estimation, it was shown that in cases with a fixed scope and an experienced team—as in the Sentinel Tenant Portal project—the CPM method remains a valid and practical tool for baseline scheduling.

The PERT (Program Evaluation and Review Technique) method was then analyzed, offering a probabilistic approach to activity duration estimation by incorporating three-point estimates (optimistic, most likely, pessimistic). The thesis described the formulas for calculating expected duration, standard deviation, and variance, as well as the use of Z-scores to determine the probability of completing the project by a given deadline. It was noted that PERT extends CPM by explicitly modeling uncertainty, although it assumes a fixed critical path, which simplifies but also limits its accuracy.

The next section introduced Monte Carlo simulation as a natural extension of PERT, addressing several of its limitations. Through iterative random sampling, this method accounts for shifting critical paths and nonlinear variations in task durations, allowing for

more accurate estimates of overall project completion probability. Its suitability for complex IT projects requiring robust risk-adjusted forecasting was substantiated.

Finally, the Fuzzy Delphi method was presented as a qualitative risk assessment tool that applies fuzzy logic to expert evaluations. The methodology includes forming an expert panel, conducting iterative surveys using linguistic scales, converting responses into triangular fuzzy numbers (TFNs), and aggregating results with weighted coefficients reflecting domain expertise. The aggregated risk probabilities and impacts serve as input parameters for Monte Carlo simulations, enabling comprehensive risk modeling.

In summary, this theoretical part demonstrated the evolution of time management methodologies—from deterministic (CPM) to probabilistic (PERT) and, ultimately, to stochastic simulation (Monte Carlo) enriched by expert-driven qualitative analysis (Fuzzy Delphi). This progression established a coherent methodological framework that will be employed in the practical part of the thesis to conduct a comparative analysis of the accuracy, reliability, and resilience of the selected planning approaches in a real-world IT project environment.

3 Practical Part

The practical part of this thesis focuses on the analysis and comparison of project planning methods within the framework of a real greenfield project — the development of the Sentinel Tenant Portal system, completed by Limestone Digital s.r.o. in 2024. This project represents an independent software development effort launched from scratch, in which a functionally rich application had to be delivered within strictly defined time constraints.

The objective of the practical part is to determine which of the planning methods — Critical Path Method (CPM) or Program Evaluation and Review Technique (PERT) — ,provides a more accurate and flexible estimation of project timelines, more effectively accounts for uncertainties, and contributes more significantly to the successful achievement of project goals within the specified timeframe.

The Critical Path Method (CPM) demonstrates the highest effectiveness in scenarios where the project comprises a clearly defined set of tasks and deterministic time estimates. It enables the identification of the sequence of activities that directly determines the project's completion time, thereby allowing project management to focus on the most critical stages. As noted by Levy et al. (1963), it focuses attention on the limited subset of tasks that are critical to project completion time, thus contributing to more reliable planning and tighter control. Accordingly, CPM facilitates adherence to project deadlines by concentrating efforts on the tasks that have the greatest impact on the overall project duration. The application of CPM in the context of the present project is justified for several reasons. Firstly, the project team had relevant experience and had previously implemented similar solutions using comparable technologies. Secondly, a complete list of tasks was developed and thoroughly agreed upon before the project commenced, ensuring a high degree of certainty. As noted by Holistique Training (2024), "CPM is typically used when activity durations are known and can be estimated with a high degree of certainty [...] CPM's structured nature makes it more effective for projects with fixed timelines and clearly defined tasks" (para. 5). Thus, the combination of task predictability, team stability, and a clearly defined scope of work makes the application of the CPM method a well-founded and effective choice for constructing the project's baseline schedule.

In contrast, the PERT method represents an extension of CPM that incorporates probabilistic estimates of activity durations. Samani (n.d.) notes that "Program Evaluation Review Technique (PERT) is a probabilistic version of CPM that considers uncertainties in task durations" ("Scheduling Techniques and Tools" section, para. 4). PERT accounts for potential variations in task length and levels of uncertainty. Although the current project is governed by fixed deadlines and a predefined task list, several functions are new to the team, and certain project risks were identified in advance. In such circumstances, as Lewis (2017) notes, "The Program Evaluation and Review Technique (PERT) encapsulates an analysis technique [...] to reduce inaccuracies in estimates and attempt to account for uncertainty in a project's schedule" ("What is PERT?" section, para. 1). Applying PERT enables a more reliable project plan by modeling likely deviations from base estimates. A further advantage

of using PERT is the possibility of conducting statistical analysis, including Monte Carlo simulation. This approach makes it possible to construct a probabilistic distribution of the project's total duration and to assess the likelihood of completing the project by a specified deadline — an especially critical factor in projects bound to external events or contractual commitments.

In the practical part of this thesis, several steps will be carried out to enable a comprehensive analysis. The practical part begins with a description of the project itself, including its objectives, business rationale, and the key functional components of the Tenant Portal. This step is essential to ensure a comprehensive understanding of the development context; without analyzing the initial conditions and requirements, it would be impossible to apply planning methods accurately. Moreover, understanding the business objectives helps identify the project's priorities and potential critical areas. The next stage involves applying the CPM and PERT methods based on the initial data to construct and analyze the project schedule. The final section presents a comparative analysis of the effectiveness of each approach and formulates well-reasoned conclusions regarding the applicability of these methods in software development contexts. This concluding phase not only highlights the differences in outcomes but also offers recommendations for selecting a planning method in similar projects. Such insights enhance the practical relevance of the work and make its findings applicable to real-world project management decisions.

3.1 Business Background

The project customer is developing self-storage facility management solutions, targeting small operators who are suffering from industry consolidation, rising prices and technological stagnation. Sentinel Systems has created a new generation of software with the aim of "simplifying the management of self-storage facilities, making booking, payment and security easier than ever before" (Sentinel Systems, 2024 , para. 2). The application is designed to provide affordable, high-quality and innovative solutions for effective business development in the face of increasing competition and changing market environment.

3.2 Justification for Developing a New Solution

When starting to develop a new application, the business client used a proprietary application developed by its internal team in 2007. Due to resource constraints, outdated technologies were used during the development phase, which eventually led to serious issues. The application was incompatible with modern systems and could not be effectively integrated with new technologies, which significantly limited its scalability and ability to adapt to changing business needs. There were also serious security issues: the application did not meet modern data protection standards, which increased the risk of leakage of customer and property information. An additional factor was the unsatisfactory user experience: tenants often complained about the lack of responsiveness on mobile devices, complex navigation, and regular failures in the payment, rental, and invoicing processes.

All these problems led to the application no longer meeting both the client's business needs and the expectations of end users.

As a result of a joint analysis conducted by the client's team and Limestone Digital s.r.o., it was determined that developing a new application from scratch would be the more cost-effective solution. At the same time, the existence of the legacy application significantly simplified the process of gathering business requirements: the new system was designed based on it, incorporating clarified and expanded requirements that align with current technological standards and user expectations.

3.3 Project environment: team resources and contractual frameworks

To implement the Sentinel Tenant Portal development project, a dedicated project team was assembled, selected based on the required competencies and the specifics of the business domain. The team consisted of five full-stack development specialists who also possessed infrastructure engineering (DevOps) expertise, ensuring the integration of development and support processes throughout all project phases. In addition to the developers, the team included two test engineers responsible for automation and functional testing, two UX/UI designers tasked with developing the user interface and ensuring adherence to usability standards, and a project manager overseeing planning, coordination, change management, and stakeholder engagement.

The project was executed under contractual conditions that stipulated a fixed scope of work (scope freeze) during the initial iteration phase. All functional change requests submitted after the start of development were processed through a formal change request procedure, followed by a reassessment of the project schedule and budget. According to the terms of the agreement, the risk of any potential increase in project duration or costs due to evolving requirements was fully assumed by the client.

3.4 New Version of the Application

Based on the analysis of the existing system and the identified limitations, it was decided to develop a new application from scratch. This allowed using modern technologies to improve performance, scalability and integration with current tools. The new solution meets modern information security requirements by using advanced data encryption, multi-factor authentication and a monitoring system for timely threat detection. Particular attention was given to improving the user experience: a fully responsive interface was implemented, navigation was simplified, and payment processes were optimized. To enhance business process efficiency, automated features were introduced, including invoice and report generation, as well as real-time data updates. Thanks to its modular architecture, the new system is designed for further functional expansion and adaptation to the global market. The new application addressed the shortcomings of the previous version, delivered a higher

level of user satisfaction, and laid the foundation for sustainable business growth in the context of digital transformation.

The developed application is a comprehensive information system consisting of several functional subsystems that cover core business processes, technical operations, and user interaction with the platform.

First and foremost, the application provides access management and user information security through secure registration procedures, two-factor authentication, and password recovery and update functions. Special attention is paid to the secure storage of personal data and minimizing the risks of unauthorized access.

In addition, the application supports user profile management, allowing for the creation, editing, and personalization of user data and settings. These capabilities provide users with convenience and a personalized experience when interacting with the system.

Furthermore, a financial and payment subsystem has been implemented, including integration with payment services, display and detailed processing of payment history with search and filtering options, as well as the automated generation of payment documents in PDF format. This significantly simplifies document flow and enhances transparency in financial operations.

The application also features a dedicated rental unit management subsystem, providing a clear visual representation of units, their statuses, and key characteristics through the use of visual indicators. Users have access to extended search and filter functionality, as well as detailed information on each unit, including related data on insurance and secondary tenants.

Additionally, functionality for managing secondary users has been implemented, enabling the creation and editing of secondary tenant records and providing timely notifications about changes or new events through automated messages. This facilitates administration and improves communication between users.

The application also includes remote physical access management features, particularly for controlling and monitoring devices such as gates. Users can send remote commands to open or close gates or activate alarms, as well as monitor equipment status in real time.

Finally, all functional capabilities are unified within a single central user interface that provides quick access to key operations, displays important notifications and data, and simplifies the execution of frequent actions such as payment operations or the management of units and access.

From a technical perspective, the application is characterized by a high level of integration with external APIs, strict information security requirements, and extensive automated regression and security testing. Key factors in project timeline and risk assessment include potential changes or disruptions in third-party services, the need for regular updates of external libraries and technologies, unplanned changes in architecture and integration requirements, and constraints related to hardware or development team resources.

3.5 Application of Project Management Methods

This section introduces the analytical phase of the project, which focuses on the practical application of network-based project management methods, specifically the Critical Path Method (CPM) and the Program Evaluation and Review Technique (PERT). Both methods have been thoroughly discussed in the theoretical part of this thesis. The objective of this stage is to construct a realistic and data-driven project schedule for the development of the Sentinel Tenant Portal, taking into account task durations, logical dependencies, and predefined deadlines.

To enable the use of these methods, it was necessary to establish a clear set of initial parameters, including the project start and end dates, as well as duration estimates for individual tasks. These values serve as essential input for network diagram construction and the subsequent application of time analysis models.

The project's required completion date was internally defined as 9 September 2024. This deadline was not only critical to allow sufficient time for internal testing and client-side preparations but also strategically set ahead of a trade show scheduled for 30 September 2024, where the final product was expected to be presented. Meeting the earlier internal deadline was essential for ensuring that all deliverables could be fully reviewed and approved in advance of the public demonstration.

Although initial discussions concerning project requirements began on 19 April 2024, the official project start date for design, development, and testing was defined as 4 June 2024. This date marks the beginning of the project implementation phase, which is the focus of the subsequent calculations.

The scope of the project, for the purpose of this analysis, is limited to the first phase of development. This phase includes only the features classified under the Minimum Viable Product (MVP), as outlined in the application description section. The feature scope is fixed, and no additional functionalities scheduled for later phases are considered in this part of the thesis.

Task durations were estimated by senior professionals specializing in software design, development, and quality assurance. These estimates are based on domain expertise and practical experience and are expressed in human-days. They serve as the foundation for both CPM and PERT calculations and are presented in the respective analytical sections of the thesis.

Prior to applying these scheduling methods, a detailed task decomposition was conducted to break the project into manageable, logically ordered activities. This step is essential for identifying dependencies, constructing the network diagram, and enabling accurate planning. The resulting task list forms the basis for all subsequent calculations and will be outlined in the next section.

All project scheduling and resource planning are based on working days as defined by the official production calendar of the Czech Republic for 2024. This includes recognition of weekends and public holidays. Within the timeline considered for this project, the only

national public holiday falling on a weekday is Friday, 5 July 2024, which commemorates Saints Cyril and Methodius Day. This non-working day is fully accounted for in the project timeline calculations.

3.6 Network Diagram Creation

As part of the comprehensive project management methodology, a strictly structured brainstorming process was implemented to decompose the entire project scope into distinct, measurable tasks.

To conduct these sessions, an interdisciplinary team was assembled, including full-stack developers, testers, and designers directly involved in the project implementation. This combination of competencies ensured a diversity of professional perspectives and contributed to a thorough analysis.

The process unfolded in two key stages. In the first stage, extended brainstorming sessions were conducted to generate a comprehensive list of tasks. In the second stage, the logical dependencies between tasks were analyzed, allowing for the construction of task sequences and the identification of potential bottlenecks.

The author of this thesis acted as a facilitator, ensuring the proactive involvement of all specialists, which played a crucial role in producing an accurate and complete task list and, consequently, increased the reliability of the project plan.

The resulting tasks were subsequently organized into epic blocks in accordance with the standards established by Limestone Digital s.r.o. Although these epics are not examined in detail in the subsequent analysis, their structural inclusion contributed to the integrity of the task architecture and established a solid foundation for effective management and prioritization.

The result of a decomposition session is represented in Appendix A.

In the subsequent phase, task dependencies were identified by assigning preceding activities to each task. This allowed for the establishment of a coherent execution sequence and the detection of tasks that can be performed in parallel or require specific ordering. The resulting dependency structure forms the basis for further analysis in this thesis and will be used in later stages to support project planning and scheduling optimization. Appendix B presents the task IDs, names, and corresponding preceding activities in a structured format.

Using the table that outlines all project activities (presented in Appendix B), a network diagram was created to map the dependencies between tasks. The initial version of the diagram included all fictitious activities and intermediate nodes to ensure accurate representation of the project's structure. The initial version of diagram is represented in Figure 1.

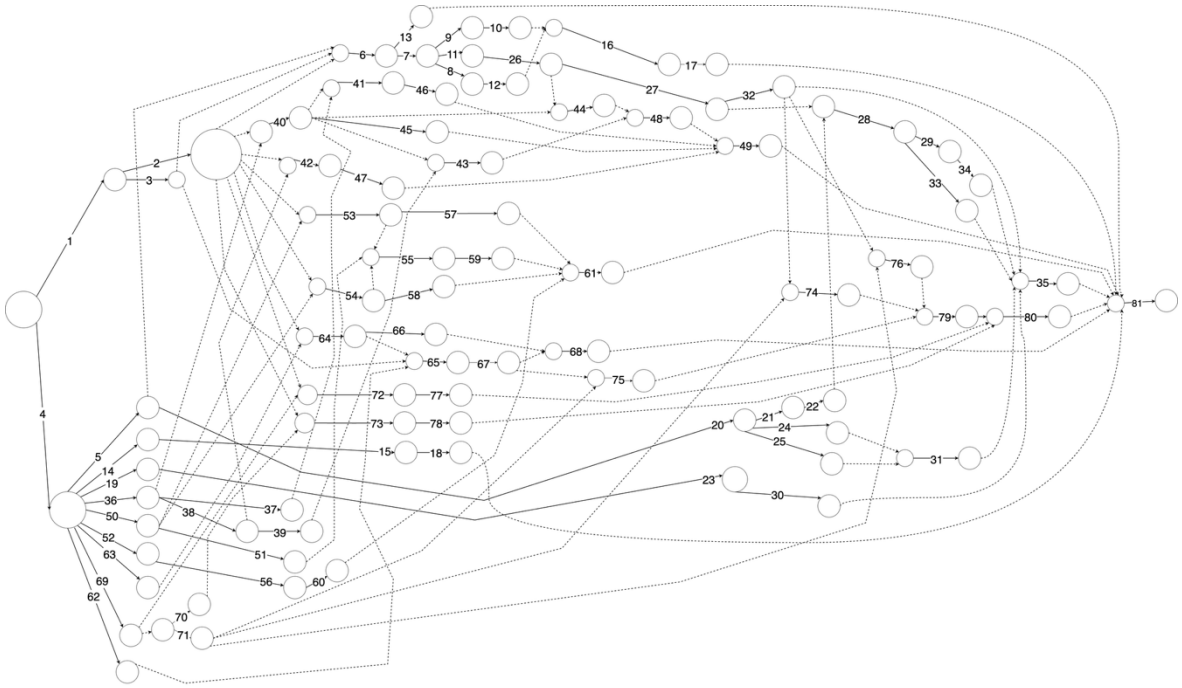


Figure 1 – Network Diagram with Dummy activities (own work)

After eliminating redundant dummy activities and intermediate nodes, the graph now comprises 86 nodes and 114 activities, including 33 dummy activities. The final diagram is represented in Figure 2.

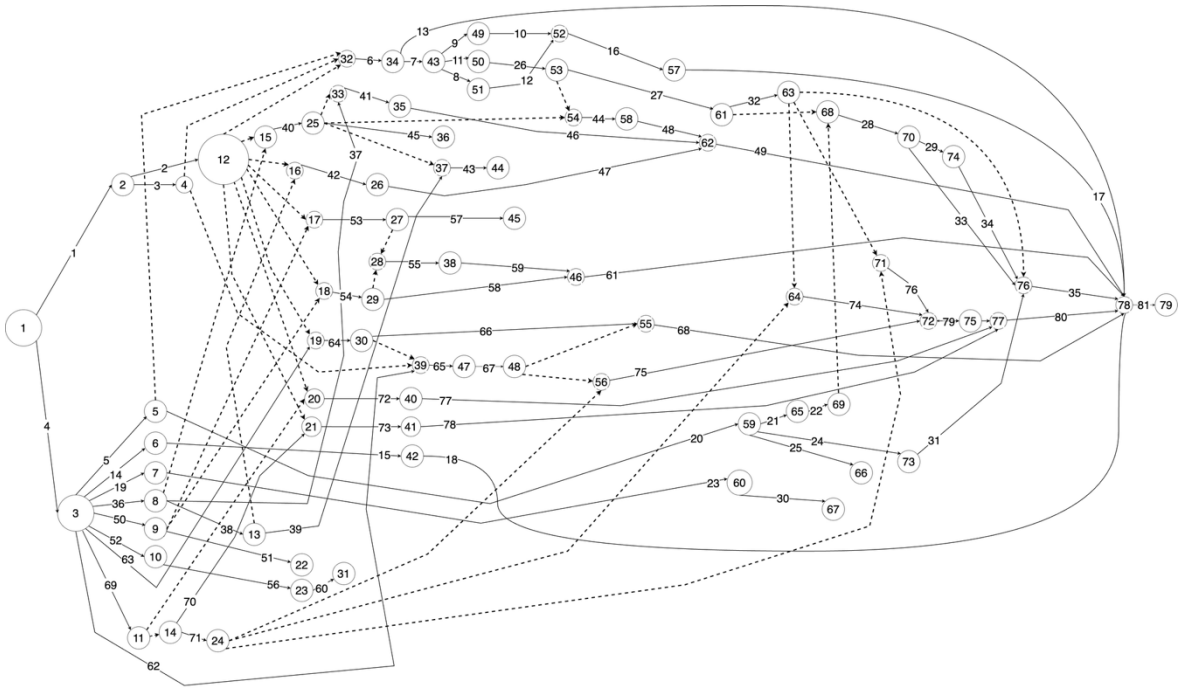


Figure 2 - Network Diagram (Final Version)

The network graph serves as a basis for the application of project management methods, where the thesis presents the CPM, PERT methods and a model of deeper probabilistic analysis of project activities.

3.7 Application of Critical Path Method (CPM)

This chapter is devoted to the application of the critical path method to the Sentinel Tenant Portal project, which is the object of study within the framework of this thesis. The theoretical foundations of CPM, including its principles and calculation methods, were considered in detail in the theoretical part of the thesis. The calculations are based on the network diagram formed in the previous chapter. The CPM method is especially effective in cases where the project has been previously subjected to detailed analysis, and the task duration estimates are accurate and reliable. According to Kerzner (2017), activity time estimates must be as realistic as possible, since they form the basis for determining the critical path and total project duration. Thus, the involvement of experienced specialists who have previously participated in similar projects justifies the validity of applying this method to the Sentinel Tenant Portal project.

The task duration was estimated using an expert approach. The experts were developers, testers, and designers who were actively involved in the project decomposition. Due to their professional experience and direct participation in the preliminary analysis and structuring of the work, these specialists had a deep understanding of the content and scope of the upcoming tasks. This made it possible to form reasonable and realistic duration estimates used in this analysis.

The total duration of the tasks presented in Appendix C is 182 person-days.

At the next stage of applying the critical path method, the early time parameters for completing tasks were calculated: the early start date and the early finish date. The calculations were based on the logical structure of the project formed at the previous stage, where dependencies were defined for each task.

Early parameters are calculated using the Forward Pass method. For the initial task of the project, the early start time is taken to be zero. For all other tasks, it is defined as the maximum value of the early finish time of all preceding tasks:

$$ES_j = max\left(EF_{all\ predecessors}\right)$$

Where:

- ES_j — Early Start time for activity j
- $EF_{\text{all predecessors}}$— Early Finish times for all immediate predecessor activities of activity j

The task's early completion is then calculated by adding the early start and the duration:

$$EF_j = ES_j + D_j$$

Where:

- EF_j — Earliest Finish time for activity j
- ES_j — Earliest Start time for activity j
- D_j— Duration of activity j

The forward pass algorithm is applied sequentially—from the initial task to the final one—ensuring that all logical dependencies are met and allowing the minimum possible project duration to be determined under the given conditions. The application of the method began with task 1, which has no predecessors. Since it represents the starting point of the project, its early start date is 0. According to the duration table data (see Appendix C), the task duration is 7 working days, respectively:

$$ES_1 = 0, EF_1 = 0 + 7 = 7$$

The final task of the project is task 81, which depends on eight predecessor tasks: 80, 68, 61, 49, 35, 17, 18, and 13. When calculating the early completion of task 81, the *EF* values of all its predecessors are analyzed. The maximum *EF* value of 42 days belongs to task 35. Given that the duration of task 81 is 3 days:

$$EF_{81} = 42 + 3 = 45$$

The remaining tasks have lower EF_j values and therefore do not influence the calculation.

The calculation results showed that, in the absence of delays and resource constraints, the project can be completed in 45 working days. Assuming the project starts on June 4, 2024, taking into account weekends and public holidays in the Czech Republic, the project is expected to be completed on August 6, 2024. The calculation results are shown in Appendix D.

The second phase of the critical path method (CPM) application is to determine the tolerable late start and late finish values for all project tasks. These calculations are performed using the Backward Pass Analysis method, in reverse order—from the final task to the starting node. The goal of this phase is to establish deadlines for each task that will keep the overall project schedule unchanged.

The calculation begins with the final task of the project — task 81 — which marks the end of all project work. Its early finish time, determined in the previous step, is 45 working days, and its duration is 3 days. Therefore, its latest finish time is also set to 45. As explained in the theoretical section (see the formulas for calculating late start), the latest start of an activity is obtained by subtracting its duration from its latest finish:

$$LS_{81} = LF_{81} - D_{81} = 45 - 3 = 42$$

Where:

- LS_{81} is the latest start time of task 81,
- LF_{81} is the latest finish time of task 81,
- D_{81} is the duration of task 81.

The same logic applies to the remaining tasks. If a task has only one immediate successor, its latest finish corresponds directly to the latest start of that successor. In cases where multiple successors follow a task, the minimum of their latest start times is used to ensure that none of the dependent activities are delayed. As outlined in the theoretical section (see the logic for backward pass calculations), the latest finish time of a task is determined by the earliest latest start among all its direct successors:

$$LF_j = min(LS_{all\ successors})$$

Where:

- LF_j is the latest finish time of task j,
- $LS_{\text{all successors}}$ are the latest start times of each activity that directly follows task j.

For example, task 32 has three successors: tasks 74, 76, and 35, with latest start times of 38, 38, and 39 respectively. Therefore, its latest finish is calculated as:

$$LF_{32} = min(38,38,39) = 38$$

Given that its duration is 1 day, the latest start is:

$$LS_{32} = LF_{32} - D_{32} = 38 - 1 = 37$$

This means task 32 must be completed by day 38 and started no later than day 37 to prevent delays in subsequent activities. All remaining tasks were analyzed using the same backward pass method, with the results summarized in Appendix E.

The third phase of the critical path method involves calculating the float, also known as slack. This metric represents how many days a specific task can be delayed without disrupting the overall project schedule. The calculations are based on the results of the forward and backward passes, during which the early and late time parameters of all tasks were determined: the early start date, the late finish date, and the task duration. As was stated in theoretical part of this thesis, float (or slack) can be determined as the difference between the latest and earliest start or finish times of a given activity:

$$Float_j = LS_j - ES_j = LF_j - EF_j$$

Where:

- $Float_j$ - the total float (or slack) of the activity
- LS_j - Latest Start time of activity j
- ES_j - Earliest Start time of activity j
- LF_j - Latest Finish time of activity j
- EF_j - Earliest Finish time of activity j

In cases where $Float_j = 0$, the task is considered critical, since any deviation from the schedule will lead to a shift in the project completion dates. In contrast, a positive float value indicates that there is a time tolerance within which the task can be delayed without violating the project plan.

The results of the calculations of the time reserve for all tasks will be presented in Appendix F. Based on these data, the critical path of the project is established - a sequence of operations that do not have a time reserve and, therefore, strictly determine the minimum duration of the project. Determining the critical path allows to focus on those tasks that require the greatest control and do not allow the slightest delays.

Thus, the project's critical path includes the following tasks: 1; 2; 6; 7; 11; 26; 27; 28; 29; 34; 35; 81. Based on the conducted analysis, which included the calculation of time parameters for each task and the identification of the critical path, a project schedule was developed. In its preparation, both the logical dependencies between tasks and the availability of the necessary resources were considered. All planned operations, including task names, assigned executors, start dates, and end dates, are presented in Appendix G.

As a result of project planning based on the duration calculation for each task and the determination of the critical path using total float analysis, the project was scheduled to be completed by August 6.

3.8 Application of Program Evaluation and Review Technique (PERT)

This section of the practical part examines the application of PERT for estimating the duration of project tasks. All steps presented below are based on the theoretical foundations outlined in the corresponding section of the thesis and are implemented in accordance with the methodological principles of stochastic project planning. The PERT method is used in situations where it is not possible to determine a fixed task duration with high accuracy, as the project environment is characterized by uncertainty and the influence of multiple factors.

For the purposes of this stage, an expert session was conducted involving specialists who had previously participated in task estimation under the Critical Path Method — developers, testers, and designers. During this session, each task was evaluated from three perspectives: the optimistic estimate reflects the minimum possible duration of the task in the absence of delays, the most likely estimate corresponds to the most realistic scenario, and the pessimistic estimate accounts for potential complications and the maximum time required for completion.

According to the PERT methodology described by Kerzner (2009, p. 513), the expected task duration is calculated as a weighted average, placing greater emphasis on the most likely estimate:

$$t_e = \frac{a + 4m + b}{6}$$

Where:

- t_e — expected time to complete the task;
- a — optimistic time estimate (minimum duration assuming no problems);
- m — most likely time estimate (based on normal working conditions);
- b — pessimistic time estimate (maximum duration assuming delays and risks).

This approach reflects the assumption that the task duration is most likely to be close to the most likely estimate, but may vary in either direction. The results of the calculations are presented in Appendix H.

Similarly to the Critical Path Method (CPM), the Early Start, Early Finish, Late Start, and Late Finish values were determined based on the expected task durations calculated using the PERT formula. These temporal parameters helped define the logical sequence of

operations and assess the flexibility in task execution. All results are presented in Appendix I.

Thus, according to the planning results, the project will take 47 working man-days.

Similar to the approach used in the critical path method, a Total Float was calculated for each task based on the early start and late finish values determined earlier. As stated earlier, this metric allows one to determine how many days a task can be delayed without affecting the overall project duration. All calculated float values are listed in Appendix J.

As a result of the total float calculation, the project's critical path was identified, including the following tasks: 1; 2; 6; 7; 11; 26; 27; 28; 29; 34; 35; 81. Yet because PERT treats activity durations as probabilistic values, variability can cause an alternative path with higher overall variance to become critical.

After identifying the critical path, which consists of tasks with zero float, an additional quantitative assessment of the uncertainty in the duration of these tasks was performed. Within the PERT methodology, such analysis is conducted using the standard deviation, which allows the estimation of the potential deviation of the actual duration from the expected value. The standard deviation is calculated only for tasks on the critical path, as these tasks directly determine the total project duration, and any uncertainty in their execution may impact the overall project schedule.

According to Kerzner (2009, p. 513), the standard deviation in PERT is calculated using the following formula:

$$\sigma = \frac{b - a}{6}$$

Where:

- σ - standard deviation of the task duration
- b – pessimistic estimate
- a - optimistic estimate

This formula is based on the assumption that task duration values are distributed approximately symmetrically within the interval between optimistic and pessimistic estimates. This approach allows us to approximate this distribution using a normal law and use the resulting standard deviation for further analysis. In particular, it serves as a basis for calculating the probability of completing a project on time and assessing the reliability of a project schedule under uncertainty.

Standard deviation values were calculated exclusively for the tasks included in the critical path and are summarized in Table 2. These results provide a quantitative basis for assessing the risk of time deviations in the project's critical areas.

Table 2- Standard Deviation Calculations

ID	Task Name	Standard deviation (Critical path tasks)
1	Set up application infrastructure	1,00
2	Database Design and Management	1,00
6	User registration with secure password storage	0,25
7	User Login	0,25
11	Test User Login	0,03
26	Integrate payment gateway	0,11
27	Unit balances Payment Flow	0,25
28	Transaction Details	0,25
29	Downloadable receipt feature (PDF)	0,11
34	Test downloadable receipt feature	0,03
35	Smoke, regression, and security testing for Account and Payment Management	0,11
74	Quick Action - "Pay Balance"	0,11
76	Quick Action - "Rent a Unit"	0,11
79	Test Quick action buttons ("Pay Balance," "Open Gate," "Rent a Unit")	0,11
80	Regression testing for dashboard functionalities.	0,11
81	Regression testing	0,25

Based on the above analysis, a detailed project plan was drawn up taking into account the available resources. Appendix K shows the results of detailed planning – the task, the exact date and completion, as well as the specialist who will perform the task in the specified time period.

Based on the calculations carried out using the PERT method, the total duration of the project is estimated at 47 working days. This approach accounts for uncertainties in

individual task duration estimates and enhances the overall accuracy of project planning. Accordingly, if the project begins on June 4, 2024, its completion is expected by August 9, 2024.

At the next stage, the PERT method enables a probabilistic analysis to evaluate the likelihood of completing the project within the defined timeframe, as well as to determine the probability of meeting specific time constraints. These calculations provide insights into deadline adherence and help establish the required timeframes at various confidence levels.

The official start date of the project is June 4, 2024, and the final deadline is set for September 9, 2024. Taking into account weekends and public holidays, the total available time for project execution is 69 working days. Since the critical path defines the minimum possible duration of the project, its length must be the focus of the probabilistic analysis to determine whether the available time is sufficient to meet the deadline.

The expected duration of each task was previously calculated using the PERT weighted average formula:

$$t_e = \frac{a + 4m + b}{6}$$

Where:

- t_e — expected time to complete the task;
- a — optimistic time estimate
- m — most likely time estimate
- b — pessimistic time estimate

The expected durations of tasks on the critical path, based on this formula, are as follows: Set up application infrastructure – 7.33 human-days, Database Design and Management – 8.50, User Registration with Secure Passwords – 3.17, User Login – 2.17, Test User Login – 1.17, Integrate Payment Gateway – 6.33, Unit Balances Payment Flow – 5.83, Transaction Details – 2.83, Downloadable Receipt Feature (PDF) – 2.00, Test Downloadable Receipt Feature – 1.17, Smoke, Regression, and Security Testing – 3.00, and Regression Testing – 3.50 human-days. Since these tasks determine the minimum possible duration of the project, their expected durations play a crucial role in the probabilistic analysis.

The total expected project duration is obtained by summing the expected durations of all tasks in the critical path. According to Kerzner, the total expected time for project completion is calculated as the sum of the expected times t_e of all activities along the critical path:

$$T_{cp} = \sum t_e$$

Where:

- T_{cp} - total expected duration of the project (critical path)
- t_e - expected duration of each activity along the critical path

Substituting the values:

$$T_{cp} = 7.33+8.50+3.17+2.17+1.17+6.33+5.83+2.83+2.00+1.17+3.00+3.50 =$$

$$= 47.00 \text{ human-days}$$

Thus, the expected total duration of the project is 47.00 human-days. This value represents the most probable completion time for all critical tasks, assuming no additional delays or unforeseen circumstances. Since the available working time for project completion is 69 working days, and the expected duration is 47.00 human-days, a preliminary assessment suggests that the project should be completed on time. However, due to potential variability in task durations, it is essential to analyze the standard deviation of the critical path, which will allow for probability estimation regarding project completion within the available timeframe.

The standard deviation is a statistical measure that quantifies the level of variability or uncertainty in a dataset. In PERT analysis, it represents the expected variation in task duration estimates. A higher standard deviation indicates greater uncertainty, meaning the task duration is more unpredictable, while a lower standard deviation suggests that the estimated duration is more reliable. By calculating the standard deviation for the critical path, we can assess how much the project duration might deviate from its expected value and use this information to estimate the probability of completing the project within the available timeframe.

The standard deviation for each task is determined using the formula:

$$\sigma = \frac{b - a}{6}$$

Where:

- σ - standard deviation of the task duration
- b – pessimistic estimate
- a - optimistic estimate

For example, for the task "Set up application infrastructure", with an optimistic estimate of a=5 days and a pessimistic estimate of b=11 days, the standard deviation is calculated as follows:

$$\sigma_1 = \frac{11 - 5}{6} = 1$$

Using the same approach, the standard deviation was calculated for all other tasks on the critical path:

Table 3- Standard Deviation of Task Durations Along the Critical Path

ID	Task Name	Human days	Standard deviation (Critical path tasks)
1	Set up application infrastructure	7,33	1,00
2	Database Design and Management	8,50	1,00
6	User registration with secure password storage	3,17	0,25
7	User Login	2,17	0,25
11	Test User Login	1,17	0,03
26	Integrate payment gateway	6,33	0,11
27	Unit balances Payment Flow	5,83	0,25
28	Transaction Details	2,83	0,25
29	Downloadable receipt feature (PDF)	2,00	0,11
34	Test downloadable receipt feature	1,17	0,03
35	Smoke, regression, and security testing for Account and Payment Management	3,00	0,11
81	Regression testing	3,50	0,25

To evaluate the overall uncertainty of the project's completion time, we need to consider the combined variability of all critical path tasks. Since task durations are assumed to be

independent, the total variance of the project duration is obtained by summing the variances of individual tasks.

The variance is simply the square of the standard deviation, and the total variance is calculated as follows:

$$\sigma_{cp}^2 = \Sigma\sigma_i^2$$

Substituting the computed variances for all critical path tasks:

$$\sigma_{cp}^2 = 1.00^2 + 1.00^2 + 0.25^2 + 0.25^2 + 0.03^2 + 0.11^2 + 0.25^2 + 0.25^2 + 0.11^2 + 0.03^2 + 0.11^2 + 0.25^2 = 2.47$$

The overall standard deviation of the critical path is then obtained by taking the square root of the total variance:

$$\sigma_{cp} = \sqrt{\sigma_{cp}^2} = \sqrt{2.47^2} = 1.57$$

Thus, the standard deviation of the critical path is 1.57 human-days, representing the expected variation in the project's total duration. This value will be used in the next step to determine the probability of completing the project within the given deadline and to analyze different probability levels for project completion.

To determine the probability of completing the project within the available timeframe, it is necessary to calculate the Z-score, which indicates how many standard deviations the given deadline deviates from the expected project duration. The Z-score formula is as follows:

$$Z = \frac{T_{deadline} - T_{cp}}{\sigma_{cp}}$$

Where:

- $T_{deadline}$ is the total available working time (69 days);
- T_{cp} is the expected duration of the critical path (47 days); σ_{cp} is the standard deviation of the critical path (1.57 days).

Substituting the values:

$$Z = \frac{69 - 47}{1.57} = \frac{22.00}{1.57} = 14.01$$

Using the standard normal distribution table, a Z-score of 14.01 corresponds to a probability of nearly 100%. Since standard tables typically provide values only up to Z ≈ 3.99, and even at this level the probability exceeds 99.99%, it is safe to conclude that the probability of completing the project within the 69 available working days is effectively 100%.

This result indicates that, under the current estimates, there is no significant risk of missing the deadline. However, to ensure robustness in project planning, further analysis can be

performed to determine the project duration required to meet different confidence levels (e.g., 75%, 90%, or 100% probability of completion).

3.8.1 Risk Evaluation

To determine the full list of project risks within the framework of the developed system, the structured brainstorming method was chosen, which has proven itself as an effective tool in managing project uncertainties. As noted in the theoretical part of this study, this method ensures high-quality identification of risks by stimulating collective thinking and knowledge sharing between experts. It is especially effective in the early stages of project activities, when it is important to record as many potential threats as possible before the start of a formal assessment.

A cross-functional team of experts participated in the session, which was involved in the process of decomposing the project into individual tasks. Thanks to this, the participants were already deeply immersed in the specifics of the project at the time of risk analysis, which made it possible to minimize the likelihood of missing significant risks. The team included engineers, designers, and testers. This combination of professional competencies provided a multifaceted view of possible threats and uncertainties, which was fully consistent with the concept of a comprehensive assessment outlined in the theoretical justification of the method.

The session itself was organized and facilitated by the author of this thesis, who acted as the session moderator. As described in the theoretical section, clearly defining the discussion objective is a key element of effective brainstorming. In this case, the participants were informed in advance that the goal was to develop a comprehensive and relevant list of risks associated with the project implementation. This approach helped steer the discussion in a constructive direction and focus attention on the key aspects.

As Young (2025) emphasizes, “Brainstorming techniques provide a platform for effective collaboration among team members, allowing them to freely express their ideas, concerns, and potential risks associated with the project. These sessions encourage active participation and engagement, fostering a creative environment where team members can build upon each other’s suggestions” (“Brainstorming” section, para. 2). Adhering to this methodology ensured an open, dynamic, and productive atmosphere in which risks were examined comprehensively and systematically, fully aligning with the theoretical framework presented in the theoretical part of this thesis.

Given the specifics of the project involving the development of a complex IT system, members of the expert group were able not only to identify potential risks at an early stage, but also to determine which specific project tasks would be affected if each risk were to materialize. This was made possible by the fact that the team had already participated in the project decomposition and possessed a deep understanding of the task structure.

Thus, as part of the risk identification process, an additional analytical step was introduced: for each identified risk, a list of tasks potentially impacted in the event of its occurrence was

documented. This approach made it possible to establish cause-and-effect links in advance between potential threats and elements of the project plan, which later served as a foundation for more accurate prioritization and the development of appropriate risk response strategies.

The final table includes the risk category, a detailed description of each risk, and the list of project tasks that may be affected. This extended format significantly enhanced the analytical value of the data and provided a solid foundation for integrating the results into the risk management plan and conducting impact analysis on the project timeline using simulation modeling. The final list of identified risks is presented in Table 4.

Table 4 - Project Risks

Category	Risk Name	Risk Description	Tasks Impacted
Third-Party Integration	External API Instability	External APIs used for payments and notifications may change data formats or become unstable, leading to integration failures, data corruption, or service disruptions.	24, 26, 27, 56, 73
Third-Party Integration	Mandatory Library Update Due to Vulnerabilities	If a vulnerability is discovered in a third-party library (e.g., identified and patched by the provider), the project will need to update to the latest version to mitigate security risks. This may require code adjustments, compatibility testing, and increased development effort.	1
Technical	Unforeseen Limitations	External APIs may have undocumented or unforeseen limitations (e.g., rate limits, payload size limits) that could cause unexpected failures, performance degradation, or incomplete data processing.	24, 26, 27, 56, 73
Technical	New Setup Requirements	During development, additional requirements for integration with the payment system emerged, requiring a review of the architecture.	24, 26, 27, 28
Security	Unforeseen Security Requirements	New security requirements were identified during development, leading to changes in authentication, access control, and payment processing.	3, 6, 26, 35, 65, 68
Technical	Unforeseen Database Changes	New database requirements were discovered during third-party integration, leading to additional changes in the database structure.	2

Hardware	Gate Access Hardware Failures	Physical access devices may malfunction or fail to sync with the software, causing delays in integration and testing.	64, 65, 66, 67, 68
Hardware	Unannounced Client Hardware Updates	If the client updates hardware without notice, adjustments and fixes will be needed, causing delays and increased workload.	62, 63, 64, 65, 66, 67, 68
Resource Constraints	Unplanned absences	Unplanned absences due to illness, resignation, or burnout	Multiple tasks

Once the risks were identified, each expert assessed them using two parameters: probability of occurrence and impact on the project. The assessment was carried out using a five-point linguistic scale, where the values had the following meaning:

- 5 – High
- 4 – Above Average
- 3 – Average
- 2 – Below Average
- 1 – Low

This type of scale enables experts to express subjective judgments in a clear and standardized manner, thereby providing a solid foundation for converting these assessments into fuzzy values.

A group of eight specialists was involved in the risk assessment, including five developers, two testers, and a project manager – the author of the thesis.

Below is a summary table of all experts' assessments for the Probability and Impact parameters. The table uses the following expert role abbreviations: D – Developer, QA – Quality Assurance Specialist, PM – Project Manager.

Table 5 - Risks Evaluation (Probability and Impact)

Risk Name	**Probability**							
	D1	D2	D3	D4	D5	QA1	QA2	PM
External API Instability	1	1	1	1	1	1	2	2
Mandatory Library Update Due to Vulnerabilities	2	2	3	2	3	3	2	2
Unforeseen Limitations	2	4	3	3	1	4	3	4

New Setup Requirements	2	3	3	4	2	3	2	3
Unforeseen Security Requirements	1	3	1	1	2	2	2	2
Unforeseen Database Changes	2	3	2	3	2	1	2	1
Gate Access Hardware Failures	2	4	2	3	2	3	4	3
Unannounced Client Hardware Updates	1	2	1	1	1	2	3	2
Unplanned absences	0	0	0	0	0	0	0	1
Risk Name	**Impact**							
External API Instability	2	3	2	2	1	3	3	3
Mandatory Library Update Due to Vulnerabilities	1	3	2	2	2	3	2	3
Unforeseen Limitations	3	5	4	2	3	3	3	3
New Setup Requirements	2	3	3	2	2	3	4	4
Unforeseen Security Requirements	2	3	2	2	1	5	4	3
Unforeseen Database Changes	3	4	2	2	4	2	3	2
Gate Access Hardware Failures	1	2	3	3	2	5	5	5
Unannounced Client Hardware Updates	2	3	3	2	2	2	2	1
Unplanned absences	0	0	0	0	0	0	0	1

According to the Fuzzy Delphi methodology, each numerical value was converted into a triangular fuzzy number based on the fixed equivalents presented in Table 6:

Table 6 - Conversion of Scores to Triangular Fuzzy Numbers

Score	**Fuzzy Value** (a_{ij}, b_{ij}, c_{ij})
5	[0.75, 1.0, 1.0]
4	[0.6, 0.75, 0.9]

3	[0.4, 0.5, 0.6]
2	[0.2, 0.3, 0.4]
1	[0.0, 0.0, 0.2]

Then, to account for differences in the competence of experts across risk categories. Each role was assigned a weighting factor depending on the type of risk. The result is presented in Table 7:

Table 7 - Experts Weights Mapping by Risk Category

Specialist	**Weight (w_i)**				
	Security	**Technical**	**Resource Constraints**	**Hardware**	**Third-Party Integration**
Dev	1.0	1.0	0.0	1.0	1.0
QA	1	0.3	0.0	0.4	0.5
PM	0.3	0.0	1.0	0.0	0.1

At the next stage, each expert estimate (a_{ij}, b_{ij}, c_{ij}) was multiplied by the corresponding weighting factor w_i for the specific risk category. This resulted in weighted triangular fuzzy numbers. All weighted evaluations for each risk were then aggregated, with separate aggregation carried out for probability and impact. Following the transformation of expert assessments into triangular fuzzy numbers and their adjustment based on individual competence weights w_i, the aggregation phase was implemented. As detailed in the theoretical part of this thesis, this step aims to produce a consolidated risk evaluation that accounts for the contribution of each expert. The weighted values reflected the domain-specific relevance of each expert: for instance, in the evaluation of technical risks, developers' opinions were given higher priority than those of testers, while for information security risks, the input of QA specialists was weighted more heavily. This approach allowed for the consideration of each expert's specialized knowledge, thereby enhancing the reliability of the final results.

Aggregation was carried out separately for the two parameters: the Probability of risk and its Impact. For each risk j, aggregated triangular fuzzy numbers were constructed, consisting of three values: the lower bound L, the most likely value M, and the upper bound U. These values were calculated using the following general formula:

$$L_j^{(X)} = \sum_i \left(w_i \cdot a_{ij}^{(X)} \right)$$

$$M_j^{(X)} = \sum_i \left(w_i \cdot b_{ij}^{(X)} \right)$$

$$U_j^{(X)} = \sum_i \left(w_i \cdot c_{ij}^{(X)} \right)$$

Where:

- $X \in \{P, I\}$, meaning the formula is applied to both Probability (P), and Impact (I);
- w_i denotes the weight of expert i;
- $a_{ij}^{(X)}$, $b_{ij}^{(X)}$, $c_{ij}^{(X)}$ are the components of the triangular fuzzy number representing the individual expert evaluation of parameter X for risk j, corresponding to the minimum, most likely, and maximum values, respectively.

Thus, for each risk j, two triangular fuzzy numbers are obtained.

For Probability:

$$\widetilde{R_j^{(P)}} = \left(L_j^{(P)}, M_j^{(P)}, U_j^{(P)} \right)$$

For Impact:

$$\widetilde{R_j^{(I)}} = \left(L_j^{(I)}, M_j^{(I)}, U_j^{(I)} \right)$$

These values represent not only aggregated expert assessments, but functional parameters used for risk modeling via the Monte Carlo method. Each aggregated value is interpreted as a triangular distribution, where L denotes the minimum possible level, U the maximum, and M the most likely value.

During each iteration of the simulation, the probability of risk j occurring is determined by randomly generating a value from the distribution:

$$x \sim \text{Triangular}\left(L_j^{(P)}, M_j^{(P)}, U_j^{(P)} \right)$$

If the generated value indicates that the risk has materialized, the simulation proceeds to the second phase—modeling the impact value:

$$y \sim \text{Triangular}\left(L_j^{(I)}, M_j^{(I)}, U_j^{(I)} \right)$$

The resulting value y is interpreted as the percentage increase in the duration of all tasks affected by the given risk. For example, if $y = 0.15$, the duration of affected tasks is increased by 15%. In this way, each simulation run reflects a unique scenario, accounting for both the uncertainty in the likelihood of risk occurrence and the variability of its consequences. This approach enables realistic modeling of risk impacts on the project, fully aligned with contemporary practices in project uncertainty management and the theoretical principles outlined earlier in this thesis.

The final values of the aggregated assessments are presented in Table 8 and serve as input for the probabilistic analysis using the Monte Carlo method. The approach to this analysis, along with its results, is described in detail in the following section of the thesis.

Table 8 - Fuzzy Risk Estimates (Probability & Impact)

Risk Name	**Fuzzy Probability**	**Fuzzy Impact**
Unplanned absences	[0.0, 0.0, 0.2]	[0.0, 0.0, 0.2]
External API Instability	[0.004, 0.006, 0.204]	[0.236, 0.32, 0.436]
Unforeseen Limitations	[0.32, 0.41, 0.54]	[0.463, 0.598, 0.689]
New Setup Requirements	[0.36, 0.47, 0.58]	[0.304, 0.406, 0.509]
Mandatory Library Update Due to Vulnerabilities	[0.278, 0.378, 0.478]	[0.22, 0.303, 0.42]
Unforeseen Security Requirements	[0.125, 0.168, 0.325]	[0.338, 0.452, 0.559]
Gate Access Hardware Failures	[0.32, 0.43, 0.54]	[0.31, 0.414, 0.517]
Unannounced Client Hardware Updates	[0.04, 0.06, 0.24]	[0.269, 0.369, 0.469]
Unforeseen Database Changes	[0.28, 0.38, 0.48]	[0.389, 0.507, 0.625]

To complement the tabular presentation of aggregated triangular fuzzy numbers, a risk heatmap was constructed (Figure 3) from the most-likely values (M) of each distribution. The X-axis (Probability) and Y-axis (Impact) are partitioned into Low (0.00–0.33), Medium (0.34–0.66) and High (0.67–1.00) zones.

The chart shows that Unforeseen Limitations (Probability ≈ 0.42, Impact ≈ 0.60) and Unforeseen Database Changes (Probability ≈ 0.38, Impact ≈ 0.50) occupy the upper part of the matrix, where medium probability meets the highest impact. These two risks are therefore the most critical for the schedule and require priority mitigation.

The remaining risks are less severe. New Setup Requirements and Gate Access Hardware Failures lie in the medium-probability, medium-impact zone and need regular monitoring. Unforeseen Security Requirements has a lower probability but elevated impact, so a contingency plan is advisable. Mandatory Library Update falls in the medium-probability, low-impact area and can usually be absorbed by existing buffers. External API Instability and Unannounced Client Hardware Updates sit in low-probability, medium-impact positions, warranting periodic review. Unplanned Absences rests in the low-probability, low-impact corner and poses minimal concern under current staffing.

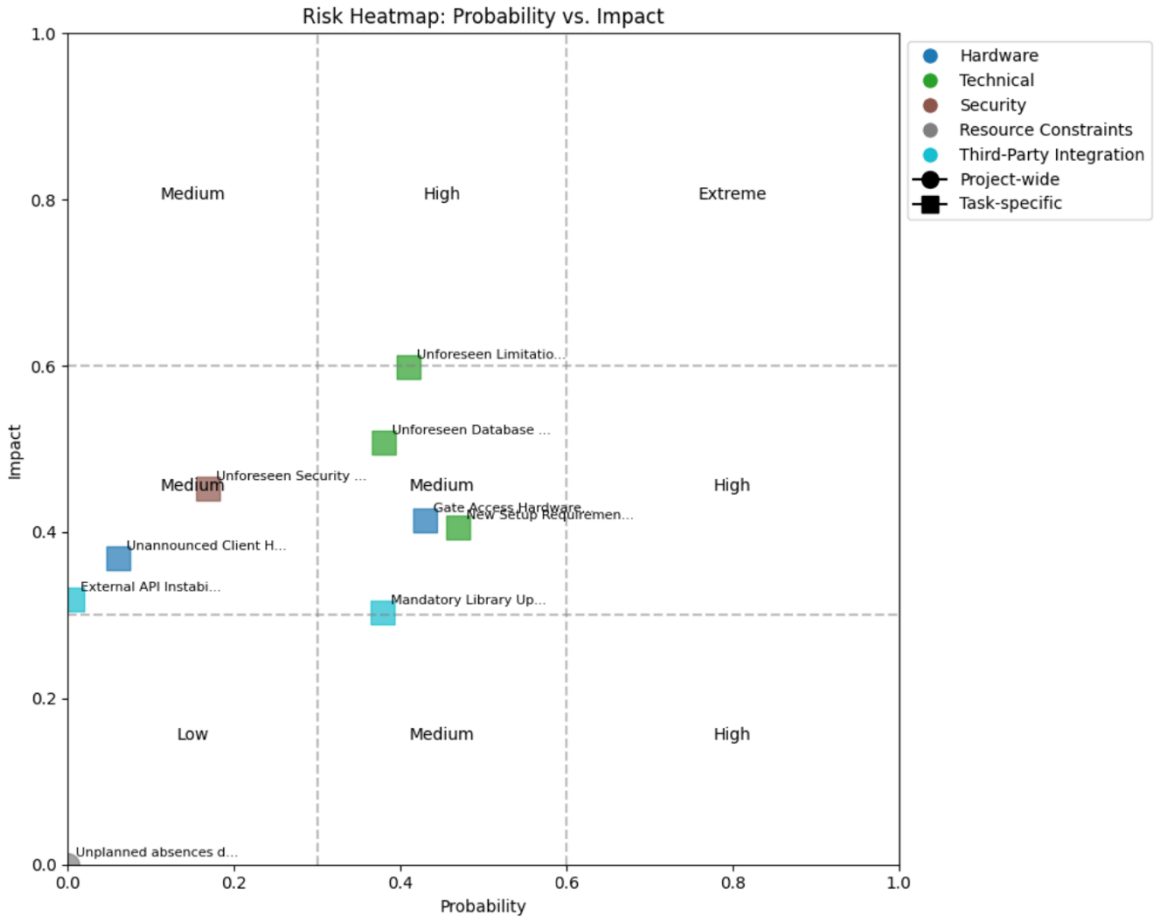


Figure 3 - Risk Heatmap: Probability vs. Impact

In summary, the heatmap converts the fuzzy estimates into a clear visual hierarchy of threats. These priorities feed directly into the subsequent Monte Carlo simulation, ensuring that the stochastic model places proportionate emphasis on the most influential risks and thereby strengthening the reliability of the overall schedule forecast.

3.8.2 Application of the Monte Carlo Method

The Monte Carlo method is applied to perform probabilistic analysis of the total project duration, taking into account the uncertainty of initial task estimates and potential risks identified in the previous stages of research. The simulation is based on a network diagram, described in the theoretical section (*Network Diagram Creation*), which establishes clear

logical dependencies and sequence of task execution. Thus, within this simulation, no recalculation of the critical path is performed, and only the previously established logical relationships between tasks are utilized.

In the first stage of each iteration, a duration value is randomly generated for all tasks. The selection is based on three estimation points—optimistic (*O*), most likely (*M*), and pessimistic (*P*)—which were determined previously. For the sampling of random values, the beta-PERT distribution is utilized, which is characterized by an increased probability of the random value falling closer to the most likely estimate. The parameters of this distribution are calculated as follows:

$$\alpha = 1 + 4 \cdot \frac{M - O}{P - O},$$

$$\beta = 1 + 4 \cdot \frac{P - M}{P - O}$$

Where:

α and β are shape parameters of the beta distribution, determining the degree of its asymmetry;

- *O* is the optimistic estimate (minimum possible task duration);
- *M* is the most likely estimate (the most frequently occurring task duration);
- *P* is the pessimistic estimate (maximum possible task duration).

Hence, the random duration (*D*) of each task is calculated using the formula:

$$D = O + (P - O) \cdot X;$$

$$X \sim \text{Beta}(\alpha, \beta)$$

Here, *X* is a random variable distributed according to the standard beta distribution with the calculated shape parameters. This approach provides a probabilistically grounded estimation of task durations.

At the second stage of each iteration, risk modeling is conducted. For each risk, the probability of occurrence is randomly determined based on the previously aggregated parameters of the triangular distribution. Should the generated value indicate that the risk occurs, its impact is subsequently modeled using the corresponding triangular distribution derived from weighted expert evaluations. The resulting value is interpreted as the percentage increase in the duration of all tasks affected by the given risk. Consequently, the previously simulated task durations are adjusted by proportionally increasing them according to the modeled impact.

After incorporating the effects of all realized risks on task durations, the total project duration is calculated based on the dependencies established earlier in the network diagram. Each iteration concludes with the computation of the overall project duration, and the process is repeated 100,000 times. The simulation was performed using proprietary

software developed in-house by Limestone Digital. As the tool is not publicly available, it is referenced here with acknowledgment of its origin and intellectual property status. Detailed implementation, including code, algorithms, and output visualization, is provided in Appendix L.

Upon completion of all iterations, a distribution of the total project duration is formed, based on which the mean, median, 90th percentile, and the probability of meeting the target deadline are calculated. The simulation results are presented as histograms (Figure 4 below), illustrating the significant impact that risk consideration has on the overall project timeline.

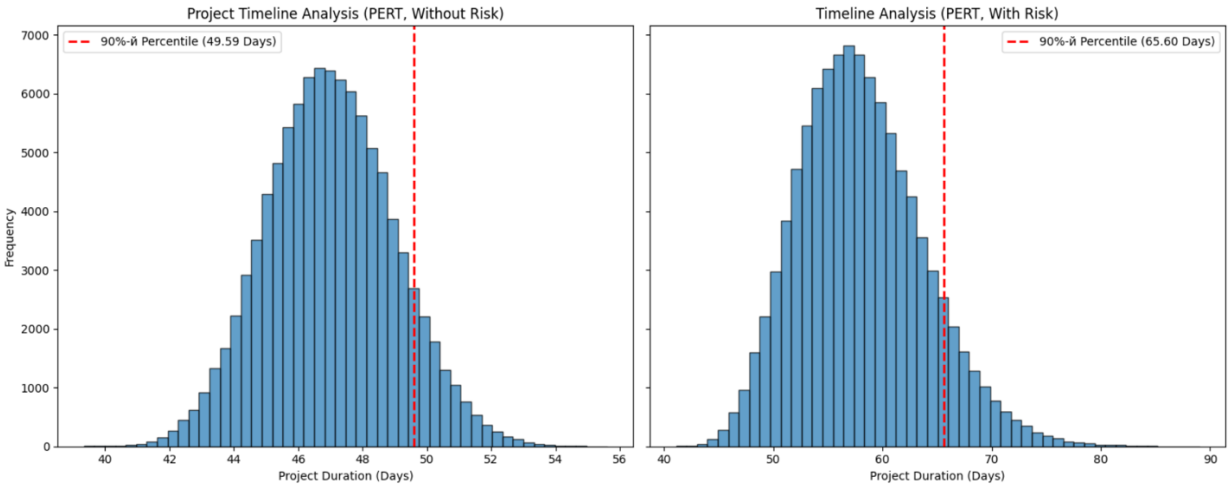


Figure 4 - Project Duration Distribution: Comparison of Timelines Without and With Risk Adjustment. Produced with internal tools developed by Limestone Digital (access restricted)

The results of the Monte Carlo simulation are shown in the figure above. They represent the empirical distribution of project duration under two conditions: without taking risk into account (left) and with taking risk into account (right). In each case, 100,000 iterations were performed to ensure statistical reliability of the estimates.

In this context, the 90th percentile was chosen as the reference point for schedule reliability because it reflects a widely accepted standard in both industry and public-sector project management. According to Barreras (2011), P90 is often used to include the management reserve in project estimates, and Monte Carlo simulation tools routinely highlight P80 and P90 values to support decision-making . This approach provides a high level of confidence while avoiding excessive contingency that could lead to inefficient time use. Similarly, the U.S. Government Accountability Office (GAO, 2012) notes that planning with percentiles in the 80–90% range is considered an appropriately cautious strategy. The GAO emphasizes that certainty comes at a high cost in terms of schedule length, and that each organization should choose a risk tolerance based on context, with many adopting 80–90% as a realistic benchmark to avoid unnecessary time padding (GAO, 2012). Therefore, the use of the 90th percentile in this study ensures that the schedule remains both achievable and efficient.

In the baseline scenario, based on task-level three-point PERT estimates, the 90th percentile of the project duration is 49.59 days. This means there is a 90% probability that the project will be completed within this timeframe. The simulation was conducted without incorporating additional project risks, relying solely on the inherent uncertainty of task

durations defined by optimistic, most likely, and pessimistic estimates. The result reflects stable project conditions and demonstrates the natural schedule variability captured through stochastic modeling alone.

When risk factors are incorporated (the risk-adjusted scenario), the distribution shifts significantly: the 90th percentile reaches 65.71 days, indicating a substantial increase in the variability of project duration. Furthermore, the mean value rises by approximately 16.01 days compared to the baseline scenario. Nevertheless, the predefined limit of 69 working days remains attainable—the probability of meeting this deadline, according to the simulation, exceeds 90%; however, the schedule reliability approaches a critical threshold. This conclusion is further supported by the cumulative distribution function shown in Figure 5.

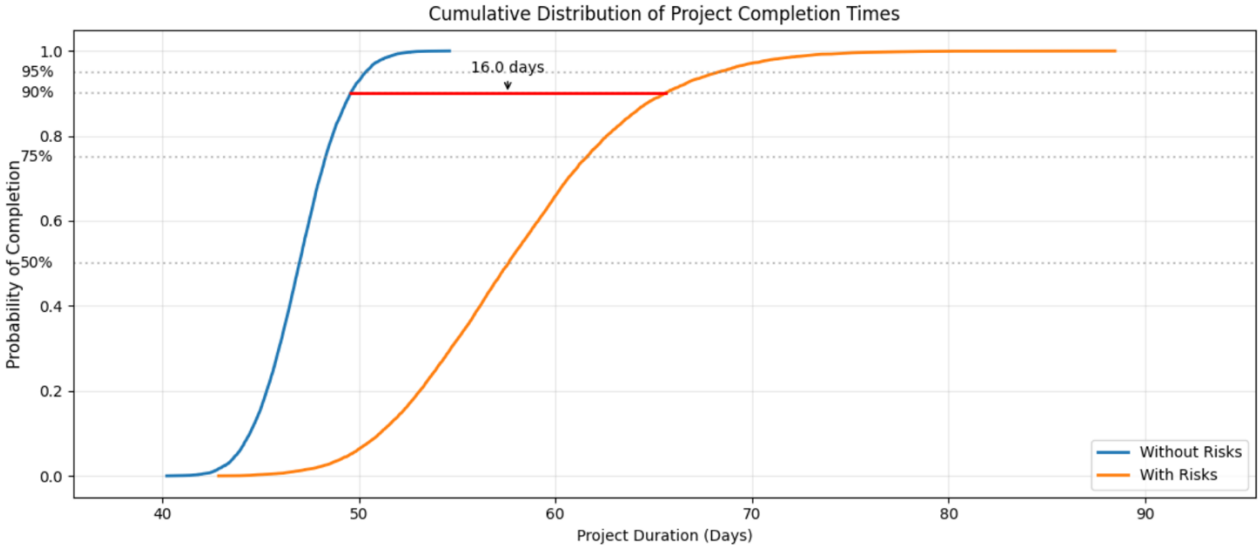


Figure 5 - Cumulative Distribution of Project Completion Times. Produced with internal tools developed by Limestone Digital (access restricted)

While histograms illustrate the shape of distribution, the cumulative view highlights the probability of completing the project by any specific date. The blue curve represents the baseline scenario (without risk adjustments), while the orange curve reflects the impact of identified risks on project duration.

The steeper ascent of the blue curve between days 45 and 50 indicates relatively low variability and high confidence in the schedule when risks are not considered. In contrast, the more gradual slope of the orange curve in the 52–69 day range reflects increased uncertainty caused by the realization of various risks. The vertical gap between the two curves at the 90th percentile is approximately 16 days, visually representing the "risk buffer" — the additional time required to preserve the same level of schedule confidence under uncertain conditions.

Hence, the simulation underscores the necessity of incorporating a time buffer to ensure, for instance, a 95% confidence level. The 69-day deadline can be considered realistic only under the condition of proactive risk management and close monitoring of the tasks that contribute most significantly to overall schedule variability.

3.9 Comparative Analysis of Planning Methods

In conditions of high uncertainty — typical of outsourced greenfield projects with strict deadlines — the choice of an adequate planning methodology becomes one of the key success factors. However, in industry practice, the phases of analysis and planning are often underestimated due to budget limitations, client pressure, or the desire to move to development as quickly as possible. This results in the use of simplified models, such as the Critical Path Method, which make it possible to build a schedule quickly, but do not take into account variability and risks.

In contrast, the Sentinel Tenant Portal project, was approached by following a different approach. Despite a expedite date tied to an industry exhibition, the company deliberately invested time and resources into the research phase. This made it possible to conduct a structured analysis of requirements at an early stage, identify potential risks, and develop a realistic project plan. A further advantage was the involvement of an experienced team that had already worked on similar solutions in terms of functionality and technology. Together, these factors created favorable conditions for precise and manageable planning.

This study considered and compared three approaches to schedule management: CPM, PERT, and PERT enhanced with Monte Carlo simulation.

The CPM method established a fixed project duration of 45 working days, based on a clearly defined network structure and deterministic task duration estimates. This approach was effective for building a baseline schedule, but it does not reflect uncertainty or external influences. Even during the preparation stage, risks were identified which, if realized, could have led to reduced client confidence, shortened user testing before the exhibition, or a reduction in functionality—outcomes incompatible with the fixed-scope contract.

The PERT method, which applies three-point estimates (optimistic, most likely, pessimistic), provided a more realistic average duration — approximately 47 working days. A Z-score calculation confirmed an almost 100% probability of completing the project by the contractual deadline of September 9, 2024. Despite its simplicity, PERT demonstrated high reliability, thanks in large part to the team's expertise. With skilled specialists capable of providing accurate task estimates and identifying risks, even relatively basic probabilistic methods can serve as reliable planning tools.

The highest level of accuracy and analytical depth was achieved using a combined method—PERT complemented by Monte Carlo simulation. This approach enabled consideration of a wide range of risk scenarios and quantitative assessment of their impact on the project timeline. A key factor was the preliminary risk assessment stage, which helped both the client, and the project team understand the nature of potential threats and transform qualitative assumptions into quantitative model inputs.

The simulation was conducted under two scenarios. In the first—without risk modeling—using only expert three-point estimates and beta distribution, the 90th percentile reached 49.59 working days, which is 2.3 days longer than the average duration calculated via classical PERT (47.29). This demonstrates that deviations from average duration can occur

even in stable environments. In the second, risk-inclusive scenario, the 90th percentile increased to 65.60 working days, and the average duration rose by 16.01 days. This clearly illustrates how significantly external and internal risks can affect project timelines, even when the scope is fixed. Nevertheless, the final probability of completing the project within the 69-working-day limit exceeded 90%, which confirms the robustness of the combined approach. However, the results of the simulation suggest that an additional time buffer should be incorporated.

Thus, under the conditions specific to Sentinel Tenant Portal— a mature and experienced team, a conscious attitude toward planning, time allocated for preparation, and fixed external deadlines—the most justified planning methodology proved to be the combination of PERT and Monte Carlo simulation. This approach ensures a balance of realism, reliability, and flexibility, enabling the project manager to make well-informed decisions, adjust the schedule as needed, and manage client expectations effectively. It may be recommended as a methodologically sound solution for projects operating under high uncertainty and contractual constraints.

4 Conclusion

The modern IT outsourcing sector is characterized by intense competition, making the ability to plan schedules reliably and transparently a vital strategic advantage. In this context, the findings of this study can be of practical value for supporting informed project decisions within the company, particularly when selecting an appropriate planning methodology.

The examined project — Sentinel Tenant Portal, developed in 2024 by Limestone Digital — represents a greenfield initiative with a fixed scope and a rigidly defined completion deadline. Such constraints render precise planning critically important, as delays could negatively affect the client's marketing strategy, shift the product launch timeline, and diminish overall brand perception.

This study compared two primary methods — Critical Path Method (CPM) and Program Evaluation and Review Technique (PERT) — as well as their extension through Monte Carlo simulation, supplemented by qualitative risk evaluation using the Fuzzy Delphi method. CPM enabled the creation of a baseline schedule and task structure but demonstrated limited resilience to uncertainty, yielding a deterministic estimate of 45 working days. The PERT method provided a more realistic estimation of duration, producing an expected value of 47 working days. Monte Carlo simulation—augmented with risk inputs — allowed for a quantitative incorporation of uncertainties by determining the 90th-percentile project duration, which reached 65.6 working days. The use of Fuzzy Delphi facilitated the aggregation of expert opinions during the risk-analysis phase, thereby strengthening the credibility of input data. As a result, a probabilistic forecast was constructed, enabling not only the identification of sensitive points within the project but also the proactive formulation of time buffers. This confirmed the capacity of the selected methods to enhance schedule robustness, mitigate risk, and offer a transparent timeline outlook.

During the comparative analysis phase, it was established that the combined approach—based on PERT and Monte Carlo simulation — is best suited for greenfield projects with fixed deadlines, provided that implementation is carried out by an experienced team with the time and resources necessary for preliminary analysis. This approach offers an optimal balance between precision, adaptability, and control. Furthermore, the research demonstrated that a well-structured risk management process, implemented already at the planning stage, is a critical factor in achieving reliable timeline forecasting and resource estimation. This, in turn, directly contributes to the stability of project execution and the quality of the final delivery.

The author of this thesis served as the project manager during the implementation of the Sentinel Tenant Portal and actively applied critical path calculations, risk analysis results, and probabilistic schedule estimates throughout development. This enabled flexible change management, the alignment of buffer periods, and improved reliability in planning. The project was successfully completed on the 53rd working day, corresponding to August 26,

2024 — an outcome that validated both the accuracy of the simulation and the soundness of the chosen approach.

Undoubtedly, each project has its own specific characteristics, and no single planning method can be universally effective in all circumstances. Nevertheless, this research has formulated evidence-based recommendations for selecting scheduling approaches in projects marked by high uncertainty, fixed scope, and the presence of a qualified team with access to the resources required for early-stage analysis.

List of references

Abdel-Basset, M., Atef, A., Abouhawwash, M., Nam, Y., & Abdelaziz, N. (2022). *Network analysis for projects with high risk levels in uncertain environments*. *Computers, Materials & Continua, 70*(1), 1281–1296. https://doi.org/10.32604/cmc.2022.018947

Accidental PM. (2024, September 22). *Top 5 project scheduling techniques every manager should know*. *Accidental Project Manager*. https://www.accidentalpm.online/blog/top-5-project-scheduling-techniques-every-manager-should-know

Adler, M., & Ziglio, E. (Eds.). (1996). *Gazing into the oracle: The Delphi method and its application to social policy and public health*. Jessica Kingsley Publishers.

Ali, S. (2023, September 20). *There is no strict limitation on the number of experts required in a fuzzy Delphi method study* [Comment on the online forum post "How many experts are needed in Fuzzy Delphi Method?"]. *ResearchGate*. https://www.researchgate.net/post/How_many_experts_are_needed_in_Fuzzy_Delphi_Method

Association for Project Management. (n.d.). *What is project management?* Retrieved June 26, 2025, from https://www.apm.org.uk/resources/what-is-project-management/

Bagshaw, K. B. (2021a). *PERT and CPM in project management with practical examples*. *American Journal of Operations Research, 11*(4), 215–226. https://doi.org/10.4236/ajor.2021.114013

Bagshaw, K. B. (2021b). *Project scheduling and control: Planning techniques and applications*. Routledge. https://doi.org/10.4324/9780429298776

Barghi, B., & Shadrokh, S. (2020). Qualitative and quantitative project risk assessment using a hybrid PMBOK model developed under uncertainty conditions. *Heliyon, 6*(1), e03097. https://doi.org/10.1016/j.heliyon.2019.e03097

Barreras, A. J. (2011, October). *Risk management: Monte Carlo simulation in cost estimating*. Paper presented at PMI® Global Congress 2011—North America, Dallas, TX, United States. Project Management Institute. https://www.pmi.org/learning/library/risk-management-monte-carlo-cost-estimating-7048

Broadleaf Capital International. (2014, July). *Beta PERT origins*. https://broadleaf.com.au/resource-material/beta-pert-origins/

Business and Technology University. (2023). *Z-score and its applications in project management* [Lecture notes]. https://www.btu.edu.ge/edu-materials/statistics-z-score

Clark, C. E. (1962). The PERT model for the distribution of an activity time. *Operations Research, 10*(3), 405–406. https://doi.org/10.1287/opre.10.3.405

Desticioğlu Taşdemir, B. (2022). Project planning with CPM and PERT methods: Example of defence industry. *Journal of Naval Sciences and Engineering, 18*(2), 363–385.

FasterCapital. (2025, April 4). *Project evaluation and review technique (PERT): The impact of PERT on startup success: Lessons from the field*. *FasterCapital*. https://fastercapital.com/content/Project-Evaluation-and-Review-Technique--PERT---The-Impact-of-PERT-on-Startup-Success--Lessons-from-the-Field.html

Fateminia, S. H., Nguyen, P. H. D., & Fayek, A. R. (2021). An adaptive hybrid model for determining subjective causal relationships in fuzzy system dynamics models for analyzing construction risks. *CivilEng, 2*(3), 747–764. https://doi.org/10.3390/civileng2030041

Fiala, P. (2014). *Řízení projektů: Modely, metody, analýzy* (2nd ed.). Professional Publishing.

Hegazy, T., & Menesi, W. (2010). Critical path segments scheduling technique. *Journal of Construction Engineering and Management, 136*(10), 1078–1085. https://doi.org/10.1061/(ASCE)CO.1943-7862.0000212

Helmi, J. (2024, September 24). *Comparing deterministic and probabilistic scheduling in project management*. *LinkedIn*. https://www.linkedin.com/pulse/comparing-deterministic-probabilistic-scheduling-project-pmp-k6a2c/

Holistique Training. (2023). *Unlocking project success with CPM: Techniques, benefits, and best practices*. https://holistiquetraining.com/en/news/unlocking-project-success-with-cpm-techniques-benefits-and-best-practices

Investopedia Team. (2025, May 19). *Law of large numbers: What it is, how it's used, examples*. *Investopedia*. https://www.investopedia.com/terms/l/lawoflargenumbers.asp

Kaplan-Moss, J. (2021, June 8). Estimating software projects: So you messed up. Now what? *Jacob Kaplan-Moss*. https://jacobian.org/2021/jun/8/incorrect-estimates/

Kerzner, H. (2009). *Project management: A systems approach to planning, scheduling, and controlling* (10th ed.). John Wiley & Sons.

Kerzner, H. (2017). *Project management: A systems approach to planning, scheduling, and controlling* (12th ed.). John Wiley & Sons.

Khan, U. U., Ali, Y., Garai-Fodor, M., & Csiszárik-Kocsir, Á. (2023). Application of project management techniques for timeline and budgeting estimates of startups. *Sustainability, 15*(21), 15526. https://doi.org/10.3390/su152115526

Levy, F. K., Thompson, G. L., & Wiest, J. D. (1963). The ABCs of the critical path method. *Harvard Business Review, 41*(5), 133–142.

Lewis, J. P. (2017). *PERT and CPM differences*. In *Project planning and scheduling notes* (pp. 1–2). Scribd. https://fr.scribd.com/doc/7025970/CPM-PERT

Lv, L., Li, H., Wang, L., Xia, Q., & Ji, L. (2019). Failure mode and effect analysis (FMEA) with extended MULTIMOORA method based on interval-valued intuitionistic fuzzy set: Application in operational risk evaluation for infrastructure. *Information, 10*(10), 313. https://doi.org/10.3390/info10100313

Malcolm, D. G., Roseboom, J. H., Clark, C. E., & Fazar, W. (1959). Application of a technique for research and development program evaluation. *Operations Research, 7*(5), 646–669. https://doi.org/10.1287/opre.7.5.646

Main, L. (1989). CPM and PERT in library management. *Special Libraries, 80*(1), 39–44.

Microsoft Support. (n.d.). *The project triangle*. Retrieved June 26, 2025, from https://support.microsoft.com/en-us/topic/triangle-8c892e06-d761-4d40-8e1f-17b33fdcf810

Mohd Yusoff, H., Heng, P. P., Hj Illias, M. R., Karrupayah, S., Fadhli, M. A., & Hod, R. (2024). A qualitative exploration and a fuzzy Delphi validation of high-risk scaffolding tasks and fatigue-related safety behavioural deviation among scaffolders. *Heliyon, 10*(15), e34599. https://doi.org/10.1016/j.heliyon.2024.e34599

Ocampo, L., Acedillo, V., Bacunador, A., Balo, C., Lagdameo, Y., & Tupa, N. (2018). A historical review of the development of organizational citizenship behavior (OCB) and its implications for the twenty-first century. *Personnel Review, 47*(4), 821–862. https://doi.org/10.1108/PR-04-2017-0136

Passenheim, O. (2010). *Enterprise risk management*. Ventus Publishing ApS. https://bookboon.com/en/enterprise-risk-management-ebook

Project Management Institute. (2021a). *A guide to the project management body of knowledge* (PMBOK® Guide) (7th ed.). Project Management Institute.

Project Management Institute. (2021b). *Program evaluation and review technique (PERT)* (Version 3.2). In *PMI Lexicon of Project Management Terms*. Project Management Institute.

Ribaudo, V. (2022, May 13). *Diagramme de PERT: Un outil pour planifier vos projets*. *Prium Transition*. https://www.prium-transition.com

Roldán López de Hierro, A. F., Sánchez, M., Puente-Fernández, D., Montoya-Juárez, R., & Roldán, C. (2021). A fuzzy Delphi consensus methodology based on a fuzzy ranking. *Mathematics, 9*(18), 2323. https://doi.org/10.3390/math9182323

Samani, N. (n.d.). *Optimizing resource utilization through strategic production scheduling*. *Deskera Blog*. Retrieved June 26, 2025, from https://www.deskera.com/blog/optimizing-resource-utilization-strategic-production-scheduling/

Sentinel Systems. (2024). *Management software*. https://www.sentinelsystems.com/management-software

Siraj, S., Zakaria, A. R., Alias, N., Dewitt, D., Kannan, P., & Ganapathy, J. (2012). Future projection on patriotism among school students using Delphi technique. *Creative Education, 3*(6A), 1053–1059. https://doi.org/10.4236/ce.2012.326158

SixSigma.us. (2025, March 3). *Program evaluation and review technique (PERT) in Lean Six Sigma methodology*. https://www.6sigma.us/project-management/program-evaluation-and-review-technique-pert

Tuni, A., Ijomah, W. L., Gutteridge, F., Mirpourian, M., Pfeifer, S., & Copani, G. (2023). Risk assessment for circular business models: A fuzzy Delphi study application for composite materials. *Journal of Cleaner Production, 389*, 135722. https://doi.org/10.1016/j.jclepro.2022.135722

Udoumoh, E. F., & Ebong, D. W. (2017). A review of activity time distributions in risk analysis. *American Journal of Operations Research, 7*(6), 365–377. https://doi.org/10.4236/ajor.2017.76027

U.S. Government Accountability Office. (2012). *GAO cost estimating and assessment guide: Best practices for developing and managing capital program costs* (GAO-12-120G). https://www.gao.gov/assets/gao-12-120g.pdf

U.S. Government Accountability Office. (2016). *Schedule assessment guide: Best practices for project schedules* (GAO-16-89G). https://www.gao.gov/products/gao-16-89g

Van Slyke, R. (1963). Monte Carlo methods and the PERT problems. *Operations Research, 11*(5), 839–860. https://doi.org/10.1287/opre.11.5.839

Wyrozębski, P., & Wyrozębska, A. (2013). Challenges of project planning in the probabilistic approach using PERT, GERT and Monte Carlo. *Journal of Management and Marketing, 1*(1), 1–8.

Yin, T. T., & Hanif, H. (2024). Fuzzy Delphi method: A step-by-step guide to obtain expert consensus on MUETBot functionalities. *International Journal of Academic Research in Business and Social Sciences, 14*(4), 1109–1116. https://doi.org/10.6007/IJARBSS/v14-i4/21307

Young, S. M. (2025, February). *8 effective risk identification techniques for successful project management*. *PRINCE2 Study Guide*. https://prince2studyguide.com/8-effective-risk-identification-techniques-for-successful-project-management/

Zavala-Quinones, A. (2024, June 10). *Why very few digital and IT project managers know about PERT and even fewer work with it*. *LinkedIn*. https://www.linkedin.com

ZEP GmbH. (n.d.). *Network planning technology*. Retrieved June 26, 2025, from https://www.zep.de/en/glossary/network-planning-technique

Appendices

Appendix A: Project's Tasks Decomposition (The List of Tasks)

ID	Task Name
General	
1	Set up application infrastructure
2	Database Design and Management
3	Security Implementation
4	Visual Design phase
Authentication and Authorization	
5	Design User Authentication and Authorization
6	User registration with secure password storage
7	User Login
8	Two-factor authentication
9	Password recovery
10	Test Password recovery
11	Test User Login
12	Test Two-factor authentication
13	Test User registration
	User Profile and Preferences
14	Design Profile creation and settings
15	Profile creation and settings
16	Password update
17	Test Password update
18	Test Profile creation and settings

ID	Task Name
Account and Payment Management	
19	Design Account information display
20	Design Payment History with search and filter functionalities
21	Design Unit balances Payment Flow
22	Design Transaction Details
23	Account Information Display (name, customer number, address)
24	Payment History
25	Search, filter functionality
26	Integrate payment gateway
27	Unit balances Payment Flow
28	Transaction Details
29	Downloadable receipt feature (PDF)
30	Test account information display
31	Verify payment history functionality (rendering, search, filter, and pagination)
32	Test unit balance display and payment flow
33	Test transaction details rendering
34	Test downloadable receipt feature
35	Smoke, regression, and security testing for Account and Payment Management
Unit Management	
36	Design Unit List Display
37	Design Search and Filter
38	Design Detailed Unit View (e.g., insurance, secondary tenants)
39	Design Visual Indicators
40	Unit List Display
41	Search and filter

ID	Task Name
42	Detailed view pages for each unit - Insurance, Secondary tenants
43	Visual indicators (e.g., color-coded badges for Rented, Delinquent, Reserved)
44	"Pay" button for each unit, linking to the payment gateway
45	Test Unit List Display
46	Test Search and Filter
47	Test Detailed View
48	Payment Button Testing
49	Smoke, regression, and security testing for Account and Unit Management
Secondary tenant Management	
50	Design form for adding and creating a secondary tenant.
51	Design interface for editing secondary tenant information.
52	Design email notification template: "You were added as a secondary tenant."
53	Create new secondary tenant
54	Add secondary tenant from list
55	Edit secondary tenant information
56	Email notification "You were added as a secondary tenant"
57	Test Creating new secondary tenant
58	Test Adding a secondary tenant from the list
59	Test Edit secondary tenant information
60	Test Email notification "You were added as a secondary tenant"
61	Smoke, regression, and security testing for Account and Secondary Tenant Management
Gate and Access Control	
62	Design remote gate control dropdown with buttons for "Open Gate," "Close Gate," and "Alarm."
63	Design gates list associated with account

ID	Task Name
64	Display all gates associated with the account
65	Remote gate control - "Open Gate", "Close Gate", "Alarm"
66	Test Display all gates
67	Test Remote gate control - "Open Gate", "Close Gate", "Alarm"
68	Smoke, regression, and security testing for Gate and Access Control
Dashboard	
69	Design dashboard layout
70	Design notification display for updates (e.g., payment reminders, access updates)
71	Design quick action buttons for "Pay Balance," "Open Gate," and "Rent a Unit."
72	The list of rented units
73	Notifications (e.g., payment reminders, access updates).
74	Quick Action - "Pay Balance"
75	Quick Action - "Open Gate"
76	Quick Action - "Rent a Unit"
77	Test The list of rented units.
78	Test Notifications (e.g., payment reminders, access updates).
79	Test Quick action buttons ("Pay Balance," "Open Gate," "Rent a Unit")
80	Regression testing for dashboard functionalities.
81	Regression Testing

Appendix B: The result of Dependencies' Analysis (The List of Tasks with Previous Activities/Predecessors)

ID	Task Name	Preceding Activity
General		
1	Set up application infrastructure	
2	Database Design and Management	1
3	Security Implementation	1
4	Visual Design phase	
Authentication and Authorization		
5	Design User Authentication and Authorization	4
6	User registration with secure password storage	2; 3; 5
7	User Login	6
8	Two-factor authentication	7
9	Password recovery	7
10	Test Password recovery	9
11	Test User Login	7
12	Test Two-factor authentication	8
13	Test User registration	6
User Profile and Preferences		
14	Design Profile creation and settings	4
15	Profile creation and settings	14
16	Password update	12; 10

ID	Task Name	Preceding Activity
17	Test Password update	16
18	Test Profile creation and settings	15
Account and Payment Management		
19	Design Account information display	4
20	Design Payment History with search and filter functionalities	19
21	Design Unit balances Payment Flow	20
22	Design Transaction Details	21
23	Account Information Display (name, customer number, address)	19
24	Payment History	20
25	Search, filter functionality	20
26	Integrate payment gateway	11
27	Unit balances Payment Flow	26
28	Transaction Details	22; 27
29	Downloadable receipt feature (PDF)	28
30	Test account information display	23
31	Verify payment history functionality (rendering, search, filter, and pagination)	24
32	Test unit balance display and payment flow	27
33	Test transaction details rendering	28
34	Test downloadable receipt feature	29
35	Smoke, regression, and security testing for Account and Payment Management	31;32;33;34

ID	Task Name	Preceding Activity
Unit Management		
36	Design Unit List Display	4
37	Design Search and Filter	36
38	Design Detailed Unit View (e.g., insurance, secondary tenants)	36
39	Design Visual Indicators	38
40	Unit List Display	36;2
41	Search and filter	37;40
42	Detailed view pages for each unit - Insurance, Secondary tenants	2;38
43	Visual indicators (e.g., color-coded badges for Rented, Delinquent, Reserved)	39;40
44	"Pay" button for each unit, linking to the payment gateway	26;40
45	Test Unit List Display	40
46	Test Search and Filter	41
47	Test Detailed View	42
48	Payment Button Testing	44
49	Smoke, regression, and security testing for Account and Unit Management	46;47;48
Secondary tenant Management		
50	Design form for adding and creating a secondary tenant.	4
51	Design interface for editing secondary tenant information.	50
52	Design email notification template: "You were added as a secondary tenant."	4
53	Create new secondary tenant	2;50

ID	Task Name	Preceding Activity
54	Add secondary tenant from list	2;50
55	Edit secondary tenant information	53;54
56	Email notification "You were added as a secondary tenant"	52
57	Test Creating new secondary tenant	53
58	Test Adding a secondary tenant from the list	54
59	Test Edit secondary tenant information	55
60	Test Email notification "You were added as a secondary tenant"	56
61	Smoke, regression, and security testing for Account and Secondary Tenant Management	58;59
Gate and Access Control		
62	Design remote gate control dropdown with buttons for "Open Gate," "Close Gate," and "Alarm."	4
63	Design gates list associated with account	4
64	Display all gates associated with the account	2;63
65	Remote gate control - "Open Gate", "Close Gate", "Alarm"	3;62;64
66	Test Display all gates	64
67	Test Remote gate control - "Open Gate", "Close Gate", "Alarm"	65
68	Smoke, regression, and security testing for Gate and Access Control	66;67
Dashboard		
69	Design dashboard layout	4
70	Design notification display for updates (e.g., payment reminders, access updates)	69

ID	Task Name	Preceding Activity
71	Design quick action buttons for "Pay Balance," "Open Gate," and "Rent a Unit."	69
72	The list of rented units	2;69
73	Notifications (e.g., payment reminders, access updates).	2;70
74	Quick Action - "Pay Balance"	32;71
75	Quick Action - "Open Gate"	67;71
76	Quick Action - "Rent a Unit"	32;71
77	Test The list of rented units.	72
78	Test Notifications (e.g., payment reminders, access updates).	73
79	Test Quick action buttons ("Pay Balance," "Open Gate," "Rent a Unit")	74;75;76
80	Regression testing for dashboard functionalities.	77;78;79
81	Regression testing	80; 68; 61; 49; 35; 17; 18; 13

Appendix C - Estimated Duration of tasks (CPM)

ID	Task Name	Estimation in Human Days
1	Set up application infrastructure	7
2	Database Design and Management	8
3	Security Implementation	7
4	Visual Design phase	8
5	Design User Authentication and Authorization	1
6	User registration with secure password storage	3
7	User Login	2
8	Two-factor authentication	4
9	Password recovery	2
10	Test Password recovery	1
11	Test User Login	1
12	Test Two-factor authentication	1
13	Test User registration	2
14	Design Profile creation and settings	2
15	Profile creation and settings	3
16	Password update	2
17	Test Password update	1
18	Test Profile creation and settings	1
19	Design Account information display	2
20	Design Payment History with search and filter functionalities	3

ID	Task Name	Estimation in Human Days
21	Design Unit balances Payment Flow	4
22	Design Transaction Details	2
23	Account Information Display (name, customer number, address)	3
24	Payment History	4
25	Search, filter functionality	3
26	Integrate payment gateway	6
27	Unit balances Payment Flow	6
28	Transaction Details	3
29	Downloadable receipt feature (PDF)	2
30	Test account information display	1
31	Verify payment history functionality (rendering, search, filter, and pagination)	1
32	Test unit balance display and payment flow	1
33	Test transaction details rendering	1
34	Test downloadable receipt feature	1
35	Smoke, regression, and security testing for Account and Payment Management	3
36	Design Unit List Display	2
37	Design Search and Filter	1
38	Design Detailed Unit View (e.g., insurance, secondary tenants)	2
39	Design Visual Indicators	1
40	Unit List Display	3

ID	Task Name	Estimation in Human Days
41	Search and filter	3
42	Detailed view pages for each unit - Insurance, Secondary tenants	3
43	Visual indicators (e.g., color-coded badges for Rented, Delinquent, Reserved)	1
44	"Pay" button for each unit, linking to the payment gateway	2
45	Test Unit List Display	1
46	Test Search and Filter	1
47	Test Detailed View	1
48	Payment Button Testing	1
49	Smoke, regression, and security testing for Account and Unit Management	2
50	Design form for adding and creating a secondary tenant.	2
51	Design interface for editing secondary tenant information.	1
52	Design email notification template: "You were added as a secondary tenant."	1
53	Create new secondary tenant	4
54	Add secondary tenant from list	2
55	Edit secondary tenant information	1
56	Email notification "You were added as a secondary tenant"	2
57	Test Creating new secondary tenant	1
58	Test Adding a secondary tenant from the list	1
59	Test Edit secondary tenant information	1
60	Test Email notification "You were added as a secondary tenant"	1

ID	Task Name	Estimation in Human Days
61	Smoke, regression, and security testing for Account and Secondary Tenant Management	1
62	Design remote gate control dropdown with buttons for "Open Gate," "Close Gate," and "Alarm."	1
63	Design gates list associated with account	2
64	Display all gates associated with the account	3
65	Remote gate control - "Open Gate", "Close Gate", "Alarm"	4
66	Test Display all gates	1
67	Test Remote gate control - "Open Gate", "Close Gate", "Alarm"	2
68	Smoke, regression, and security testing for Gate and Access Control	3
69	Design dashboard layout	2
70	Design notification display for updates (e.g., payment reminders, access updates)	1
71	Design quick action buttons for "Pay Balance," "Open Gate," and "Rent a Unit."	1
72	The list of rented units	3
73	Notifications (e.g., payment reminders, access updates).	2
74	Quick Action - "Pay Balance"	1
75	Quick Action - "Open Gate"	1
76	Quick Action - "Rent a Unit"	1
77	Test The list of rented units.	2
78	Test Notifications (e.g., payment reminders, access updates).	2
79	Test Quick action buttons ("Pay Balance," "Open Gate," "Rent a Unit")	2

ID	Task Name	Estimation in Human Days
80	Regression testing for dashboard functionalities.	1
81	Regression testing	3

Appendix D - The Result of Early Start, Early Finish Calculation based on Tasks Durations (CPM)

ID	Task Name	ES_j	EF_j
1	Set up application infrastructure	0	7
2	Database Design and Management	7	15
3	Security Implementation	7	14
4	Visual Design phase	0	8
5	Design User Authentication and Authorization	8	9
6	User registration with secure password storage	15	18
7	User Login	18	20
8	Two-factor authentication	20	24
9	Password recovery	20	22
10	Test Password recovery	22	23
11	Test User Login	20	21
12	Test Two-factor authentication	24	25
13	Test User registration	18	20
14	Design Profile creation and settings	8	10
15	Profile creation and settings	10	13
16	Password update	25	27
17	Test Password update	27	28
18	Test Profile creation and settings	13	14
19	Design Account information display	8	10
20	Design Payment History with search and filter functionalities	10	13

ID	Task Name	ES_j	EF_j
21	Design Unit balances Payment Flow	13	17
22	Design Transaction Details	17	19
23	Account Information Display (name, customer number, address)	10	13
24	Payment History	13	17
25	Search, filter functionality	13	16
26	Integrate payment gateway	21	27
27	Unit balances Payment Flow	27	33
28	Transaction Details	33	36
29	Downloadable receipt feature (PDF)	36	38
30	Test account information display	13	14
31	Verify payment history functionality (rendering, search, filter, and pagination)	17	18
32	Test unit balance display and payment flow	33	34
33	Test transaction details rendering	36	37
34	Test downloadable receipt feature	38	39
35	Smoke, regression, and security testing for Account and Payment Management	39	42
36	Design Unit List Display	8	10
37	Design Search and Filter	10	11
38	Design Detailed Unit View (e.g., insurance, secondary tenants)	10	12
39	Design Visual Indicators	12	13
40	Unit List Display	15	18
41	Search and filter	18	21
42	Detailed view pages for each unit - Insurance, Secondary tenants	15	18

ID	Task Name	ES_j	EF_j
43	Visual indicators (e.g., color-coded badges for Rented, Delinquent, Reserved)	18	19
44	"Pay" button for each unit, linking to the payment gateway	27	29
45	Test Unit List Display	18	19
46	Test Search and Filter	21	22
47	Test Detailed View	18	19
48	Payment Button Testing	29	30
49	Smoke, regression, and security testing for Account and Unit Management	30	32
50	Design form for adding and creating a secondary tenant.	8	10
51	Design interface for editing secondary tenant information.	10	11
52	Design email notification template: "You were added as a secondary tenant."	8	9
53	Create new secondary tenant	15	19
54	Add secondary tenant from list	15	17
55	Edit secondary tenant information	19	20
56	Email notification "You were added as a secondary tenant"	9	11
57	Test Creating new secondary tenant	19	20
58	Test Adding a secondary tenant from the list	17	18
59	Test Edit secondary tenant information	20	21
60	Test Email notification "You were added as a secondary tenant"	11	12
61	Smoke, regression, and security testing for Account and Secondary Tenant Management	21	22
62	Design remote gate control dropdown with buttons for "Open Gate," "Close Gate," and "Alarm."	8	9
63	Design gates list associated with account	8	10

ID	Task Name	ES_j	EF_j
64	Display all gates associated with the account	15	18
65	Remote gate control - "Open Gate", "Close Gate", "Alarm"	18	22
66	Test Display all gates	18	19
67	Test Remote gate control - "Open Gate", "Close Gate", "Alarm"	22	24
68	Smoke, regression, and security testing for Gate and Access Control	24	27
69	Design dashboard layout	8	10
70	Design notification display for updates (e.g., payment reminders, access updates)	10	11
71	Design quick action buttons for "Pay Balance," "Open Gate," and "Rent a Unit."	10	11
72	The list of rented units	15	18
73	Notifications (e.g., payment reminders, access updates).	15	17
74	Quick Action - "Pay Balance"	34	35
75	Quick Action - "Open Gate"	24	25
76	Quick Action - "Rent a Unit"	34	35
77	Test The list of rented units.	18	20
78	Test Notifications (e.g., payment reminders, access updates).	17	19
79	Test Quick action buttons ("Pay Balance," "Open Gate," "Rent a Unit")	35	37
80	Regression testing for dashboard functionalities.	37	38
81	Regression testing	42	45

Appendix E - The Result of Late Start, Late Finish Calculation based on Tasks Durations (CPM)

ID	Task Name	LS_{ij}	LF_{ij}
1	Set up application infrastructure	0	7
2	Database Design and Management	7	15
3	Security Implementation	8	15
4	Visual Design phase	6	14
5	Design User Authentication and Authorization	14	15
6	User registration with secure password storage	15	18
7	User Login	18	20
8	Two-factor authentication	34	38
9	Password recovery	36	38
10	Test Password recovery	38	39
11	Test User Login	20	21
12	Test Two-factor authentication	38	39
13	Test User registration	40	42
14	Design Profile creation and settings	36	38
15	Profile creation and settings	38	41
16	Password update	39	41
17	Test Password update	41	42
18	Test Profile creation and settings	41	42
19	Design Account information display	22	24
20	Design Payment History with search and filter functionalities	24	27

ID	Task Name	LS_{ij}	LF_{ij}
21	Design Unit balances Payment Flow	27	31
22	Design Transaction Details	31	33
23	Account Information Display (name, customer number, address)	38	41
24	Payment History	34	38
25	Search, filter functionality	35	38
26	Integrate payment gateway	21	27
27	Unit balances Payment Flow	27	33
28	Transaction Details	33	36
29	Downloadable receipt feature (PDF)	36	38
30	Test account information display	41	42
31	Verify payment history functionality (rendering, search, filter, and pagination)	38	39
32	Test unit balance display and payment flow	37	38
33	Test transaction details rendering	38	39
34	Test downloadable receipt feature	38	39
35	Smoke, regression, and security testing for Account and Payment Management	39	42
36	Design Unit List Display	31	33
37	Design Search and Filter	35	36
38	Design Detailed Unit View (e.g., insurance, secondary tenants)	34	36
39	Design Visual Indicators	37	38
40	Unit List Display	33	36
41	Search and filter	36	39

ID	Task Name	LS_{ij}	LF_{ij}
42	Detailed view pages for each unit - Insurance, Secondary tenants	36	39
43	Visual indicators (e.g., color-coded badges for Rented, Delinquent, Reserved)	38	39
44	"Pay" button for each unit, linking to the payment gateway	37	39
45	Test Unit List Display	39	40
46	Test Search and Filter	39	40
47	Test Detailed View	39	40
48	Payment Button Testing	39	40
49	Smoke, regression, and security testing for Account and Unit Management	40	42
50	Design form for adding and creating a secondary tenant.	33	35
51	Design interface for editing secondary tenant information.	38	39
52	Design email notification template: "You were added as a secondary tenant."	37	38
53	Create new secondary tenant	35	39
54	Add secondary tenant from list	37	39
55	Edit secondary tenant information	39	40
56	Email notification "You were added as a secondary tenant"	38	40
57	Test Creating new secondary tenant	40	41
58	Test Adding a secondary tenant from the list	40	41
59	Test Edit secondary tenant information	40	41
60	Test Email notification "You were added as a secondary tenant"	40	41
61	Smoke, regression, and security testing for Account and Secondary Tenant Management	41	42
62	Design remote gate control dropdown with buttons for "Open Gate," "Close Gate," and "Alarm."	31	32

ID	Task Name	LS_{ij}	LF_{ij}
63	Design gates list associated with account	27	29
64	Display all gates associated with the account	29	32
65	Remote gate control - "Open Gate", "Close Gate", "Alarm"	32	36
66	Test Display all gates	38	39
67	Test Remote gate control - "Open Gate", "Close Gate", "Alarm"	36	38
68	Smoke, regression, and security testing for Gate and Access Control	39	42
69	Design dashboard layout	34	36
70	Design notification display for updates (e.g., payment reminders, access updates)	36	37
71	Design quick action buttons for "Pay Balance," "Open Gate," and "Rent a Unit."	37	38
72	The list of rented units	36	39
73	Notifications (e.g., payment reminders, access updates).	37	39
74	Quick Action - "Pay Balance"	38	39
75	Quick Action - "Open Gate"	38	39
76	Quick Action - "Rent a Unit"	38	39
77	Test The list of rented units.	39	41
78	Test Notifications (e.g., payment reminders, access updates).	39	41
79	Test Quick action buttons ("Pay Balance," "Open Gate," "Rent a Unit")	39	41
80	Regression testing for dashboard functionalities.	41	42
81	Regression testing	42	45

Appendix F - The result of Total Float/Slack Calculations based on Tasks Durations (CPM)

ID	Task Name	Total Float
1	Set up application infrastructure	0
2	Database Design and Management	0
3	Security Implementation	1
4	Visual Design phase	6
5	Design User Authentication and Authorization	6
6	User registration with secure password storage	0
7	User Login	0
8	Two-factor authentication	14
9	Password recovery	16
10	Test Password recovery	16
11	Test User Login	0
12	Test Two-factor authentication	14
13	Test User registration	22
14	Design Profile creation and settings	28
15	Profile creation and settings	28
16	Password update	14
17	Test Password update	14
18	Test Profile creation and settings	28
19	Design Account information display	14
20	Design Payment History with search and filter functionalities	14

ID	Task Name	Total Float
21	Design Unit balances Payment Flow	14
22	Design Transaction Details	14
23	Account Information Display (name, customer number, address)	28
24	Payment History	21
25	Search, filter functionality	22
26	Integrate payment gateway	0
27	Unit balances Payment Flow	0
28	Transaction Details	0
29	Downloadable receipt feature (PDF)	0
30	Test account information display	28
31	Verify payment history functionality (rendering, search, filter, and pagination)	21
32	Test unit balance display and payment flow	4
33	Test transaction details rendering	2
34	Test downloadable receipt feature	0
35	Smoke, regression, and security testing for Account and Payment Management	0
36	Design Unit List Display	23
37	Design Search and Filter	25
38	Design Detailed Unit View (e.g., insurance, secondary tenants)	24
39	Design Visual Indicators	25
40	Unit List Display	18
41	Search and filter	18
42	Detailed view pages for each unit - Insurance, Secondary tenants	21

ID	Task Name	Total Float
43	Visual indicators (e.g., color-coded badges for Rented, Delinquent, Reserved)	20
44	"Pay" button for each unit, linking to the payment gateway	10
45	Test Unit List Display	21
46	Test Search and Filter	18
47	Test Detailed View	21
48	Payment Button Testing	10
49	Smoke, regression, and security testing for Account and Unit Management	10
50	Design form for adding and creating a secondary tenant.	25
51	Design interface for editing secondary tenant information.	28
52	Design email notification template: "You were added as a secondary tenant."	29
53	Create new secondary tenant	20
54	Add secondary tenant from list	22
55	Edit secondary tenant information	20
56	Email notification "You were added as a secondary tenant"	29
57	Test Creating new secondary tenant	21
58	Test Adding a secondary tenant from the list	23
59	Test Edit secondary tenant information	20
60	Test Email notification "You were added as a secondary tenant"	29
61	Smoke, regression, and security testing for Account and Secondary Tenant Management	20
62	Design remote gate control dropdown with buttons for "Open Gate," "Close Gate," and "Alarm."	23
63	Design gates list associated with account	19

ID	Task Name	Total Float
64	Display all gates associated with the account	14
65	Remote gate control - "Open Gate", "Close Gate", "Alarm"	14
66	Test Display all gates	20
67	Test Remote gate control - "Open Gate", "Close Gate", "Alarm"	14
68	Smoke, regression, and security testing for Gate and Access Control	15
69	Design dashboard layout	26
70	Design notification display for updates (e.g., payment reminders, access updates)	26
71	Design quick action buttons for "Pay Balance," "Open Gate," and "Rent a Unit."	27
72	The list of rented units	21
73	Notifications (e.g., payment reminders, access updates).	22
74	Quick Action - "Pay Balance"	4
75	Quick Action - "Open Gate"	14
76	Quick Action - "Rent a Unit"	4
77	Test The list of rented units.	21
78	Test Notifications (e.g., payment reminders, access updates).	22
79	Test Quick action buttons ("Pay Balance," "Open Gate," "Rent a Unit")	4
80	Regression testing for dashboard functionalities.	4
81	Regression testing	0

Appendix G - Project Schedule based on CPM

ID	Name	Start	Finish	Resource Name
1	Set up application infrastructure	04.06.2024	12.06.2024	Developer 1
2	Database Design and Management	13.06.2024	24.06.2024	Developer 1
3	Security Implementation	13.06.2024	21.06.2024	Developer 2
4	Visual Design phase	04.06.2024	13.06.2024	Designer 1
5	Design User Authentication and Authorization	14.06.2024	14.06.2024	Designer 1
6	User registration with secure password storage	25.06.2024	27.06.2024	Developer 1
7	User Login	28.06.2024	01.07.2024	Developer 1
8	Two-factor authentication	02.07.2024	08.07.2024	Developer 3
9	Password recovery	03.07.2024	04.07.2024	Developer 4
10	Test Password recovery	08.07.2024	08.07.2024	Tester 1
11	Test User Login	02.07.2024	02.07.2024	Tester 1
12	Test Two-factor authentication	09.07.2024	09.07.2024	Tester 1
13	Test User registration	04.07.2024	08.07.2024	Tester 2
14	Design Profile creation and settings	14.06.2024	17.06.2024	Designer 2
15	Profile creation and settings	18.06.2024	20.06.2024	Developer 5
16	Password update	10.07.2024	11.07.2024	Developer 4
17	Test Password update	12.07.2024	12.07.2024	Tester 1
18	Test Profile creation and settings	21.06.2024	21.06.2024	Tester 1
19	Design Account information display	18.06.2024	19.06.2024	Designer 2
20	Design Payment History with search and filter functionalities	20.06.2024	24.06.2024	Designer 2

ID	Name	Start	Finish	Resource Name
21	Design Unit balances Payment Flow	25.06.2024	28.06.2024	Designer 2
22	Design Transaction Details	01.07.2024	02.07.2024	Designer 2
23	Account Information Display (name, customer number, address)	20.06.2024	24.06.2024	Developer 4
24	Payment History	25.06.2024	28.06.2024	Developer 4
25	Search, filter functionality	25.06.2024	27.06.2024	Developer 5
26	Integrate payment gateway	03.07.2024	11.07.2024	Developer 1
27	Unit balances Payment Flow	12.07.2024	19.07.2024	Developer 1
28	Transaction Details	22.07.2024	24.07.2024	Developer 1
29	Downloadable receipt feature (PDF)	25.07.2024	26.07.2024	Developer 1
30	Test account information display	26.06.2024	26.06.2024	Tester 1
31	Verify payment history functionality (rendering, search, filter, and pagination)	02.07.2024	02.07.2024	Tester 2
32	Test unit balance display and payment flow	22.07.2024	22.07.2024	Tester 2
33	Test transaction details rendering	26.07.2024	26.07.2024	Tester 2
34	Test downloadable receipt feature	29.07.2024	29.07.2024	Tester 1
35	Smoke, regression, and security testing for Account and Payment Management	30.07.2024	01.08.2024	Tester 1
36	Design Unit List Display	03.07.2024	04.07.2024	Designer 2
37	Design Search and Filter	08.07.2024	08.07.2024	Designer 2
38	Design Detailed Unit View (e.g., insurance, secondary tenants)	09.07.2024	10.07.2024	Designer 2
39	Design Visual Indicators	11.07.2024	11.07.2024	Designer 2
40	Unit List Display	09.07.2024	11.07.2024	Developer 2

ID	Name	Start	Finish	Resource Name
41	Search and filter	12.07.2024	16.07.2024	Developer 3
42	Detailed view pages for each unit - Insurance, Secondary tenants	12.07.2024	16.07.2024	Developer 2
43	Visual indicators (e.g., color-coded badges for Rented, Delinquent, Reserved)	12.07.2024	12.07.2024	Developer 5
44	"Pay" button for each unit, linking to the payment gateway	15.07.2024	17.07.2024	Developer 5
45	Test Unit List Display	12.07.2024	12.07.2024	Tester 1
46	Test Search and Filter	17.07.2024	17.07.2024	Tester 1
47	Test Detailed View	17.07.2024	17.07.2024	Tester 2
48	Payment Button Testing	23.07.2024	23.07.2024	Tester 2
49	Smoke, regression, and security testing for Account and Unit Management	25.07.2024	26.07.2024	Tester 1
50	Design form for adding and creating a secondary tenant.	17.06.2024	18.06.2024	Designer 1
51	Design interface for editing secondary tenant information.	19.06.2024	19.06.2024	Designer 1
52	Design email notification template: "You were added as a secondary tenant."	20.06.2024	20.06.2024	Designer 1
53	Create new secondary tenant	25.06.2024	28.06.2024	Developer 3
54	Add secondary tenant from list	01.07.2024	02.07.2024	Developer 4
55	Edit secondary tenant information	08.07.2024	08.07.2024	Developer 4
56	Email notification "You were added as a secondary tenant"	21.06.2024	24.06.2024	Developer 5
57	Test Creating new secondary tenant	03.07.2024	03.07.2024	Tester 2
58	Test Adding a secondary tenant from the list	03.07.2024	03.07.2024	Tester 1

ID	Name	Start	Finish	Resource Name
59	Test Edit secondary tenant information	12.07.2024	12.07.2024	Tester 2
60	Test Email notification "You were added as a secondary tenant"	04.07.2024	08.07.2024	Tester 2
61	Smoke, regression, and security testing for Account and Secondary Tenant Management	17.07.2024	17.07.2024	Tester 1
62	Design remote gate control dropdown with buttons for "Open Gate," "Close Gate," and "Alarm."	20.06.2024	20.06.2024	Designer 1
63	Design gates list associated with account	21.06.2024	24.06.2024	Designer 1
64	Display all gates associated with the account	25.06.2024	27.06.2024	Developer 2
65	Remote gate control - "Open Gate", "Close Gate", "Alarm"	28.06.2024	03.07.2024	Tester 2
66	Test Display all gates	09.07.2024	09.07.2024	Tester 2
67	Test Remote gate control - "Open Gate", "Close Gate", "Alarm"	04.07.2024	08.07.2024	Tester 2
68	Smoke, regression, and security testing for Gate and Access Control	23.07.2024	25.07.2024	Tester 1
69	Design dashboard layout	25.06.2024	26.06.2024	Designer 1
70	Design notification display for updates (e.g., payment reminders, access updates)	27.06.2024	27.06.2024	Designer 1
71	Design quick action buttons for "Pay Balance," "Open Gate," and "Rent a Unit."	27.06.2024	27.06.2024	Designer 1
72	The list of rented units	01.07.2024	03.07.2024	Developer 2
73	Notifications (e.g., payment reminders, access updates).	28.06.2024	01.07.2024	Developer 5
74	Quick Action - "Pay Balance"	23.07.2024	23.07.2024	Developer 2
75	Quick Action - "Open Gate"	09.07.2024	09.07.2024	Developer 3

ID	Name	Start	Finish	Resource Name
76	Quick Action - "Rent a Unit"	23.07.2024	23.07.2024	Developer 3
77	Test The list of rented units.	17.07.2024	18.07.2024	Tester 2
78	Test Notifications (e.g., payment reminders, access updates).	02.07.2024	03.07.2024	Tester 1
79	Test Quick action buttons ("Pay Balance," "Open Gate," "Rent a Unit")	24.07.2024	25.07.2024	Tester 2
80	Regression testing for dashboard functionalities.	26.07.2024	26.07.2024	Tester 1
81	Regression testing	02.08.2024	06.08.2024	Tester 1

Appendix H - Task Duration Estimates According to PERT: Optimistic, Most Likely, Pessimistic, and Expected Values

ID	Task Name	Human days (Optimistic)	Human days (ML)	Human days (Pessimistic)	PERT
1	Set up application infrastructure	5	7	11	7,33
2	Database Design and Management	6	8	12	8,50
3	Security Implementation	6	7	10	8,17
4	Visual Design phase	7	8	11	8,33
5	Design User Authentication and Authorization	1	1	2	1,17
6	User registration with secure password storage	2	3	5	3,17
7	User Login	1	2	4	2,17
8	Two-factor authentication	3	4	6	4,17
9	Password recovery	1	2	4	2,17
10	Test Password recovery	1	1	2	1,17
11	Test User Login	1	1	2	1,17
12	Test Two-factor authentication	1	1	3	1,33
13	Test User registration	1	2	4	2,17
14	Design Profile creation and settings	1	2	4	2,83
15	Profile creation and settings	1	3	6	2,50
16	Password update	1	2	5	1,67
17	Test Password update	1	1	3	1,33
18	Test Profile creation and settings	1	1	2	1,17

ID	Task Name	Human days (Optimistic)	Human days (ML)	Human days (Pessimistic)	PERT
19	Design Account information display	1	2	4	2,17
20	Design Payment History with search and filter functionalities	2	3	5	3,17
21	Design Unit balances Payment Flow	3	4	7	4,33
22	Design Transaction Details	1	2	3	2,00
23	Account Information Display (name, customer number, address)	2	4	5	3,83
24	Payment History	3	4	7	4,33
25	Search, filter functionality	2	3	5	3,17
26	Integrate payment gateway	6	6	8	6,33
27	Unit balances Payment Flow	4	6	7	5,83
28	Transaction Details	1	3	4	2,83
29	Downloadable receipt feature (PDF)	1	2	3	2,00
30	Test account information display	1	1	2	1,17
31	Verify payment history functionality (rendering, search, filter, and pagination)	1	1	3	1,33
32	Test unit balance display and payment flow	1	1	3	1,33
33	Test transaction details rendering	1	1	2	1,17
34	Test downloadable receipt feature	1	1	2	1,17
35	Smoke, regression, and security testing for Account and Payment Management	2	3	4	3,00
36	Design Unit List Display	1	3	5	3,00

ID	Task Name	Human days (Optimistic)	Human days (ML)	Human days (Pessimistic)	PERT
37	Design Search and Filter	1	1	3	1,33
38	Design Detailed Unit View (e.g., insurance, secondary tenants)	1	2	4	2,17
39	Design Visual Indicators	1	1	2	1,17
40	Unit List Display	2	3	6	3,33
41	Search and filter	2	3	6	3,33
42	Detailed view pages for each unit - Insurance, Secondary tenants	2	3	5	3,17
43	Visual indicators (e.g., color-coded badges for Rented, Delinquent, Reserved)	1	1	3	1,33
44	"Pay" button for each unit, linking to the payment gateway	1	2	5	2,33
45	Test Unit List Display	1	1	2	1,17
46	Test Search and Filter	1	1	2	1,17
47	Test Detailed View	1	1	2	1,17
48	Payment Button Testing	1	1	4	1,50
49	Smoke, regression, and security testing for Account and Unit Management	1	2	4	1,50
50	Design form for adding and creating a secondary tenant.	1	2	4	2,17
51	Design interface for editing secondary tenant information.	1	1	3	1,33
52	Design email notification template: "You were added as a secondary tenant."	1	1	3	1,33

ID	Task Name	Human days (Optimistic)	Human days (ML)	Human days (Pessimistic)	PERT
53	Create new secondary tenant	3	4	6	4,17
54	Add secondary tenant from list	1	2	4	2,17
55	Edit secondary tenant information	1	1	4	1,50
56	Email notification "You were added as a secondary tenant"	1	2	3	2,00
57	Test Creating new secondary tenant	1	1	3	1,33
58	Test Adding a secondary tenant from the list	1	1	2	1,17
59	Test Edit secondary tenant information	1	1	2	1,17
60	Test Email notification "You were added as a secondary tenant"	1	1	3	1,33
61	Smoke, regression, and security testing for Account and Secondary Tenant Management	1	1	4	2,17
62	Design remote gate control dropdown with buttons for "Open Gate," "Close Gate," and "Alarm."	1	1	2	1,17
63	Design gates list associated with account	1	2	3	2,00
64	Display all gates associated with the account	2	3	6	3,33
65	Remote gate control - "Open Gate", "Close Gate", "Alarm"	3	4	8	4,50
66	Test Display all gates	1	1	3	1,33
67	Test Remote gate control - "Open Gate", "Close Gate", "Alarm"	1	2	4	2,17

ID	Task Name	Human days (Optimistic)	Human days (ML)	Human days (Pessimistic)	PERT
68	Smoke, regression, and security testing for Gate and Access Control	2	3	5	3,17
69	Design dashboard layout	1	2	4	2,17
70	Design notification display for updates (e.g., payment reminders, access updates)	1	1	3	1,33
71	Design quick action buttons for "Pay Balance," "Open Gate," and "Rent a Unit."	1	1	3	1,33
72	The list of rented units	2	3	5	3,17
73	Notifications (e.g., payment reminders, access updates).	1	2	4	2,17
74	Quick Action - "Pay Balance"	1	1	3	1,33
75	Quick Action - "Open Gate"	1	1	3	1,33
76	Quick Action - "Rent a Unit"	1	1	3	1,33
77	Test The list of rented units.	1	2	3	2,00
78	Test Notifications (e.g., payment reminders, access updates).	1	2	3	2,00
79	Test Quick action buttons ("Pay Balance," "Open Gate," "Rent a Unit")	1	2	3	2,00
80	Regression testing for dashboard functionalities.	1	1	3	1,33
81	Regression testing	3	3	6	3,50

Appendix I - The Result of Early Start, Early Finish, Late Start, Late Finish Calculations Using PERT Expected durations of tasks

ID	Task Name	ES_{ij}	EF_{ij}	LS_{ij}	LF_{ij}
1	Set up application infrastructure	0,00	7,33	0,00	7,33
2	Database Design and Management	7,33	15,83	7,33	15,83
3	Security Implementation	7,33	15,50	7,67	15,83
4	Visual Design phase	0,00	8,33	6,33	14,67
5	Design User Authentication and Authorization	8,33	9,50	14,67	15,83
6	User registration with secure password storage	15,83	19,00	15,83	19,00
7	User Login	19,00	21,17	19,00	21,17
8	Two-factor authentication	21,17	25,33	35,00	39,17
9	Password recovery	21,17	23,33	37,17	39,33
10	Test Password recovery	23,33	24,50	39,33	40,50
11	Test User Login	21,17	22,33	21,17	22,33
12	Test Two-factor authentication	25,33	26,67	39,17	40,50
13	Test User registration	19,00	21,17	41,33	43,50
14	Design Profile creation and settings	8,33	11,17	37,00	39,83
15	Profile creation and settings	11,17	13,67	39,83	42,33
16	Password update	26,67	28,33	40,50	42,17
17	Test Password update	28,33	29,67	42,17	43,50
18	Test Profile creation and settings	13,67	14,83	42,33	43,50
19	Design Account information display	8,33	10,50	22,83	25,00

ID	Task Name	ES_{ij}	EF_{ij}	LS_{ij}	LF_{ij}
20	Design Payment History with search and filter functionalities	10,50	13,67	25,00	28,17
21	Design Unit balances Payment Flow	13,67	18,00	28,17	32,50
22	Design Transaction Details	18,00	20,00	32,50	34,50
23	Account Information Display (name, customer number, address)	10,50	14,33	38,50	42,33
24	Payment History	13,67	18,00	34,83	39,17
25	Search, filter functionality	13,67	16,83	36,00	39,17
26	Integrate payment gateway	22,33	28,67	22,33	28,67
27	Unit balances Payment Flow	28,67	34,50	28,67	34,50
28	Transaction Details	34,50	37,33	34,50	37,33
29	Downloadable receipt feature (PDF)	37,33	39,33	37,33	39,33
30	Test account information display	14,33	15,50	42,33	43,50
31	Verify payment history functionality (rendering, search, filter, and pagination)	18,00	19,33	39,17	40,50
32	Test unit balance display and payment flow	34,50	35,83	37,50	38,83
33	Test transaction details rendering	37,33	38,50	39,33	40,50
34	Test downloadable receipt feature	39,33	40,50	39,33	40,50
35	Smoke, regression, and security testing for Account and Payment Management	40,50	43,50	40,50	43,50
36	Design Unit List Display	8,33	11,33	31,17	34,17
37	Design Search and Filter	11,33	12,67	36,17	37,50
38	Design Detailed Unit View (e.g., insurance, secondary tenants)	11,33	13,50	35,50	37,67
39	Design Visual Indicators	13,50	14,67	38,00	39,17

ID	Task Name	ES_{ij}	EF_{ij}	LS_{ij}	LF_{ij}
40	Unit List Display	15,83	19,17	34,17	37,50
41	Search and filter	19,17	22,50	37,50	40,83
42	Detailed view pages for each unit - Insurance, Secondary tenants	15,83	19,00	37,67	40,83
43	Visual indicators (e.g., color-coded badges for Rented, Delinquent, Reserved)	19,17	20,50	39,17	40,50
44	"Pay" button for each unit, linking to the payment gateway	28,67	31,00	38,17	40,50
45	Test Unit List Display	19,17	20,33	40,83	42,00
46	Test Search and Filter	22,50	23,67	40,83	42,00
47	Test Detailed View	19,00	20,17	40,83	42,00
48	Payment Button Testing	31,00	32,50	40,50	42,00
49	Smoke, regression, and security testing for Account and Unit Management	32,50	34,00	42,00	43,50
50	Design form for adding and creating a secondary tenant.	8,33	10,50	32,33	34,50
51	Design interface for editing secondary tenant information.	10,50	11,83	37,33	38,67
52	Design email notification template: "You were added as a secondary tenant."	8,33	9,67	36,67	38,00
53	Create new secondary tenant	15,83	20,00	34,50	38,67
54	Add secondary tenant from list	15,83	18,00	36,50	38,67
55	Edit secondary tenant information	20,00	21,50	38,67	40,17
56	Email notification "You were added as a secondary tenant"	9,67	11,67	38,00	40,00
57	Test Creating new secondary tenant	20,00	21,33	40,00	41,33
58	Test Adding a secondary tenant from the list	18,00	19,17	40,17	41,33
59	Test Edit secondary tenant information	21,50	22,67	40,17	41,33

ID	Task Name	ES_{ij}	EF_{ij}	LS_{ij}	LF_{ij}
60	Test Email notification "You were added as a secondary tenant"	11,67	13,00	40,00	41,33
61	Smoke, regression, and security testing for Account and Secondary Tenant Management	22,67	24,83	41,33	43,50
62	Design remote gate control dropdown with buttons for "Open Gate," "Close Gate," and "Alarm."	8,33	9,50	31,00	32,17
63	Design gates list associated with account	8,33	10,33	26,83	28,83
64	Display all gates associated with the account	15,83	19,17	28,83	32,17
65	Remote gate control - "Open Gate", "Close Gate", "Alarm"	19,17	23,67	32,17	36,67
66	Test Display all gates	19,17	20,50	39,00	40,33
67	Test Remote gate control - "Open Gate", "Close Gate", "Alarm"	23,67	25,83	36,67	38,83
68	Smoke, regression, and security testing for Gate and Access Control	25,83	29,00	40,33	43,50
69	Design dashboard layout	8,33	10,50	34,50	36,67
70	Design notification display for updates (e.g., payment reminders, access updates)	10,50	11,83	36,67	38,00
71	Design quick action buttons for "Pay Balance," "Open Gate," and "Rent a Unit."	10,50	11,83	37,50	38,83
72	The list of rented units	15,83	19,00	37,00	40,17
73	Notifications (e.g., payment reminders, access updates).	15,83	18,00	38,00	40,17
74	Quick Action - "Pay Balance"	35,83	37,17	38,83	40,17
75	Quick Action - "Open Gate"	25,83	27,17	38,83	40,17
76	Quick Action - "Rent a Unit"	35,83	37,17	38,83	40,17
77	Test The list of rented units.	19,00	21,00	40,17	42,17
78	Test Notifications (e.g., payment reminders, access updates).	18,00	20,00	40,17	42,17

ID	Task Name	ES_{ij}	EF_{ij}	LS_{ij}	LF_{ij}
79	Test Quick action buttons ("Pay Balance," "Open Gate," "Rent a Unit")	37,17	39,17	40,17	42,17
80	Regression testing for dashboard functionalities.	39,17	40,50	42,17	43,50
81	Regression testing	43,50	47,00	43,50	47,00

Appendix J - The Result of Total Float/Slack Calculations (PERT)

ID	Task Name	Total Float
1	Set up application infrastructure	0,00
2	Database Design and Management	0,00
3	Security Implementation	0,33
4	Visual Design phase	6,33
5	Design User Authentication and Authorization	6,33
6	User registration with secure password storage	0,00
7	User Login	0,00
8	Two-factor authentication	13,83
9	Password recovery	16,00
10	Test Password recovery	16,00
11	Test User Login	0,00
12	Test Two-factor authentication	13,83
13	Test User registration	22,33
14	Design Profile creation and settings	28,67
15	Profile creation and settings	28,67
16	Password update	13,83
17	Test Password update	13,83
18	Test Profile creation and settings	28,67
19	Design Account information display	14,50
20	Design Payment History with search and filter functionalities	14,50

ID	Task Name	Total Float
21	Design Unit balances Payment Flow	14,50
22	Design Transaction Details	14,50
23	Account Information Display (name, customer number, address)	28,00
24	Payment History	21,17
25	Search, filter functionality	22,33
26	Integrate payment gateway	0,00
27	Unit balances Payment Flow	0,00
28	Transaction Details	0,00
29	Downloadable receipt feature (PDF)	0,00
30	Test account information display	28,00
31	Verify payment history functionality (rendering, search, filter, and pagination)	21,17
32	Test unit balance display and payment flow	3,00
33	Test transaction details rendering	2,00
34	Test downloadable receipt feature	0,00
35	Smoke, regression, and security testing for Account and Payment Management	0,00
36	Design Unit List Display	22,83
37	Design Search and Filter	24,83
38	Design Detailed Unit View (e.g., insurance, secondary tenants)	24,17
39	Design Visual Indicators	24,50
40	Unit List Display	18,33
41	Search and filter	18,33
42	Detailed view pages for each unit - Insurance, Secondary tenants	21,83

ID	Task Name	Total Float
43	Visual indicators (e.g., color-coded badges for Rented, Delinquent, Reserved)	20,00
44	"Pay" button for each unit, linking to the payment gateway	9,50
45	Test Unit List Display	21,67
46	Test Search and Filter	18,33
47	Test Detailed View	21,83
48	Payment Button Testing	9,50
49	Smoke, regression, and security testing for Account and Unit Management	9,50
50	Design form for adding and creating a secondary tenant.	24,00
51	Design interface for editing secondary tenant information.	26,83
52	Design email notification template: "You were added as a secondary tenant."	28,33
53	Create new secondary tenant	18,67
54	Add secondary tenant from list	20,67
55	Edit secondary tenant information	18,67
56	Email notification "You were added as a secondary tenant"	28,33
57	Test Creating new secondary tenant	20,00
58	Test Adding a secondary tenant from the list	22,17
59	Test Edit secondary tenant information	18,67
60	Test Email notification "You were added as a secondary tenant"	28,33
61	Smoke, regression, and security testing for Account and Secondary Tenant Management	18,67
62	Design remote gate control dropdown with buttons for "Open Gate," "Close Gate," and "Alarm."	22,67
63	Design gates list associated with account	18,50

ID	Task Name	Total Float
64	Display all gates associated with the account	13,00
65	Remote gate control - "Open Gate", "Close Gate", "Alarm"	13,00
66	Test Display all gates	19,83
67	Test Remote gate control - "Open Gate", "Close Gate", "Alarm"	13,00
68	Smoke, regression, and security testing for Gate and Access Control	14,50
69	Design dashboard layout	26,17
70	Design notification display for updates (e.g., payment reminders, access updates)	26,17
71	Design quick action buttons for "Pay Balance," "Open Gate," and "Rent a Unit."	27,00
72	The list of rented units	21,17
73	Notifications (e.g., payment reminders, access updates).	22,17
74	Quick Action - "Pay Balance"	3,00
75	Quick Action - "Open Gate"	13,00
76	Quick Action - "Rent a Unit"	3,00
77	Test The list of rented units.	21,17
78	Test Notifications (e.g., payment reminders, access updates).	22,17
79	Test Quick action buttons ("Pay Balance," "Open Gate," "Rent a Unit")	3,00
80	Regression testing for dashboard functionalities.	3,00
81	Regression testing	0,00

Appendix K - Project Schedule based on PERT

ID	Name	Start	Finish	Resource Name
1	Set up application infrastructure	04.06.2024	13.06.2024	Developer 1
2	Database Design and Management	13.06.2024	25.06.2024	Developer 1
3	Security Implementation	13.06.2024	25.06.2024	Developer 2
4	Visual Design phase	04.06.2024	14.06.2024	Designer 1
5	Design User Authentication and Authorization	14.06.2024	17.06.2024	Designer 1
6	User registration with secure password storage	25.06.2024	28.06.2024	Developer 1
7	User Login	01.07.2024	03.07.2024	Developer 1
8	Two-factor authentication	03.07.2024	09.07.2024	Developer 3
9	Password recovery	08.07.2024	10.07.2024	Developer 2
10	Test Password recovery	11.07.2024	15.07.2024	Tester 1
11	Test User Login	03.07.2024	04.07.2024	Tester 1
12	Test Two-factor authentication	09.07.2024	10.07.2024	Tester 2
13	Test User registration	01.07.2024	03.07.2024	Tester 2
14	Design Profile creation and settings	14.06.2024	19.06.2024	Designer 2
15	Profile creation and settings	19.06.2024	21.06.2024	Developer 5
16	Password update	15.07.2024	16.07.2024	Developer 4
17	Test Password update	16.07.2024	18.07.2024	Tester 1
18	Test Profile creation and settings	21.06.2024	25.06.2024	Tester 1
19	Design Account information display	19.06.2024	21.06.2024	Designer 2

ID	Name	Start	Finish	Resource Name
20	Design Payment History with search and filter functionalities	21.06.2024	26.06.2024	Designer 2
21	Design Unit balances Payment Flow	26.06.2024	03.07.2024	Designer 2
22	Design Transaction Details	03.07.2024	05.07.2024	Designer 2
23	Account Information Display (name, customer number, address)	21.06.2024	27.06.2024	Developer 4
24	Payment History	27.06.2024	03.07.2024	Developer 2
25	Search, filter functionality	27.06.2024	02.07.2024	Developer 5
26	Integrate payment gateway	04.07.2024	15.07.2024	Developer 1
27	Unit balances Payment Flow	15.07.2024	23.07.2024	Developer 1
28	Transaction Details	23.07.2024	26.07.2024	Developer 1
29	Downloadable receipt feature (PDF)	26.07.2024	30.07.2024	Developer 1
30	Test account information display	27.06.2024	28.06.2024	Tester 1
31	Verify payment history functionality (rendering, search, filter, and pagination)	03.07.2024	04.07.2024	Tester 2
32	Test unit balance display and payment flow	24.07.2024	25.07.2024	Tester 2
33	Test transaction details rendering	26.07.2024	29.07.2024	Tester 2
34	Test downloadable receipt feature	30.07.2024	31.07.2024	Tester 1
35	Smoke, regression, and security testing for Account and Payment Management	31.07.2024	05.08.2024	Tester 1
36	Design Unit List Display	05.07.2024	10.07.2024	Designer 2
37	Design Search and Filter	10.07.2024	11.07.2024	Designer 2
38	Design Detailed Unit View (e.g., insurance, secondary tenants)	11.07.2024	15.07.2024	Designer 2

ID	Name	Start	Finish	Resource Name
39	Design Visual Indicators	15.07.2024	16.07.2024	Designer 2
40	Unit List Display	10.07.2024	15.07.2024	Developer 2
41	Search and filter	15.07.2024	18.07.2024	Developer 3
42	Detailed view pages for each unit - Insurance, Secondary tenants	22.07.2024	25.07.2024	Developer 3
43	Visual indicators (e.g., color-coded badges for Rented, Delinquent, Reserved)	18.07.2024	19.07.2024	Developer 2
44	"Pay" button for each unit, linking to the payment gateway	15.07.2024	17.07.2024	Developer 2
45	Test Unit List Display	16.07.2024	17.07.2024	Tester 2
46	Test Search and Filter	22.07.2024	24.07.2024	Tester 1
47	Test Detailed View	25.07.2024	26.07.2024	Tester 2
48	Payment Button Testing	23.07.2024	24.07.2024	Tester 2
49	Smoke, regression, and security testing for Account and Unit Management	01.08.2024	02.08.2024	Tester 2
50	Design form for adding and creating a secondary tenant.	17.06.2024	19.06.2024	Designer 1
51	Design interface for editing secondary tenant information.	20.06.2024	21.06.2024	Designer 1
52	Design email notification template: "You were added as a secondary tenant."	21.06.2024	24.06.2024	Designer 1
53	Create new secondary tenant	25.06.2024	01.07.2024	Developer 3
54	Add secondary tenant from list	28.06.2024	02.07.2024	Developer 4
55	Edit secondary tenant information	02.07.2024	03.07.2024	Developer 4
56	Email notification "You were added as a secondary tenant"	24.06.2024	26.06.2024	Developer 5

ID	Name	Start	Finish	Resource Name
57	Test Creating new secondary tenant	05.07.2024	08.07.2024	Tester 2
58	Test Adding a secondary tenant from the list	02.07.2024	03.07.2024	Tester 1
59	Test Edit secondary tenant information	04.07.2024	08.07.2024	Tester 1
60	Test Email notification "You were added as a secondary tenant"	26.06.2024	28.06.2024	Tester 2
61	Smoke, regression, and security testing for Account and Secondary Tenant Management	08.07.2024	10.07.2024	Tester 1
62	Design remote gate control dropdown with buttons for 'Open Gate,' 'Close Gate,' and 'Alarm.'	25.06.2024	26.06.2024	Designer 1
63	Design gates list associated with account	26.06.2024	28.06.2024	Designer 1
64	Display all gates associated with the account	03.07.2024	09.07.2024	Developer 4
65	Remote gate control - 'Open Gate', 'Close Gate', 'Alarm'	12.07.2024	18.07.2024	Tester 2
66	Test Display all gates	10.07.2024	11.07.2024	Tester 1
67	Test Remote gate control - 'Open Gate', 'Close Gate', 'Alarm'	19.07.2024	23.07.2024	Tester 2
68	Smoke, regression, and security testing for Gate and Access Control	24.07.2024	30.07.2024	Tester 1
69	Design dashboard layout	28.06.2024	02.07.2024	Designer 1
70	Design notification display for updates (e.g., payment reminders, access updates)	02.07.2024	03.07.2024	Designer 1
71	Design quick action buttons for 'Pay Balance,' 'Open Gate,' and 'Rent a Unit.'	03.07.2024	05.07.2024	Designer 1
72	The list of rented units	02.07.2024	05.07.2024	Developer 5

ID	Name	Start	Finish	Resource Name
73	Notifications (e.g., payment reminders, access updates)	03.07.2024	05.07.2024	Developer 2
74	Quick Action - 'Pay Balance'	26.07.2024	29.07.2024	Developer 2
75	Quick Action - 'Open Gate'	24.07.2024	25.07.2024	Developer 3
76	Quick Action - 'Rent a Unit'	26.07.2024	29.07.2024	Developer 3
77	Test The list of rented units.	09.07.2024	11.07.2024	Tester 2
78	Test Notifications (e.g., payment reminders, access updates).	18.07.2024	22.07.2024	Tester 1
79	Test Quick action buttons ('Pay Balance,' 'Open Gate,' 'Rent a Unit')	29.07.2024	31.07.2024	Tester 2
80	Regression testing for dashboard functionalities.	31.07.2024	01.08.2024	Tester 2
81	Regression testing	06.08.2024	09.08.2024	Tester 1

Appendix L. Software Implementation of Monte Carlo Simulation and Research

The source code presented in this appendix is the intellectual property of Limestone Digital s.r.o. and is used here with permission for academic purposes only. Some code comments may appear in Ukrainian, reflecting the internal development practices of the company.

```
Critical Path (PERT): 46.83 days
Critical Path Tasks (PERT): [1, 2, 6, 7, 11, 26, 27, 28, 29, 34, 35, 81]

Critical Path (Most Likely): 45.00 days
Critical Path Tasks (Most Likely): [1, 2, 6, 7, 11, 26, 27, 28, 29, 34, 35, 81]

Critical Path Tasks (PERT):
 Task ID  Start  Finish  Duration (PERT)
       1   0.00    7.33             7.33
       2   7.33   15.67             8.33
       6  15.67   18.83             3.17
       7  18.83   21.00             2.17
      11  21.00   22.17             1.17
      26  22.17   28.50             6.33
      27  28.50   34.33             5.83
      28  34.33   37.17             2.83
      29  37.17   39.17             2.00
      34  39.17   40.33             1.17
      35  40.33   43.33             3.00
      81  43.33   46.83             3.50

Critical Path Tasks (Most Likely):
 Task ID  Start  Finish  Duration (Most Likely)
       1    0.0     7.0                     7.0
       2    7.0    15.0                     8.0
       6   15.0    18.0                     3.0
       7   18.0    20.0                     2.0
      11   20.0    21.0                     1.0
      26   21.0    27.0                     6.0
      27   27.0    33.0                     6.0
      28   33.0    36.0                     3.0
      29   36.0    38.0                     2.0
      34   38.0    39.0                     1.0
      35   39.0    42.0                     3.0
      81   42.0    45.0                     3.0

--- Comparison Summary ---
PERT Critical Path Duration: 46.83 days
Most Likely Critical Path Duration: 45.00 days
Difference (Most Likely - PERT): -1.83 days
The sequence of tasks in the critical path is the SAME for both methods.
```

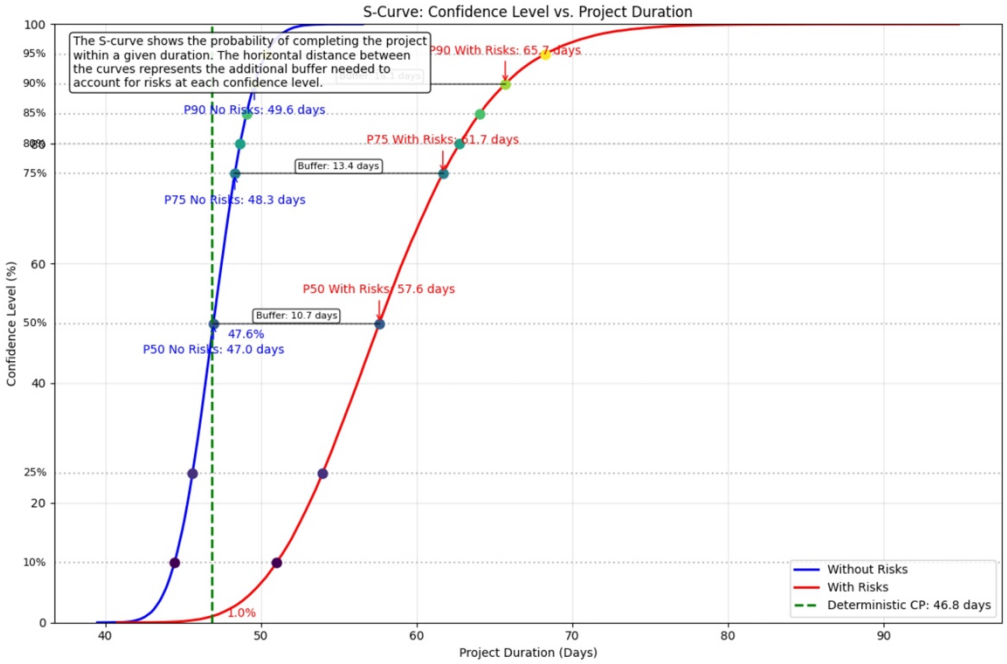
```
print(f"Critical Path (PERT): {proj_finish_pert:.2f} days")
print(f"Critical Path Tasks (PERT): {cp_tasks_pert}")

print(f"\nCritical Path (Most Likely): {proj_finish_ml:.2f} days")
print(f"Critical Path Tasks (Most Likely): {cp_tasks_ml}")

# Display tables for comparison

# PERT Critical Path Table
table_data_pert = []
for tid in cp_tasks_pert:
    task = all_tasks_pert[tid]
    table_data_pert.append({
        'Task ID': tid,
        'Start': task['start'],
        'Finish': task['finish'],
        'Duration (PERT)': task['duration']
    })
df_cp_pert = pd.DataFrame(table_data_pert)
df_cp_pert['Start'] = df_cp_pert['Start'].round(2)
df_cp_pert['Finish'] = df_cp_pert['Finish'].round(2)
df_cp_pert['Duration (PERT)'] = df_cp_pert['Duration (PERT)'].round(2)

print("\nCritical Path Tasks (PERT):")
print(df_cp_pert.to_string(index=False))

# Most Likely Critical Path Table
table_data_ml = []
for tid in cp_tasks_ml:
    task = all_tasks_ml[tid]
    table_data_ml.append({
        'Task ID': tid,
        'Start': task['start'],
        'Finish': task['finish'],
        'Duration (Most Likely)': task['duration']
    })
df_cp_ml = pd.DataFrame(table_data_ml)
df_cp_ml['Start'] = df_cp_ml['Start'].round(2)
df_cp_ml['Finish'] = df_cp_ml['Finish'].round(2)
df_cp_ml['Duration (Most Likely)'] = df_cp_ml['Duration (Most Likely)'].round(2)

print("\nCritical Path Tasks (Most Likely):")
print(df_cp_ml.to_string(index=False))

# Comparison Summary
print("\n--- Comparison Summary ---")
print(f"PERT Critical Path Duration: {proj_finish_pert:.2f} days")
print(f"Most Likely Critical Path Duration: {proj_finish_ml:.2f} days")
difference = proj_finish_ml - proj_finish_pert
print(f"Difference (Most Likely - PERT): {difference:.2f} days")

if cp_tasks_pert == cp_tasks_ml:
    print("The sequence of tasks in the critical path is the SAME for both methods.")
else:
    print("The sequence of tasks in the critical path is DIFFERENT between the two me
    print(f"  PERT CP Tasks: {cp_tasks_pert}")
    print(f"  Most Likely CP Tasks: {cp_tasks_ml}")
```

```
        dep_str = str(row.get('Previous_Activity', '')).strip()
        if dep_str == "" or dep_str.lower() == "nan":
            dependencies = []
        else:
            try:
                dependencies = [int(x.strip()) for x in dep_str.split(';') if x.strip
            except Exception as e:
                raise ValueError(f"Error processing Previous_Activity for task {task_

        tasks[task_id] = {
            "duration": duration,
            "dependencies": dependencies,
            "start": None,
            "finish": None
        }

    remaining = set(tasks.keys())
    while remaining:
        progress = False
        for t in list(remaining):
            deps = tasks[t]["dependencies"]
            if all(dep in tasks and tasks[dep]["finish"] is not None for dep in deps)
                start_time = max((tasks[dep]["finish"] for dep in deps), default=0)
                tasks[t]["start"] = start_time
                tasks[t]["finish"] = start_time + tasks[t]["duration"]
                remaining.remove(t)
                progress = True
        if not progress:
            raise ValueError("Cyclic dependencies or invalid Previous_Activity detect

    project_finish_ml = max(task["finish"] for task in tasks.values())

    # Backtracking to get a critical path for Most Likely
    cp_ml = []
    candidate_ml = None
    for tid, t in tasks.items():
         if abs(t["finish"] - project_finish_ml) < 1e-6:
            candidate_ml = tid
            break
    if candidate_ml is None:
        raise ValueError("No task found with finish equal to project finish for Most

    current_ml = candidate_ml
    while True:
         cp_ml.insert(0, current_ml)
         deps_ml = tasks[current_ml]["dependencies"]
         if not deps_ml:
             break
         # Find the predecessor that finishes latest to trace back the critical path
         current_ml = max(deps_ml, key=lambda d: tasks[d]["finish"] if tasks[d]["fini
         # Ensure the selected predecessor is actually part of a path that could lead
         if not any(abs(tasks[d]["finish"] + tasks[current_ml]["duration"] - tasks[cu
             # If no direct predecessor seems to be the cause, it might be a task wit
             # For simplicity, if multiple predecessors finish at the same time, this
             # A more robust CP algorithm might be needed for complex graphs.
             pass # Keep current_ml as is if logic is complex, or break if it implies

    return project_finish_ml, cp_ml, tasks

# Calculate Critical Path using PERT (already done in a previous cell, but let's call
proj_finish_pert, cp_tasks_pert, all_tasks_pert = compute_critical_path_details()

# Calculate Critical Path using Most Likely estimates
proj_finish_ml, cp_tasks_ml, all_tasks_ml = compute_critical_path_most_likely_details
```

```
    if not unique_risks_list:
        print("Не знайдено ризиків у 'risk_mapping'.")
        return

    df_final_risks = pd.DataFrame(unique_risks_list)

    print("Фінальна таблиця унікальних ризиків (з округленими значеннями):")
    # Виведення з форматуванням Fuzzy значень як списків
    print(df_final_risks[['Risk_Name', 'Fuzzy_Probability', 'Fuzzy_Impact', 'Category

    # Експорт DataFrame у CSV файл
    try:
        df_final_risks.to_csv('final_risk_table.csv', index=False)
        print("\nФінальну таблицю ризиків експортовано у 'final_risk_table.csv'")
    except Exception as e:
        print(f"\nПомилка під час експорту у CSV: {e}")

# Переконайтеся, що risk_mapping існує перед викликом функції
if 'risk_mapping' in globals():
    display_final_risk_table(risk_mapping)
else:
    print("Змінна 'risk_mapping' не визначена. Будь ласка, запустіть попередні комірк
```

```
Фінальна таблиця унікальних ризиків (з округленими значеннями):

Risk_Name     Fuzzy_Probability          Fuzzy_Impact                Category
Unplanned absences due to illness, resignation, or burnout may reduce available resour
ces and delay progress.       [0.0, 0.0, 0.2]       [0.0, 0.0, 0.2]    Resource Constr
aints
                                                                                      E
xternal API Instability [0.004, 0.006, 0.204]  [0.236, 0.32, 0.436] Third-Party Integr
ation

Unforeseen Limitations    [0.32, 0.41, 0.54] [0.463, 0.598, 0.689]               Techn
ical

New Setup Requirements    [0.36, 0.47, 0.58] [0.304, 0.406, 0.509]               Techn
ical
                                                                  Mandatory Library Update
Due to Vulnerabilities [0.278, 0.378, 0.478]   [0.22, 0.303, 0.42] Third-Party Integra
tion
                                                                               Unforesee
n Security Requirements [0.125, 0.168, 0.325] [0.338, 0.452, 0.559]                Sec
urity
                                                                                  Gate A
ccess Hardware Failures    [0.32, 0.43, 0.54]  [0.31, 0.414, 0.517]                Har
dware
                                                                             Unannounced
Client Hardware Updates    [0.04, 0.06, 0.24] [0.269, 0.369, 0.469]                Har
dware
                                                                                   Unfo
reseen Database Changes    [0.28, 0.38, 0.48] [0.389, 0.507, 0.625]                Tech
nical

Фінальну таблицю ризиків експортовано у 'final_risk_table.csv'
```

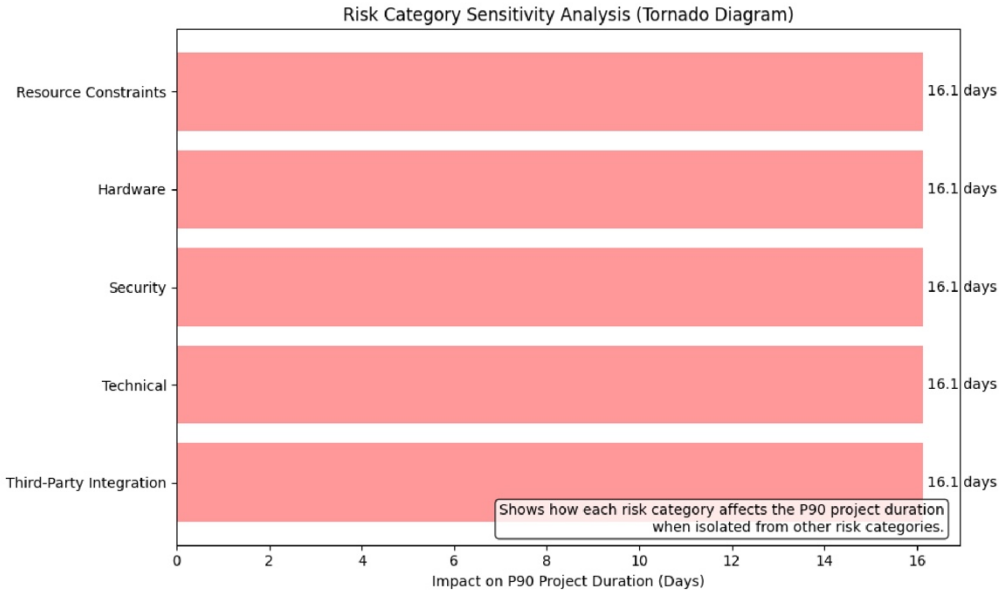
In [45]:
```
# Function to compute critical path using Most Likely estimates
def compute_critical_path_most_likely_details():
    tasks = {}
    for idx, row in backlog.iterrows():
        task_id = int(row['ID'])
        try:
            # Use Most_Likely for duration
            duration = float(str(row['Most_Likely']).replace(',', '.'))
        except Exception as e:
            raise ValueError(f"Error processing task {task_id} for Most Likely durati
```



In [44]:

```python
# Код для нової комірки Jupyter Notebook

def display_final_risk_table(risk_map):
    """
    Збирає унікальні ризики з risk_mapping, виводить їх у вигляді таблиці та експорту
    Значення Fuzzy_Probability та Fuzzy_Impact округлюються до 3 знаків.
    """
    unique_risks_list = []
    seen_risk_names = set()

    # Функція для додавання унікального ризику
    def process_risk(risk_item):
        risk_name = risk_item['Risk_Name']
        if risk_name not in seen_risk_names:
            # Округлення значень у Fuzzy_Probability та Fuzzy_Impact
            rounded_probability = [round(p, 3) for p in risk_item['Fuzzy_Probability']
            rounded_impact = [round(i, 3) for i in risk_item['Fuzzy_Impact']]

            unique_risks_list.append({
                'Risk_Name': risk_name,
                'Fuzzy_Probability': rounded_probability,
                'Fuzzy_Impact': rounded_impact,
                'Category': risk_item['Risk_Category']
            })
            seen_risk_names.add(risk_name)

    # Обробка загальнопроектних ризиків
    if 'project_wide' in risk_map:
        for risk in risk_map['project_wide']:
            process_risk(risk)

    # Обробка ризиків, специфічних для задач
    for task_id, risks_for_task in risk_map.items():
        if task_id == 'project_wide':
            continue
        for risk in risks_for_task:
            process_risk(risk)
```

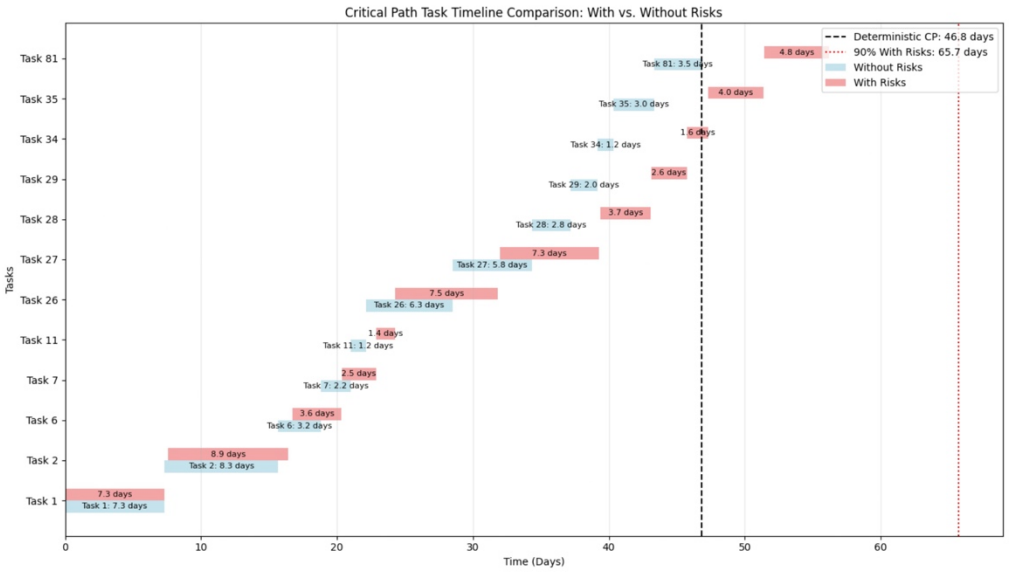
```
        if cl in [50, 75, 90]:
            buffer = duration_with_risks - duration_no_risks
            ax.annotate(f"P{cl} No Risks: {duration_no_risks:.1f} days",
                        xy=(duration_no_risks, cl),
                        xytext=(duration_no_risks, cl-5),
                        arrowprops=dict(arrowstyle="->", color='blue'),
                        color='blue', ha='center')

            ax.annotate(f"P{cl} With Risks: {duration_with_risks:.1f} days",
                        xy=(duration_with_risks, cl),
                        xytext=(duration_with_risks, cl+5),
                        arrowprops=dict(arrowstyle="->", color='red'),
                        color='red', ha='center')

            # Draw a horizontal line between the two points to show buffer
            ax.plot([duration_no_risks, duration_with_risks], [cl, cl], 'k-', alpha=0
            # Add text in the middle to show buffer size
            mid_x = (duration_no_risks + duration_with_risks) / 2
            ax.text(mid_x, cl + 0.5, f"Buffer: {buffer:.1f} days", ha='center', va='b
                   bbox=dict(boxstyle='round', facecolor='white', alpha=0.8))

    # Add the deterministic duration
    deterministic_cp = compute_critical_path()
    ax.axvline(x=deterministic_cp, color='green', linestyle='--', linewidth=2,
               label=f'Deterministic CP: {deterministic_cp:.1f} days')

    # Calculate the probability of meeting the deterministic duration
    prob_no_risks = np.mean(results_no_risks <= deterministic_cp) * 100
    prob_with_risks = np.mean(results_with_risks <= deterministic_cp) * 100

    # Add annotations for these probabilities
    ax.annotate(f"{prob_no_risks:.1f}%",
                xy=(deterministic_cp, prob_no_risks),
                xytext=(deterministic_cp + 1, prob_no_risks),
                color='blue', fontsize=10)

    ax.annotate(f"{prob_with_risks:.1f}%",
                xy=(deterministic_cp, prob_with_risks),
                xytext=(deterministic_cp + 1, prob_with_risks),
                color='red', fontsize=10)

    # Add a legend, title and labels
    ax.legend(loc='lower right')
    ax.set_title('S-Curve: Confidence Level vs. Project Duration')
    ax.set_xlabel('Project Duration (Days)')
    ax.set_ylabel('Confidence Level (%)')
    ax.grid(True, alpha=0.3)

    # Limit y-axis to 0-100%
    ax.set_ylim(0, 100)

    # Add explanatory text
    ax.text(0.02, 0.98,
           "The S-curve shows the probability of completing the project\n"
           "within a given duration. The horizontal distance between\n"
           "the curves represents the additional buffer needed to\n"
           "account for risks at each confidence level.",
           transform=ax.transAxes, ha='left', va='top',
           bbox=dict(boxstyle='round', facecolor='white', alpha=0.9))

    plt.tight_layout()
    plt.show()

# Create the S-curve comparison
create_s_curve_comparison()
```



In [43]:

```python
# Create an S-Curve comparison with multiple confidence levels
def create_s_curve_comparison():
    """Create an S-curve visualization with multiple confidence levels"""
    # Sort the simulation results
    sorted_no_risks = np.sort(results_no_risks)
    sorted_with_risks = np.sort(results_with_risks)

    # Calculate the cumulative probability
    p = np.linspace(0, 100, len(sorted_no_risks))

    # Create the figure
    fig, ax = plt.subplots(figsize=(12, 8))

    # Plot the S-curves
    ax.plot(sorted_no_risks, p, label='Without Risks', color='blue', linewidth=2)
    ax.plot(sorted_with_risks, p, label='With Risks', color='red', linewidth=2)

    # Highlight key confidence levels
    confidence_levels = [10, 25, 50, 75, 80, 85, 90, 95]

    # Create a colormap for the confidence levels
    cm = plt.cm.get_cmap('viridis', len(confidence_levels))

    # Add reference lines for each confidence level
    for i, cl in enumerate(confidence_levels):
        # Calculate the durations at this confidence level
        duration_no_risks = np.percentile(sorted_no_risks, cl)
        duration_with_risks = np.percentile(sorted_with_risks, cl)

        # Add horizontal line
        ax.axhline(y=cl, color='gray', linestyle=':', alpha=0.5)

        # Add markers at the intersections
        ax.plot(duration_no_risks, cl, 'o', color=cm(i), markersize=8)
        ax.plot(duration_with_risks, cl, 'o', color=cm(i), markersize=8)

        # Add text labels for the confidence level
        ax.text(ax.get_xlim()[0] - 0.5, cl, f"{cl}%", va='center', ha='right', fontsi

        # Add text for durations
```

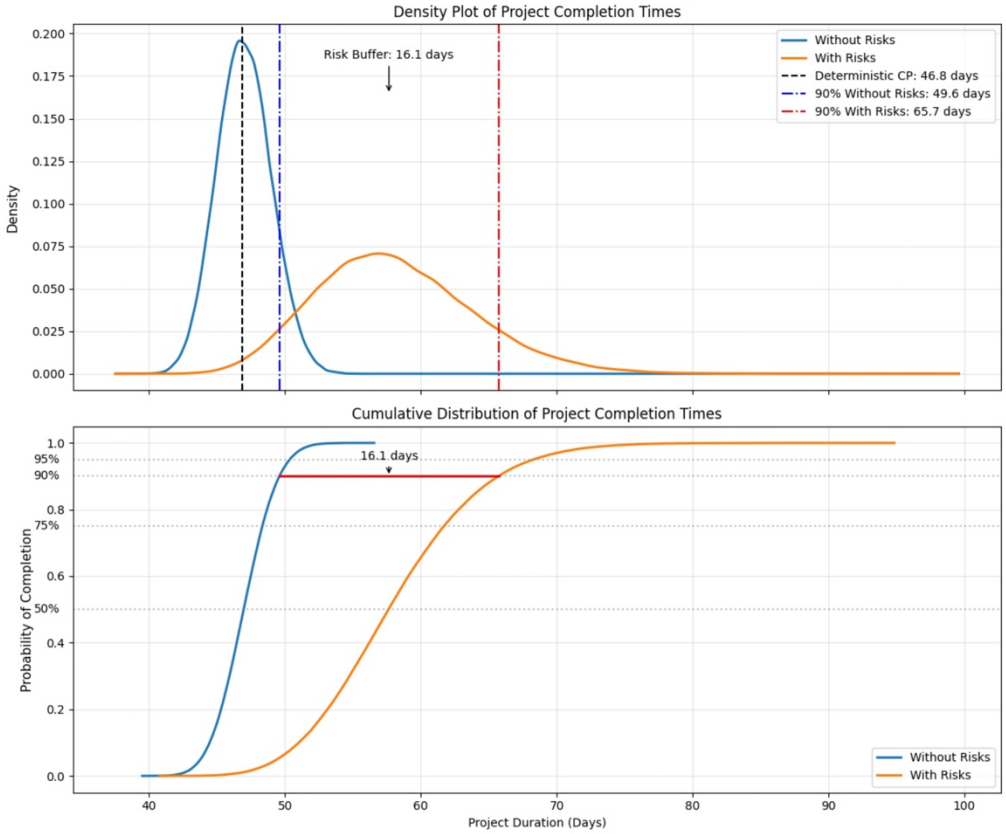
```
        if category_mapping:
            results = results_with_risks
            p90 = np.percentile(results, 90)
            category_results[category] = p90 - base_p90

    # Sort categories by impact
    sorted_categories = sorted(category_results.items(), key=lambda x: abs(x[1]), rev

    # Create the tornado diagram
    fig, ax = plt.subplots(figsize=(10, 6))

    # Plot the bars
    y_pos = np.arange(len(sorted_categories))
    category_names = [cat[0] for cat in sorted_categories]
    impacts = [cat[1] for cat in sorted_categories]

    # Use different colors based on positive/negative impact
    colors = ['#ff9999' if x > 0 else '#66b3ff' for x in impacts]

    bars = ax.barh(y_pos, impacts, color=colors)

    # Add data labels
    for i, bar in enumerate(bars):
        width = bar.get_width()
        label_x_pos = width + 0.1 if width > 0 else width - 0.1
        ax.text(label_x_pos, bar.get_y() + bar.get_height()/2, f"{width:.1f} days",
                va='center', ha='left' if width > 0 else 'right')

    # Add a vertical line at 0
    ax.axvline(x=0, color='black', linestyle='-', alpha=0.3)

    # Set the labels and title
    ax.set_yticks(y_pos)
    ax.set_yticklabels(category_names)
    ax.set_xlabel('Impact on P90 Project Duration (Days)')
    ax.set_title('Risk Category Sensitivity Analysis (Tornado Diagram)')

    # Add a text annotation explaining the chart
    ax.text(0.98, 0.02,
            "Shows how each risk category affects the P90 project duration\n"
            "when isolated from other risk categories.",
            transform=ax.transAxes, ha='right', va='bottom',
            bbox=dict(boxstyle='round', facecolor='white', alpha=0.8))

    plt.tight_layout()
    plt.show()

    return category_results

# Create the tornado diagram
tornado_results = create_tornado_diagram()
```



In [42]:

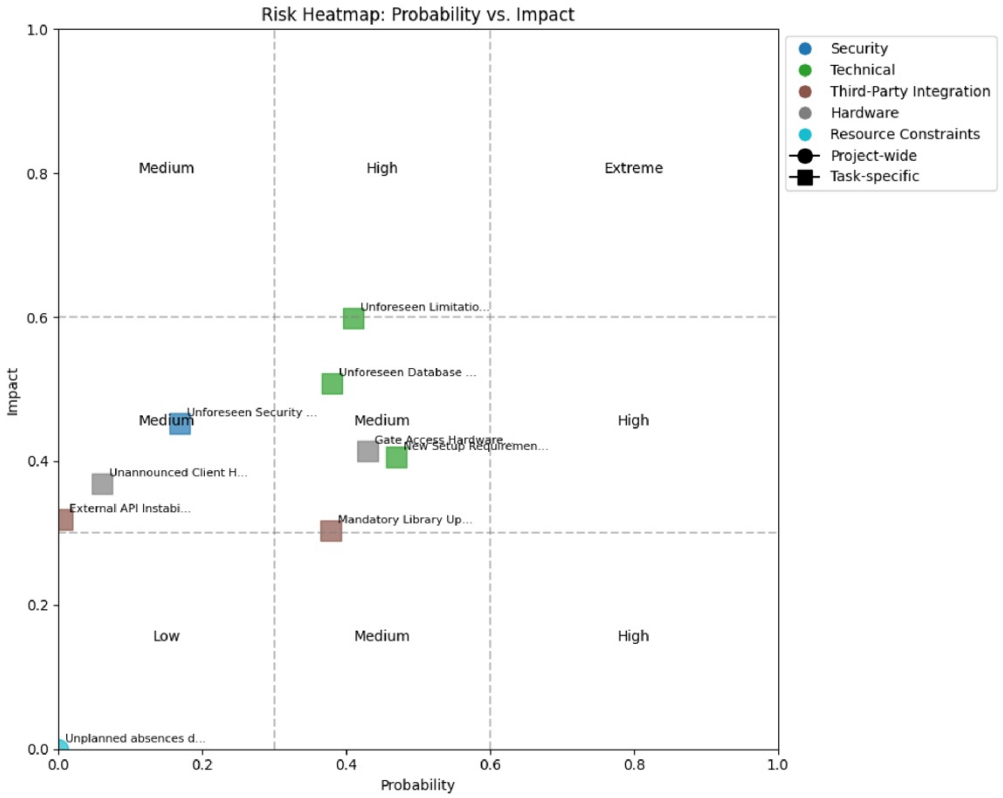
```python
# Create a Tornado Diagram to show sensitivity analysis of project risks
def create_tornado_diagram():
    """Create a tornado diagram showing which risk factors have the greatest impact o
    # Run base simulation for reference
    base_results = results_no_risks
    base_p90 = np.percentile(base_results, 90)

    # Define risk categories to analyze
    risk_categories = {}

    # First identify all risk categories in the model
    for task_id, risks in risk_mapping.items():
        if task_id == 'project_wide':
            for risk in risks:
                category = risk['Risk_Category']
                if category not in risk_categories:
                    risk_categories[category] = []
                risk_categories[category].append(risk)
        else:
            for risk in risks:
                category = risk['Risk_Category']
                if category not in risk_categories:
                    risk_categories[category] = []
                risk_categories[category].append(risk)

    # Create modified risk mappings for each category (only include that category's r
    category_results = {}
    for category in risk_categories.keys():
        # Create a new risk mapping with only risks from this category
        category_mapping = {}

        for task_id, risks in risk_mapping.items():
            if task_id == 'project_wide':
                category_mapping[task_id] = [risk for risk in risks if risk['Risk_Cat
                if not category_mapping[task_id]:
                    del category_mapping[task_id]
            else:
                category_risks = [risk for risk in risks if risk['Risk_Category'] ==
                if category_risks:
                    category_mapping[task_id] = category_risks

        # Run simulation with only this category's risks
```

```
    # Define y positions for each task
    y_positions = {task['Task_ID']: i for i, task in enumerate(task_data)}

    # Plot tasks without risks
    for task in task_data:
        y_pos = y_positions[task['Task_ID']]
        ax.barh(y_pos - 0.15, task['Duration'], left=task['Start'],
                height=0.3, color=color_no_risks, alpha=0.7, label='Without Risks' if

        # Add text for duration
        ax.text(task['Start'] + task['Duration']/2, y_pos - 0.15,
                f"Task {task['Task_ID']}: {task['Duration']:.1f} days",
                ha='center', va='center', fontsize=8, color='black')

    # Plot tasks with risks (using estimated delays)
    for task in task_data:
        y_pos = y_positions[task['Task_ID']]

        # Apply a delay based on position in the timeline
        # Later tasks get proportionally more delay
        position_ratio = task['Start'] / proj_finish
        task_delay_factor = delay_factor * position_ratio

        # Calculate new start, duration, and finish with risks
        risk_start = task['Start'] * (1 + task_delay_factor * 0.5)  # Less delay at s
        risk_duration = task['Duration'] * (1 + task_delay_factor)  # Full delay on d

        ax.barh(y_pos + 0.15, risk_duration, left=risk_start,
                height=0.3, color=color_with_risks, alpha=0.7, label='With Risks' if y

        # Add text for duration with risks
        ax.text(risk_start + risk_duration/2, y_pos + 0.15,
                f"{risk_duration:.1f} days",
                ha='center', va='center', fontsize=8, color='black')

    # Configure y-axis
    plt.yticks([y_positions[task['Task_ID']] for task in task_data],
               [f"Task {task['Task_ID']}" for task in task_data])

    # Add legend, title and labels
    plt.legend()
    plt.title('Critical Path Task Timeline Comparison: With vs. Without Risks')
    plt.xlabel('Time (Days)')
    plt.ylabel('Tasks')
    plt.grid(True, axis='x', alpha=0.3)

    # Add vertical line for deterministic critical path finish
    plt.axvline(x=deterministic_cp, color='black', linestyle='--',
                label=f'Deterministic CP: {deterministic_cp:.1f} days')

    # Add vertical line for 90th percentile with risks
    p90_with_risks = np.percentile(results_with_risks, 90)
    plt.axvline(x=p90_with_risks, color='red', linestyle=':',
                label=f'90% With Risks: {p90_with_risks:.1f} days')

    plt.legend(loc='upper right')
    plt.tight_layout()
    plt.show()

# Plot task timeline comparison
plot_task_timeline_comparison()
```



In [41]:
```python
# Create a visualization of task timelines with and without risks
def plot_task_timeline_comparison():
    """Create a visualization of critical path tasks with and without risks"""
    # Get the critical path tasks
    proj_finish, cp_tasks, all_tasks = compute_critical_path_details()

    # Use the deterministic schedule for reference
    task_data = []
    for task_id in cp_tasks:
        if task_id in all_tasks:
            task = all_tasks[task_id]
            task_data.append({
                'Task_ID': task_id,
                'Start': task['start'],
                'Finish': task['finish'],
                'Duration': task['duration']
            })

    # Sort tasks by start time
    task_data.sort(key=lambda x: x['Start'])

    # Create the Gantt chart visualization
    fig, ax = plt.subplots(figsize=(14, 8))

    # Calculate expected delays from risk simulations
    # Use 90th percentile as the reference point
    delay_factor = np.percentile(results_with_risks, 90) / proj_finish - 1

    # Define colors
    color_no_risks = 'lightblue'
    color_with_risks = 'lightcoral'
```

```
    target_with_risks = np.percentile(results_with_risks, target_percentile)

    ax1.axvline(x=target_no_risks, color='blue', linestyle='-.',
                label=f'{target_percentile}% Without Risks: {target_no_risks:.1f} days
    ax1.axvline(x=target_with_risks, color='red', linestyle='-.',
                label=f'{target_percentile}% With Risks: {target_with_risks:.1f} days'

    # Calculate the risk buffer
    risk_buffer = target_with_risks - target_no_risks
    ax1.annotate(f'Risk Buffer: {risk_buffer:.1f} days',
                 xy=((target_no_risks + target_with_risks)/2, ax1.get_ylim()[1]*0.8),
                 xytext=((target_no_risks + target_with_risks)/2, ax1.get_ylim()[1]*0.
                 arrowprops=dict(arrowstyle='->'),
                 ha='center')

    ax1.set_title('Density Plot of Project Completion Times')
    ax1.set_ylabel('Density', fontsize=11, labelpad=10)
    ax1.legend()
    ax1.grid(True, alpha=0.3)

    # Plot 2: Empirical CDFs with visual risk buffer
    sorted_no_risks = np.sort(results_no_risks)
    sorted_with_risks = np.sort(results_with_risks)
    y = np.arange(1, len(sorted_no_risks) + 1) / len(sorted_no_risks)

    ax2.plot(sorted_no_risks, y, label='Without Risks', linewidth=2)
    ax2.plot(sorted_with_risks, y, label='With Risks', linewidth=2)

    # Reference lines at important percentiles
    percentiles = [50, 75, 90, 95]
    for p in percentiles:
        p_y = p / 100
        p_no_risks = np.percentile(results_no_risks, p)
        p_with_risks = np.percentile(results_with_risks, p)

        # Horizontal line at percentile level
        ax2.axhline(y=p_y, color='gray', linestyle=':', alpha=0.5)

        # Show the difference (risk buffer) at this percentile
        if p == target_percentile:  # Highlight the target percentile
            ax2.plot([p_no_risks, p_with_risks], [p_y, p_y], 'r-', linewidth=2)
            ax2.annotate(f'{risk_buffer:.1f} days',
                         xy=((p_no_risks + p_with_risks)/2, p_y),
                         xytext=((p_no_risks + p_with_risks)/2, p_y+0.05),
                         arrowprops=dict(arrowstyle='->'),
                         ha='center')

        # Add percentile labels
        ax2.annotate(f'{p}%', xy=(ax2.get_xlim()[0], p_y), xytext=(ax2.get_xlim()[0]-
                     ha='right', va='center')

    ax2.set_title('Cumulative Distribution of Project Completion Times')
    ax2.set_xlabel('Project Duration (Days)')
    ax2.set_ylabel('Probability of Completion', fontsize=11, labelpad=10)
    ax2.grid(True, alpha=0.3)
    ax2.legend()

    # Apply tight layout after the adjust to make sure everything fits
    plt.tight_layout()
    plt.show()

# Plot detailed completion distribution comparison
plot_completion_distribution_comparison()
```



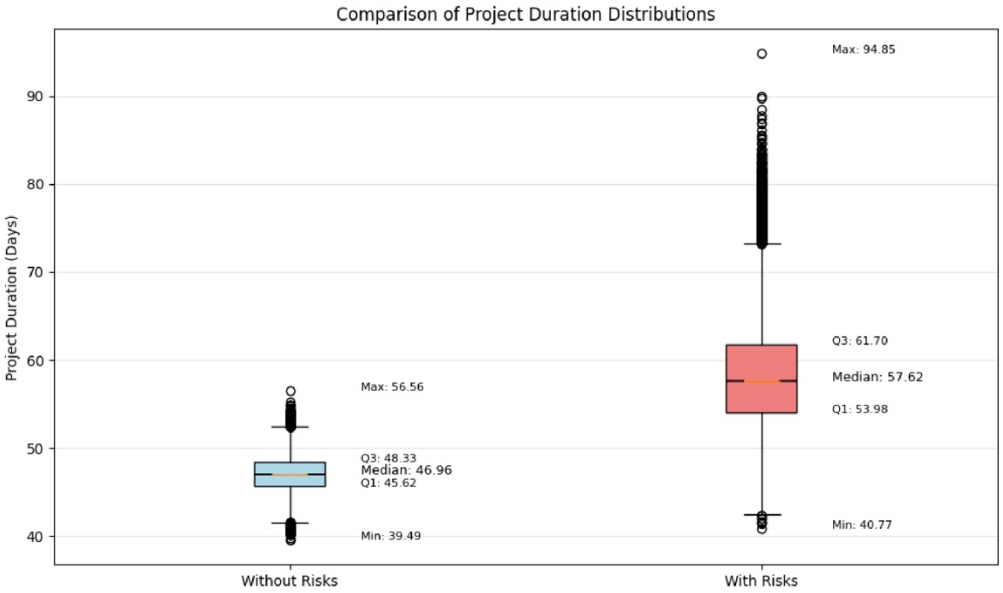
In [40]:

```python
# Visualize the impact of risks on project completion dates
def plot_completion_distribution_comparison():
    """Plot a more detailed comparison of completion date distributions"""
    # Create figure with two subplots with adjusted layout
    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 10), sharex=True)

    # Add more space on the left for y-axis labels
    plt.subplots_adjust(left=0.15)

    # Plot 1: Kernel density estimation to smoothly visualize the distribution
    from scipy.stats import gaussian_kde

    # Compute KDE for both distributions
    kde_no_risks = gaussian_kde(results_no_risks)
    kde_with_risks = gaussian_kde(results_with_risks)

    # Define the x range based on min and max values across both datasets
    x_min = min(np.min(results_no_risks), np.min(results_with_risks))
    x_max = max(np.max(results_no_risks), np.max(results_with_risks))
    x = np.linspace(x_min * 0.95, x_max * 1.05, 1000)

    # Plot the densities
    ax1.plot(x, kde_no_risks(x), label='Without Risks', linewidth=2)
    ax1.plot(x, kde_with_risks(x), label='With Risks', linewidth=2)

    # Mark deterministic critical path
    deterministic_cp = compute_critical_path()
    ax1.axvline(x=deterministic_cp, color='black', linestyle='--',
                label=f'Deterministic CP: {deterministic_cp:.1f} days')

    # Mark the target percentile for both distributions
    target_no_risks = np.percentile(results_no_risks, target_percentile)
```

```
    plt.axhline(y=0.6, color='gray', linestyle='--', alpha=0.5)
    plt.axvline(x=0.3, color='gray', linestyle='--', alpha=0.5)
    plt.axvline(x=0.6, color='gray', linestyle='--', alpha=0.5)

    # Label the risk zones
    plt.text(0.15, 0.15, "Low", fontsize=10, ha='center')
    plt.text(0.45, 0.15, "Medium", fontsize=10, ha='center')
    plt.text(0.8, 0.15, "High", fontsize=10, ha='center')
    plt.text(0.15, 0.45, "Medium", fontsize=10, ha='center')
    plt.text(0.45, 0.45, "Medium", fontsize=10, ha='center')
    plt.text(0.8, 0.45, "High", fontsize=10, ha='center')
    plt.text(0.15, 0.8, "Medium", fontsize=10, ha='center')
    plt.text(0.45, 0.8, "High", fontsize=10, ha='center')
    plt.text(0.8, 0.8, "Extreme", fontsize=10, ha='center')

    # Set the plot limits and labels
    plt.xlim(0, 1)
    plt.ylim(0, 1)
    plt.xlabel('Probability')
    plt.ylabel('Impact')
    plt.title('Risk Heatmap: Probability vs. Impact')

    # Create a legend for risk categories and types
    category_patches = [plt.Line2D([0], [0], marker='o', color='w', markerfacecolor=c
                                   markersize=10, label=cat) for cat in unique_categor
    type_patches = [plt.Line2D([0], [0], marker=marker, color='black', markersize=10,
                    for t, marker in markers.items()]
    plt.legend(handles=category_patches + type_patches, loc='upper left', bbox_to_anc

    plt.tight_layout()
    plt.show()

# Plot risk heatmap
plot_risk_heatmap()
```

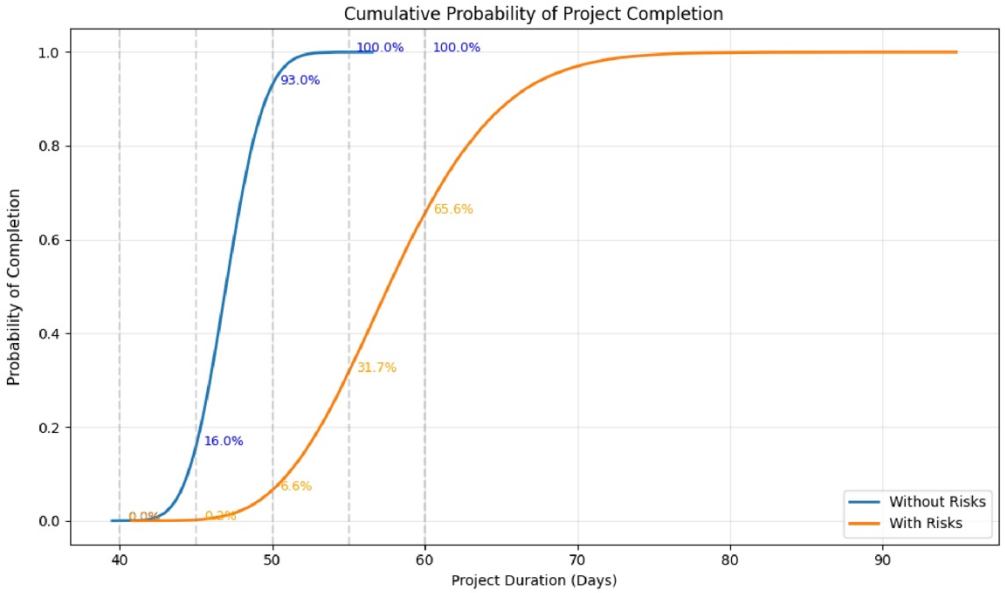
```
    # Process task-specific risks (get unique risks)
    unique_task_risks = {}
    for task_id, risks in risk_mapping.items():
        if task_id == 'project_wide':
            continue
        for risk in risks:
            risk_name = risk['Risk_Name']
            if risk_name not in unique_task_risks:
                # Calculate the center (most likely) value of fuzzy probability and i
                prob = risk['Fuzzy_Probability'][1]  # Most likely value
                impact = risk['Fuzzy_Impact'][1]  # Most likely value

                unique_task_risks[risk_name] = {
                    'Risk_Name': risk_name,
                    'Category': risk['Risk_Category'],
                    'Probability': prob,
                    'Impact': impact,
                    'Type': 'Task-specific'
                }

    # Add unique task risks to all_risks
    all_risks.extend(unique_task_risks.values())

    # If we have too many risks, just show the top ones by probability * impact
    for risk in all_risks:
        risk['Risk_Score'] = risk['Probability'] * risk['Impact']

    all_risks.sort(key=lambda x: x['Risk_Score'], reverse=True)
    risks_to_show = all_risks[:15]  # Show top 15 risks

    # Create the heatmap
    plt.figure(figsize=(10, 8))

    # Prepare data for the scatter plot
    probabilities = [r['Probability'] for r in risks_to_show]
    impacts = [r['Impact'] for r in risks_to_show]
    categories = [r['Category'] for r in risks_to_show]
    types = [r['Type'] for r in risks_to_show]
    names = [r['Risk_Name'] for r in risks_to_show]

    # Create a colormap based on categories
    unique_categories = list(set(categories))
    category_colors = plt.cm.tab10(np.linspace(0, 1, len(unique_categories)))
    category_to_color = {cat: category_colors[i] for i, cat in enumerate(unique_categ
    colors = [category_to_color[cat] for cat in categories]

    # Create markers based on risk type
    markers = {'Project-wide': 'o', 'Task-specific': 's'}
    marker_list = [markers[t] for t in types]

    # Create the scatter plot
    for i in range(len(risks_to_show)):
        plt.scatter(probabilities[i], impacts[i], s=200, c=[colors[i]],
                    marker=marker_list[i], alpha=0.7)

    # Add risk labels
    for i, name in enumerate(names):
        # Shorten long names
        short_name = name[:20] + "..." if len(name) > 20 else name
        plt.annotate(short_name,
                     xy=(probabilities[i], impacts[i]),
                     xytext=(probabilities[i]+0.01, impacts[i]+0.01),
                     fontsize=8)

    # Add grid lines for risk zones
    plt.axhline(y=0.3, color='gray', linestyle='--', alpha=0.5)
```

```
        plt.annotate(f"Q1: {q1:.2f}", xy=(x_pos, q1),
                     xytext=(x_pos + 0.15, q1), fontsize=8)
        plt.annotate(f"Q3: {q3:.2f}", xy=(x_pos, q3),
                     xytext=(x_pos + 0.15, q3), fontsize=8)

        # Annotations for min and max
        plt.annotate(f"Min: {min_val:.2f}", xy=(x_pos, min_val),
                     xytext=(x_pos + 0.15, min_val), fontsize=8)
        plt.annotate(f"Max: {max_val:.2f}", xy=(x_pos, max_val),
                     xytext=(x_pos + 0.15, max_val), fontsize=8)

    plt.title('Comparison of Project Duration Distributions')
    plt.ylabel('Project Duration (Days)')
    plt.grid(True, axis='y', alpha=0.3)
    plt.tight_layout()
    plt.show()

# Plot box plot comparison
plot_boxplot_comparison()
```



In [39]:
```
# Create a risk heatmap to visualize probability vs. impact
def plot_risk_heatmap():
    """Create a heatmap visualization of risks based on probability and impact"""
    # Collect all risks (both project-wide and task-specific)
    all_risks = []

    # Process project-wide risks
    if 'project_wide' in risk_mapping:
        for risk in risk_mapping['project_wide']:
            # Calculate the center (most likely) value of fuzzy probability and impac
            prob = risk['Fuzzy_Probability'][1]  # Most likely value
            impact = risk['Fuzzy_Impact'][1]  # Most likely value

            all_risks.append({
                'Risk_Name': risk['Risk_Name'],
                'Category': risk['Risk_Category'],
                'Probability': prob,
                'Impact': impact,
                'Type': 'Project-wide'
            })
```

```
    plt.ylabel('Probability of Completion', fontsize=11, labelpad=10)
    plt.grid(True, alpha=0.3)
    plt.legend()
    plt.tight_layout()
    plt.show()

# Plot CDF comparison
plot_cdf_comparison()
```



In [38]:
```
# Create box plot comparison for project duration distributions
def plot_boxplot_comparison():
    """Create box plot to compare project duration distributions with and without ris
    plt.figure(figsize=(10, 6))

    # Prepare data for boxplot
    data = [results_no_risks, results_with_risks]
    labels = ['Without Risks', 'With Risks']

    # Create box plot with customizations
    box = plt.boxplot(data, labels=labels, patch_artist=True, notch=True, showfliers=

    # Customize colors
    colors = ['lightblue', 'lightcoral']
    for patch, color in zip(box['boxes'], colors):
        patch.set_facecolor(color)

    # Add actual values for key statistics
    for i, dataset in enumerate([results_no_risks, results_with_risks]):
        min_val = np.min(dataset)
        q1 = np.percentile(dataset, 25)
        median = np.median(dataset)
        q3 = np.percentile(dataset, 75)
        max_val = np.max(dataset)

        x_pos = i + 1  # Box plot positions start at 1

        # Annotations for median
        plt.annotate(f"Median: {median:.2f}", xy=(x_pos, median),
                     xytext=(x_pos + 0.15, median), fontsize=9)

        # Annotations for quartiles
```

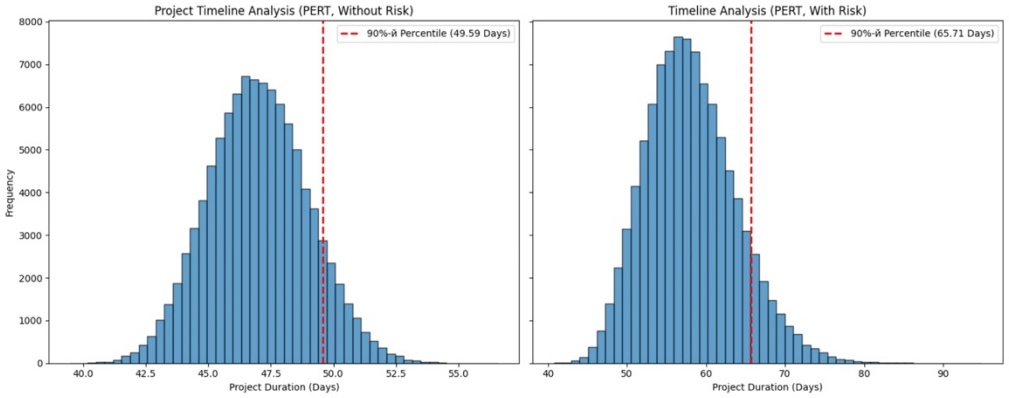
```
df_cp = pd.DataFrame(table_data)

# Set display options for wider columns and round values to 2 decimals
pd.set_option('display.width', 200)
df_cp['Start'] = df_cp['Start'].round(2)
df_cp['Finish'] = df_cp['Finish'].round(2)
df_cp['Duration'] = df_cp['Duration'].round(2)

print("\nCritical Path Tasks:")
print(df_cp.to_string(index=False))
```

```
Детермінований Critical Path займає: 46.83 днів

Critical Path Tasks:
 Task ID  Start  Finish  Duration
       1   0.00    7.33      7.33
       2   7.33   15.67      8.33
       6  15.67   18.83      3.17
       7  18.83   21.00      2.17
      11  21.00   22.17      1.17
      26  22.17   28.50      6.33
      27  28.50   34.33      5.83
      28  34.33   37.17      2.83
      29  37.17   39.17      2.00
      34  39.17   40.33      1.17
      35  40.33   43.33      3.00
      81  43.33   46.83      3.50
```

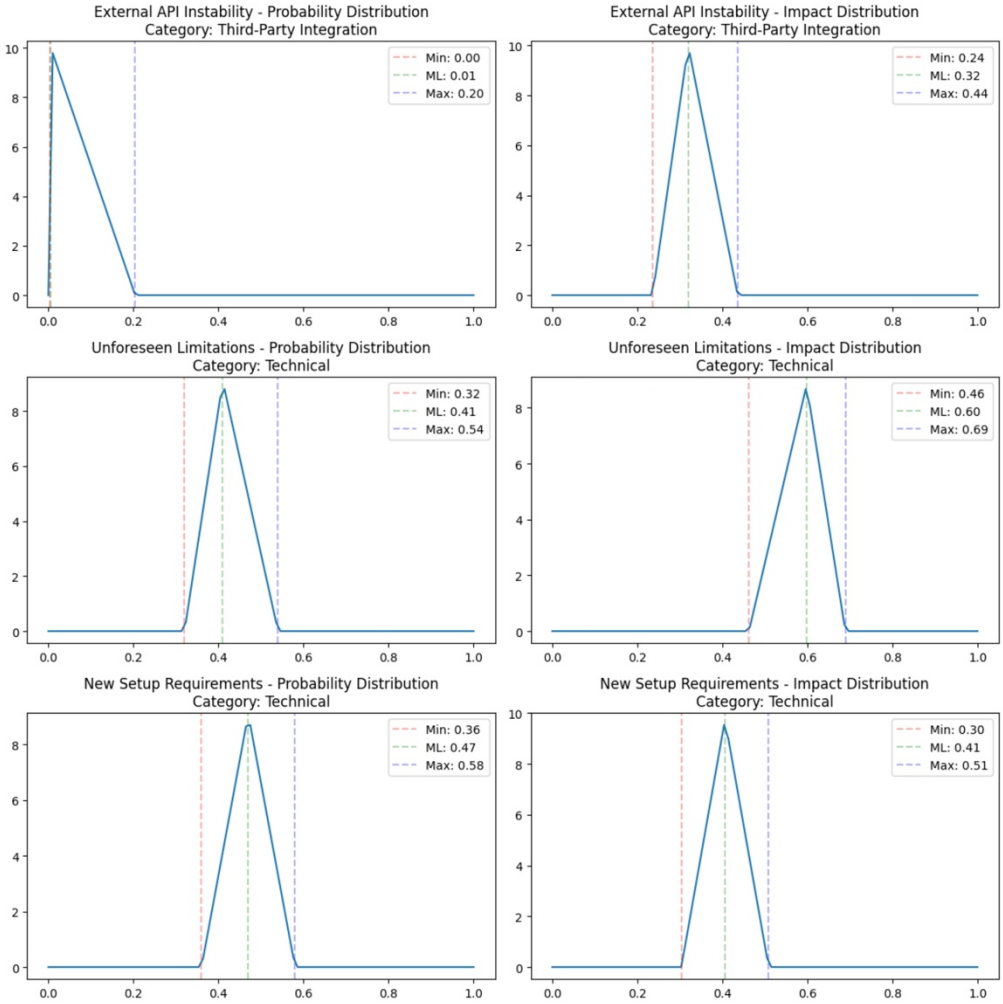
In [37]:
```
# Create CDF plot to compare project completion probabilities
def plot_cdf_comparison():
    """Plot cumulative distribution function for project completion with and without
    # Sort data for CDF calculation
    sorted_no_risks = np.sort(results_no_risks)
    sorted_with_risks = np.sort(results_with_risks)

    # Calculate the CDF values (y-axis)
    y = np.arange(1, len(sorted_no_risks) + 1) / len(sorted_no_risks)

    # Create the plot with more space for y-axis label
    plt.figure(figsize=(10, 6))

    # Add padding to the left side to make space for y-axis label
    plt.subplots_adjust(left=0.15)

    plt.plot(sorted_no_risks, y, label='Without Risks', linewidth=2)
    plt.plot(sorted_with_risks, y, label='With Risks', linewidth=2)

    # Add reference lines for target dates
    target_dates = [40, 45, 50, 55, 60]
    for date in target_dates:
        plt.axvline(date, color='gray', linestyle='--', alpha=0.3)

    # Add annotations for completion probabilities at selected dates
    for date in target_dates:
        prob_no_risks = np.mean(results_no_risks <= date) * 100
        prob_with_risks = np.mean(results_with_risks <= date) * 100
        plt.annotate(f"{prob_no_risks:.1f}%", xy=(date, np.mean(results_no_risks <= d
                    xytext=(date+0.5, np.mean(results_no_risks <= date)),
                    color='blue', fontsize=9)
        plt.annotate(f"{prob_with_risks:.1f}%", xy=(date, np.mean(results_with_risks
                    xytext=(date+0.5, np.mean(results_with_risks <= date)),
                    color='orange', fontsize=9)

    plt.title('Cumulative Probability of Project Completion')
    plt.xlabel('Project Duration (Days)')
```

```
axs[0].axvline(np.percentile(results_no_risks, target_percentile), color='red', lines
               label=f"{target_percentile}%-й Percentile ({np.percentile(results_no_r
axs[0].set_title('Project Timeline Analysis (PERT, Without Risk)')
axs[0].set_xlabel('Project Duration (Days)')
axs[0].set_ylabel('Frequency')
axs[0].legend()

# Гістограма для симуляцій з ризиками
axs[1].hist(results_with_risks, bins=50, edgecolor='black', alpha=0.7)
axs[1].axvline(np.percentile(results_with_risks, target_percentile), color='red', lin
               label=f"{target_percentile}%-й Percentile ({np.percentile(results_with
axs[1].set_title('Timeline Analysis (PERT, With Risk)')
axs[1].set_xlabel('Project Duration (Days)')
axs[1].legend()

plt.tight_layout()
plt.show()
```

```
--- Без ризиків ---
Середня тривалість проекту: 47.00 днів
Медіана тривалості проекту: 46.96 днів
Для успішного завершення з ймовірністю 90% потрібно 49.59 днів
Ймовірність завершення за 50 днів: 93.0%

--- З ризиками (з вагами експертів) ---
Середня тривалість проекту: 58.08 днів
Медіана тривалості проекту: 57.62 днів
Для успішного завершення з ймовірністю 90% потрібно 65.71 днів
Ймовірність завершення за 50 днів: 6.6%
```



In [36]:
```
# Fix the typo in the function name (critical vs ritical)
deterministic_cp = compute_critical_path()
print(f"Детермінований Critical Path займає: {deterministic_cp:.2f} днів")

# Compute detailed critical path information
proj_finish, cp_tasks, all_tasks = compute_critical_path_details()

import pandas as pd

# Build table data with task id, start, finish and duration for tasks in CP
table_data = []
for tid in cp_tasks:
    task = all_tasks[tid]
    table_data.append({
        'Task ID': tid,
        'Start': task['start'],
        'Finish': task['finish'],
        'Duration': task['duration']
    })
```



In [35]:
```python
target_percentile = 90  # Відсоток для цільової тривалості проекту

# Функція для аналізу результатів симуляцій
def analyze_results(results, label):
    duration_target = np.percentile(results, target_percentile)
    mean_duration = np.mean(results)
    median_duration = np.median(results)
    # Приклад: розрахунок ймовірності завершення проекту за 50 днів
    prob_within_50 = np.mean(results <= 50)
    print(f"--- {label} ---")
    print(f"Середня тривалість проекту: {mean_duration:.2f} днів")
    print(f"Медіана тривалості проекту: {median_duration:.2f} днів")
    print(f"Для успішного завершення з ймовірністю {target_percentile}% потрібно {dur
    print(f"Ймовірність завершення за 50 днів: {prob_within_50*100:.1f}%\n")

# Аналіз результатів симуляцій
analyze_results(results_no_risks, "Без ризиків")
analyze_results(results_with_risks, "З ризиками (з вагами експертів)")

# Створення підграфіків для порівняння результатів
fig, axs = plt.subplots(1, 2, figsize=(15, 6), sharey=True)

# Гістограма для симуляцій без ризиків
axs[0].hist(results_no_risks, bins=50, edgecolor='black', alpha=0.7)
```

In [34]:
```python
# Visualize the fuzzy risk values with expert weights
def plot_fuzzy_risk_distributions():
    """Plot distributions of risk probability and impact for selected risks"""
    # Get unique risks from the mapping
    unique_risks = {}
    for task_risks in risk_mapping.values():
        for risk in task_risks:
            risk_name = risk['Risk_Name']
            if risk_name not in unique_risks:
                unique_risks[risk_name] = risk

    if not unique_risks:
        print("No risks found in mapping")
        return

    # Plot the first 3 risks (or fewer if there are less)
    risks_to_plot = list(unique_risks.values())[:min(3, len(unique_risks))]

    fig, axes = plt.subplots(len(risks_to_plot), 2, figsize=(12, 4*len(risks_to_plot)

    if len(risks_to_plot) == 1:
        axes = [axes]

    for i, risk in enumerate(risks_to_plot):
        risk_name = risk['Risk_Name']
        category = risk['Risk_Category']
        prob = risk['Fuzzy_Probability']
        impact = risk['Fuzzy_Impact']

        # Plot probability distribution
        x = np.linspace(0, 1, 100)
        y_prob = triang.pdf(x, c=(prob[1]-prob[0])/(prob[2]-prob[0]) if prob[2]>prob[
                            loc=prob[0], scale=prob[2]-prob[0])
        axes[i][0].plot(x, y_prob)
        axes[i][0].set_title(f"{risk_name} - Probability Distribution\nCategory: {cat
        axes[i][0].axvline(prob[0], color='r', linestyle='--', alpha=0.3, label=f'Min
        axes[i][0].axvline(prob[1], color='g', linestyle='--', alpha=0.3, label=f'ML:
        axes[i][0].axvline(prob[2], color='b', linestyle='--', alpha=0.3, label=f'Max
        axes[i][0].legend()

        # Plot impact distribution
        y_impact = triang.pdf(x, c=(impact[1]-impact[0])/(impact[2]-impact[0]) if imp
                              loc=impact[0], scale=impact[2]-impact[0])
        axes[i][1].plot(x, y_impact)
        axes[i][1].set_title(f"{risk_name} - Impact Distribution\nCategory: {category
        axes[i][1].axvline(impact[0], color='r', linestyle='--', alpha=0.3, label=f'M
        axes[i][1].axvline(impact[1], color='g', linestyle='--', alpha=0.3, label=f'M
        axes[i][1].axvline(impact[2], color='b', linestyle='--', alpha=0.3, label=f'M
        axes[i][1].legend()

    plt.tight_layout()
    plt.show()

# Plot fuzzy risk distributions to understand expert opinions
plot_fuzzy_risk_distributions()
```

```
        print("Risk data is not a DataFrame with a 'Category' column")

# Analyze the effect of expert weights
analyze_expert_weights()
```

```
Expert Weights Analysis by Risk Category:

Risk Category  Hardware  Resource Constraints  Security  Technical  Third-Party Integr
ation
Expert
Dev                 1.0                   0.0       1.0        1.0
1.0
PM                  0.0                   1.0       0.3        0.0
0.1
QA                  0.4                   0.0       1.0        0.3
0.5

Risk Categories in Model:
Resource Constraints: 1 risks
Third-Party Integration: 5 risks
Technical: 10 risks
Security: 6 risks
Hardware: 12 risks

Project-Wide Risks (applied once to final project duration):
1. Unplanned absences due to illness, resignation, or burnout may reduce available res
ources and delay progress. (Category: Resource Constraints)

Task-Specific Risks:
1. External API Instability (Category: Third-Party Integration)
2. Unforeseen Limitations (Category: Technical)
3. New Setup Requirements (Category: Technical)
4. External API Instability (Category: Third-Party Integration)
5. Unforeseen Limitations (Category: Technical)
6. New Setup Requirements (Category: Technical)
7. External API Instability (Category: Third-Party Integration)
8. Unforeseen Limitations (Category: Technical)
9. External API Instability (Category: Third-Party Integration)
10. Unforeseen Limitations (Category: Technical)
11. Mandatory Library Update Due to Vulnerabilities (Category: Third-Party Integratio
n)
12. Unforeseen Limitations (Category: Technical)
13. New Setup Requirements (Category: Technical)
14. Unforeseen Security Requirements (Category: Security)
15. New Setup Requirements (Category: Technical)
16. Unforeseen Security Requirements (Category: Security)
17. Unforeseen Security Requirements (Category: Security)
18. Unforeseen Security Requirements (Category: Security)
19. Unforeseen Security Requirements (Category: Security)
20. Gate Access Hardware Failures (Category: Hardware)
21. Unannounced Client Hardware Updates (Category: Hardware)
22. Unforeseen Security Requirements (Category: Security)
23. Gate Access Hardware Failures (Category: Hardware)
24. Unannounced Client Hardware Updates (Category: Hardware)
25. Unforeseen Database Changes (Category: Technical)
26. Gate Access Hardware Failures (Category: Hardware)
27. Unannounced Client Hardware Updates (Category: Hardware)
28. Gate Access Hardware Failures (Category: Hardware)
29. Unannounced Client Hardware Updates (Category: Hardware)
30. Gate Access Hardware Failures (Category: Hardware)
31. Unannounced Client Hardware Updates (Category: Hardware)
32. Unannounced Client Hardware Updates (Category: Hardware)
33. Unannounced Client Hardware Updates (Category: Hardware)

Actual Categories from Risks DataFrame:
Risk data is not a DataFrame with a 'Category' column
```

```
print("Expert Weights Analysis by Risk Category:\n")

# Create a DataFrame to visualize the weights
weight_data = []
for expert_type, categories in EXPERT_WEIGHTS.items():
    for category, weight in categories.items():
        weight_data.append({
            'Expert': expert_type,
            'Risk Category': category,
            'Weight': weight
        })

weight_df = pd.DataFrame(weight_data)
print(weight_df.pivot(index='Expert', columns='Risk Category', values='Weight'))

# Analyze the actual risks in the model
unique_categories = set()
category_counts = {}
project_wide_risks = []
task_specific_risks = []

# Process project-wide risks separately
if 'project_wide' in risk_mapping:
    for risk in risk_mapping['project_wide']:
        category = risk['Risk_Category']
        unique_categories.add(category)
        category_counts[category] = category_counts.get(category, 0) + 1
        project_wide_risks.append(risk)

# Process task-specific risks
for task_id, risks in risk_mapping.items():
    if task_id == 'project_wide':
        continue
    for risk in risks:
        category = risk['Risk_Category']
        unique_categories.add(category)
        category_counts[category] = category_counts.get(category, 0) + 1
        risk_name = risk['Risk_Name']
        if {'Risk_Name': risk_name} not in task_specific_risks:
            task_specific_risks.append({'Risk_Name': risk_name, 'Risk_Category':

print("\nRisk Categories in Model:")
for category, count in category_counts.items():
    print(f"{category}: {count} risks")

print("\nProject-Wide Risks (applied once to final project duration):")
if project_wide_risks:
    for i, risk in enumerate(project_wide_risks, 1):
        print(f"{i}. {risk['Risk_Name']} (Category: {risk['Risk_Category']})")
else:
    print("No project-wide risks found")

print("\nTask-Specific Risks:")
if task_specific_risks:
    for i, risk in enumerate(task_specific_risks, 1):
        print(f"{i}. {risk['Risk_Name']} (Category: {risk['Risk_Category']})")
else:
    print("No task-specific risks found")

print("\nActual Categories from Risks DataFrame:")
# Check if risks is a DataFrame and has a 'Category' column
if isinstance(risks, pd.DataFrame) and 'Category' in risks.columns:
    unique_csv_categories = risks['Category'].unique()
    for cat in unique_csv_categories:
        print(f"- {cat}")
else:
```

```
            duration = (4 * duration_most_likely + duration_optimistic + duration_pes
        except Exception as e:
            raise ValueError(f"Error processing task {task_id}") from e
        dep_str = str(row.get('Previous_Activity', '')).strip()
        if dep_str == "" or dep_str.lower() == "nan":
            dependencies = []
        else:
            try:
                dependencies = [int(x.strip()) for x in dep_str.split(';') if x.strip
            except Exception as e:
                raise ValueError(f"Error processing Previous_Activity for task {task_
        tasks[task_id] = {
            "duration": duration,
            "dependencies": dependencies,
            "start": None,
            "finish": None
        }

    remaining = set(tasks.keys())
    while remaining:
        progress = False
        for t in list(remaining):
            deps = tasks[t]["dependencies"]
            if all(dep in tasks and tasks[dep]["finish"] is not None for dep in deps)
                start_time = max((tasks[dep]["finish"] for dep in deps), default=0)
                tasks[t]["start"] = start_time
                tasks[t]["finish"] = start_time + tasks[t]["duration"]
                remaining.remove(t)
                progress = True
        if not progress:
            raise ValueError("Cyclic dependencies or invalid Previous_Activity detect

    project_finish = max(task["finish"] for task in tasks.values())

    # Backtracking to get a critical path: choose one candidate with finish == projec
    cp = []
    candidate = None
    for tid, t in tasks.items():
         if abs(t["finish"] - project_finish) < 1e-6:
            candidate = tid
            break
    if candidate is None:
        raise ValueError("No task found with finish equal to project finish.")

    current = candidate
    while True:
         cp.insert(0, current)
         deps = tasks[current]["dependencies"]
         if not deps:
             break
         current = max(deps, key=lambda d: tasks[d]["finish"] if tasks[d]["finish"] i

    return project_finish, cp, tasks
```

In [16]:
```
# Налаштування кількості симуляцій
iterations = 100000

# Симуляції без ризиків
results_no_risks = simulate_project(with_risks=False, iterations=iterations)
# Симуляції з ризиками з використанням нового підходу до розрахунку ризиків
results_with_risks = simulate_project(with_risks=True, iterations=iterations, risk_ma
```

In [33]:
```
# Updated analysis to show project-wide risks separately
def analyze_expert_weights():
    """Display how expert weights affect risk assessments for different categories"""
```

```
                    # Apply the impact to the total project duration
                    project_finish += project_finish * risk_impact

        results.append(project_finish)

    return np.array(results)
```

In [13]:
```
def compute_critical_path():
    """
    Обчислює детермінований час виконання проекту за критичним шляхом
    (без варіацій і ризиків) використовуючи колонку PERT.
    """
    tasks = {}
    for idx, row in backlog.iterrows():
        task_id = int(row['ID'])
        # Калькуляція тривалості задачі по PERT
        duration_optimistic = str(row['Optimistic']).replace(',', '.')
        duration_most_likely = str(row['Most_Likely']).replace(',', '.')
        duration_pessimistic = str(row['Pessimistic']).replace(',', '.')
        duration = (4 * float(duration_most_likely) + float(duration_optimistic) + fl

        dep_str = str(row.get('Previous_Activity', '')).strip()
        if dep_str == "" or dep_str.lower() == "nan":
            dependencies = []
        else:
            try:
                dependencies = [int(x.strip()) for x in dep_str.split(';') if x.strip
            except Exception as e:
                raise ValueError(f"Помилка при обробці Previous_Activity для задачі {

        tasks[task_id] = {
            "duration": duration,
            "dependencies": dependencies,
            "start": None,
            "finish": None
        }

    # Розрахунок розкладу без варіацій
    remaining = set(tasks.keys())
    while remaining:
        progress = False
        for task_id in list(remaining):
            deps = tasks[task_id]["dependencies"]
            if all(dep in tasks and tasks[dep]["finish"] is not None for dep in deps)
                start_time = max((tasks[dep]["finish"] for dep in deps), default=0)
                tasks[task_id]["start"] = start_time
                tasks[task_id]["finish"] = start_time + tasks[task_id]["duration"]
                remaining.remove(task_id)
                progress = True
        if not progress:
            raise ValueError("Виявлено циклічні залежності або невірно задані Previou

    project_duration = max(task["finish"] for task in tasks.values())
    return project_duration
```

In [14]:
```
# New function to compute critical path details
def compute_critical_path_details():
    # ...existing code for scheduling using backlog similar to compute_critical_path.
    tasks = {}
    for idx, row in backlog.iterrows():
        task_id = int(row['ID'])
        try:
            duration_optimistic = float(str(row['Optimistic']).replace(',', '.'))
            duration_most_likely = float(str(row['Most_Likely']).replace(',', '.'))
            duration_pessimistic = float(str(row['Pessimistic']).replace(',', '.'))
```

```
                # 1. Отримуємо fuzzy значення ймовірності і впливу
                fuzzy_prob = risk['Fuzzy_Probability']
                fuzzy_impact = risk['Fuzzy_Impact']

                # 2. Обчислюємо конкретне значення ймовірності для цієї симуляції
                risk_prob = sample_triangular(fuzzy_prob)

                # 3. Перевіряємо, чи ризик спрацьовує в цій симуляції
                if np.random.rand() < risk_prob:
                    # 4. Обчислюємо конкретне значення впливу для цієї симуляції
                    risk_impact = sample_triangular(fuzzy_impact)

                    # 5. Застосовуємо вплив як відсоток від тривалості задачі
                    duration += duration * risk_impact

        # Обробка залежностей із колонки Previous_Activity
        dep_str = str(row.get('Previous_Activity', '')).strip()
        if dep_str == "" or dep_str.lower() == "nan":
            dependencies = []
        else:
            try:
                dependencies = [int(x.strip()) for x in dep_str.split(';') if x.s
            except Exception as e:
                raise ValueError(f"Помилка при обробці Previous_Activity для зада

        tasks[task_id] = {
            "duration": duration,
            "dependencies": dependencies,
            "start": None,
            "finish": None
        }

    # Розрахунок розкладу із врахуванням залежностей
    remaining = set(tasks.keys())
    while remaining:
        progress = False
        for task_id in list(remaining):
            deps = tasks[task_id]["dependencies"]
            # Якщо всі залежності завершені, розраховуємо час старту і finish
            if all(dep in tasks and tasks[dep]["finish"] is not None for dep in d
                start_time = max((tasks[dep]["finish"] for dep in deps), default=
                tasks[task_id]["start"] = start_time
                tasks[task_id]["finish"] = start_time + tasks[task_id]["duration"
                remaining.remove(task_id)
                progress = True
        if not progress:
            raise ValueError("Виявлено циклічні залежності або невірно задані Pre

    # Час завершення проекту — максимум finish серед усіх задач
    project_finish = max(task["finish"] for task in tasks.values())

    # Apply project-wide risks (those marked as 'Multiple tasks') to final projec
    if with_risks and risk_mapping and 'project_wide' in risk_mapping:
        for risk in risk_mapping['project_wide']:
            # Get fuzzy values for probability and impact
            fuzzy_prob = risk['Fuzzy_Probability']
            fuzzy_impact = risk['Fuzzy_Impact']

            # Calculate concrete probability for this simulation
            risk_prob = sample_triangular(fuzzy_prob)

            # Check if the risk materializes in this simulation
            if np.random.rand() < risk_prob:
                # Calculate concrete impact value for this simulation
                risk_impact = sample_triangular(fuzzy_impact)
```

```
    pessimistic: песимістична оцінка тривалості (найдовша)
    """
    # Параметри форми для бета-розподілу, припускаючи що мода розподілу = most_likely
    # Формула для моди бета-розподілу: mode = (alpha - 1) / (alpha + beta_param - 2)
    # В стандартному PERT: Expected = (O + 4M + P) / 6
    # Тут ми використовуємо безпосередньо бета-розподіл для отримання випадкового зна

    # Нормалізуємо часовий інтервал до [0, 1] для бета-розподілу
    range_duration = pessimistic - optimistic
    if range_duration <= 0:
        return most_likely  # Якщо інтервал нульовий або від'ємний, повертаємо most_l

    # Нормалізуємо most_likely до [0, 1]
    normalized_m = (most_likely - optimistic) / range_duration

    # Встановлюємо параметри alpha і beta для бета-розподілу
    # Використовуємо стандартні параметри PERT: alpha=4, beta=2 при normalized_m=0.5
    # Але адаптуємо їх до конкретного значення most_likely
    if 0 < normalized_m < 1:
        # Формула для підбору alpha і beta_param коли відома мода розподілу
        alpha = 1 + 4 * normalized_m
        beta_param = 1 + 4 * (1 - normalized_m)
    else:
        # Якщо most_likely збігається з одним із крайніх значень
        if normalized_m <= 0:
            alpha, beta_param = 1, 5  # Розподіл у бік оптимістичної оцінки
        else:  # normalized_m >= 1
            alpha, beta_param = 5, 1  # Розподіл у бік песимістичної оцінки

    # Генеруємо випадкове число з бета-розподілу і денормалізуємо його
    # Використовуємо beta з scipy.stats, а не локальну змінну beta_param
    beta_random = beta.rvs(alpha, beta_param)
    return optimistic + beta_random * range_duration
```

In [12]:

```
def simulate_project(with_risks: bool, iterations: int = 10000, risk_mapping=None):
    """
    Симулює виконання проекту з урахуванням залежностей та використанням бета-розподі

    with_risks: True — враховувати ризикові події, False — без ризиків
    iterations: кількість симуляцій
    risk_mapping: словник із ризиками для задач
    """
    results = []  # Список тривалостей проекту для кожної симуляції

    for _ in range(iterations):
        tasks = {}
        # Формуємо дані для кожної задачі із backlog
        for idx, row in backlog.iterrows():
            task_id = int(row['ID'])

            # Отримуємо оцінки для PERT розрахунку
            try:
                optimistic = float(str(row['Optimistic']).replace(',', '.'))
                most_likely = float(str(row['Most_Likely']).replace(',', '.'))
                pessimistic = float(str(row['Pessimistic']).replace(',', '.'))
            except Exception as e:
                raise ValueError(f"Помилка при обробці оцінок для задачі {task_id}")

            # Обчислюємо тривалість задачі за допомогою бета-розподілу
            duration = calculate_pert_duration(optimistic, most_likely, pessimistic)

            # Застосування ризиків для конкретних задач
            if with_risks and risk_mapping and task_id in risk_mapping:
                task_risks = risk_mapping[task_id]
                for risk in task_risks:
```

```
backlog = pd.read_csv('backlog.csv')
risks = pd.read_csv('risks.csv')
```

In [10]:
```
# Updated function to handle 'Multiple tasks' designation differently
def create_risk_mapping(risks_df, backlog_df):
    """Create a mapping of task IDs to their associated risks with fuzzy probability
    risk_mapping = {}
    project_wide_risks = []  # New list to store project-wide risks

    for _, row in risks_df.iterrows():
        # Extract task IDs affected by this risk
        tasks_impacted = str(row['Tasks_Impacted']).strip()

        # Get the risk category directly from the CSV if available, otherwise use a d
        risk_name = row['Risk_Name']
        risk_category = row.get('Category', 'Technical')  # Default to Technical if C

        # Get probability columns and values
        probability_columns = [col for col in row.index if col.endswith('_Probability
        probability_values = [row[col] for col in probability_columns]

        # Get impact columns and values
        impact_columns = [col for col in row.index if col.endswith('_Impact')]
        impact_values = [row[col] for col in impact_columns]

        # Aggregate expert opinions with weights based on risk category
        fuzzy_probability = aggregate_expert_opinions(probability_values, probability
        fuzzy_impact = aggregate_expert_opinions(impact_values, impact_columns, risk_

        risk_data = {
            'Risk_Name': risk_name,
            'Risk_Category': risk_category,
            'Fuzzy_Probability': fuzzy_probability,
            'Fuzzy_Impact': fuzzy_impact
        }

        # Check if risk affects multiple tasks
        if tasks_impacted.lower() == "multiple tasks":
            # Add to project-wide risks instead of individual tasks
            project_wide_risks.append(risk_data)
        else:
            # Parse tasks impacted - now using semicolons as separators
            try:
                task_ids = [int(x.strip()) for x in tasks_impacted.split(';') if x.st
                for task_id in task_ids:
                    risk_mapping.setdefault(task_id, []).append(risk_data)
            except Exception as e:
                print(f"Error processing Tasks_Impacted: {tasks_impacted} - {str(e)}"

    # Store project-wide risks in a special key
    if project_wide_risks:
        risk_mapping['project_wide'] = project_wide_risks

    return risk_mapping

# Create the risk mapping using the updated function
risk_mapping = create_risk_mapping(risks, backlog)
```

In [11]:
```
def calculate_pert_duration(optimistic, most_likely, pessimistic):
    """
    Розраховує тривалість задачі за методом PERT використовуючи бета-розподіл.

    optimistic: оптимістична оцінка тривалості (найкоротша)
    most_likely: найбільш ймовірна оцінка тривалості
```

```
    if not expert_values or not expert_columns:
        return [0.0, 0.0, 0.0]

    # Pair each value with its expert type and weight
    weighted_values = []
    total_weight = 0.0

    for value, column in zip(expert_values, expert_columns):
        if pd.isna(value):
            continue

        expert_type = get_expert_type(column)
        if not expert_type:
            continue

        # Get the weight for this expert type and risk category
        weight = EXPERT_WEIGHTS.get(expert_type, {}).get(risk_category, 0.0)

        # Skip experts with zero weight for this risk category
        if weight <= 0.0:
            continue

        fuzzy_value = estim_to_fuzzy(value)
        weighted_values.append((fuzzy_value, weight))
        total_weight += weight

    if not weighted_values or total_weight <= 0.0:
        # If no expert with positive weight, use unweighted average of all experts
        fuzzy_values = [estim_to_fuzzy(val) for val in expert_values if not pd.isna(v
        if not fuzzy_values:
            return [0.0, 0.0, 0.0]

        min_val = min(val[0] for val in fuzzy_values)
        most_likely = sum(val[1] for val in fuzzy_values) / len(fuzzy_values)
        max_val = max(val[2] for val in fuzzy_values)
        return [min_val, most_likely, max_val]

    # Compute weighted aggregations
    weighted_min = sum(val[0] * weight for val, weight in weighted_values) / total_we
    weighted_most_likely = sum(val[1] * weight for val, weight in weighted_values) /
    weighted_max = sum(val[2] * weight for val, weight in weighted_values) / total_we

    return [weighted_min, weighted_most_likely, weighted_max]

def sample_triangular(fuzzy_value):
    """Sample a value from a triangular distribution given [min, most_likely, max]"""
    min_val, mode, max_val = fuzzy_value

    if min_val == max_val:
        return min_val

    # Calculate c parameter (scaled position of the mode)
    range_val = max_val - min_val
    if range_val <= 0:
        return mode

    c = (mode - min_val) / range_val

    # Sample from triangular distribution
    return triang.rvs(c, loc=min_val, scale=range_val)
```

In [9]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import beta, triang
```

In [8]:
```python
# Risk Estimation to Fuzzy Value mapping functions and risk calculation utilities
import numpy as np
from scipy.stats import beta, triang
import pandas as pd

# Hard-coded mapping from Risk_Estim to Risk_Fuzzy_Value
RISK_ESTIM_TO_FUZZY = {
    5: [0.75, 1.0, 1.0],
    4: [0.6, 0.75, 0.9],
    3: [0.4, 0.5, 0.6],
    2: [0.2, 0.3, 0.4],
    1: [0.0, 0.0, 0.2]
}

# Expert weights based on their specialization and risk categories
# Using actual categories from the risks.csv file
EXPERT_WEIGHTS = {
    'Dev': {
        'Third-Party Integration': 1.0,
        'Technical': 1.0,
        'Security': 1.0,
        'Hardware': 1.0,
        'Resource Constraints': 0.0
    },
    'QA': {
        'Third-Party Integration': 0.5,
        'Technical': 0.3,
        'Security': 1.0,
        'Hardware': 0.4,
        'Resource Constraints': 0.0
    },
    'PM': {
        'Third-Party Integration': 0.1,
        'Technical': 0.0,
        'Security': 0.3,
        'Hardware': 0.0,
        'Resource Constraints': 1.0
    }
}

def get_expert_type(column_name):
    """Extract expert type (Dev, QA, PM) from column name"""
    if column_name.startswith('Dev'):
        return 'Dev'
    elif column_name.startswith('QA'):
        return 'QA'
    elif column_name.startswith('PM'):
        return 'PM'
    else:
        return None

def estim_to_fuzzy(estim):
    """Convert a Risk_Estim value to its corresponding fuzzy triangular value"""
    try:
        estim = int(estim)
        return RISK_ESTIM_TO_FUZZY.get(estim, [0.0, 0.0, 0.0])
    except (ValueError, TypeError):
        return [0.0, 0.0, 0.0]

def aggregate_expert_opinions(expert_values, expert_columns, risk_category):
    """
    Aggregate multiple expert opinions into a single fuzzy triangular value,
    considering expert weights based on risk category
    """
```